PIC – Workflow Guide

AlphaHelion Purchasing Intelligence Cloud. One document describing how every module works end to end, as built.

Date: 7 November 2026. Code baseline: `main` at commit `f354bbe` (database migration head 0022). Written for: the product owner, the client's process owners and the implementation team. It assumes procurement and finance knowledge, not knowledge of the code.

Contents

- How to read this guide
- The modules and how they connect
- Controls that apply in every module
- Roles
- Demo tenants
- M01 Spend & Category Intelligence
- M02 MRP & Planning
- M03 Material & Master Data (MDM)
- M04 Requisition & Guided Buying
- M05 Strategic Sourcing
- M06 Supplier Lifecycle & Performance
- M07 Contract Lifecycle Management
- M08 Purchase Orders & Supplier Collaboration
- M09 Receiving, Quality & Inventory
- M10 Invoice Matching, Payables & Finance
- M11 Savings & Value Realisation
- M12 Predictive Intelligence, Agents & Copilot
- Platform & Enterprise Administration
- Appendix A. Gaps found in the code while writing this guide
- Appendix B. Older documents that are now out of date

How to read this guide

Each module section has the same eleven parts:

1. **Purpose and scope**: what the module is for, with its Blueprint (BP) reference.
2. **Roles**: who acts in the module and with which permissions (scopes).
3. **Screens**: the screen ID from the Blueprint where there is one, and the route under `/pic`.
4. **End-to-end workflow**: the main flow step by step (who, where, what the system checks, what happens next), with the main alternative and exception paths.
5. **Document states**: the state machine of the module's documents. A command that is not allowed in the current state is refused with HTTP 409.
6. **Business rules and controls**: rule sets (by key), approval bands, segregation-of-duties (SoD) rules, attribute-based access (ABAC), mandatory fields and the calculations, with formulas.
7. **Integrations and events**: messages to and from the ERP and other systems, and the events other modules react to.
8. **Scheduled jobs**.
9. **KPIs**: code, name and formula.
10. **Demo data**: which mock dataset pack and tenant shows the flow, with document numbers to try.
11. **Known limits and open items**.

Names in `code format` are the real names in the system (routes, rule-set keys, statuses, roles, event and message types), so they can be found in the screens, the API and the audit trail. Statements were checked against the code. Where an older document disagrees, this guide follows the code; the older documents that are now out of date are listed in Appendix B.

The modules and how they connect

Module	What it covers	Main documents
M01 Spend & Category Intelligence	Spend cube, classification, maverick spend, duplicate suppliers, category plans, forecast, data quality	Spend line, classification review, maverick hit, category plan
M02 MRP & Planning	Demand, MRP runs, planned orders, exceptions, Kanban and reorder point	MRP run, planned order, MRP exception
M03 Material & Master Data	New materials, duplicate check, stewardship, material changes	Material request, material change request
M04 Requisition & Guided Buying	Requisitions from catalogue, punch-out, templates, SOW forms, free text	Requisition
M05 Strategic Sourcing	RFI/RFQ/RFP, auctions, sealed bids, scoring, TCO, award	Sourcing event, bid, award decision
M06 Supplier Lifecycle & Performance	Registration, onboarding, qualification, bank and master changes, risk, scorecards	Supplier, qualification, bank change request
M07 Contract Lifecycle Management	Contracts and clauses, signature, consumption, obligations, renewal, rebates	Contract, amendment, rebate agreement
M08 Purchase Orders & Supplier Collaboration	PO creation, approval, release, transmission, confirmation, changes	Purchase order and its versions
M09 Receiving, Quality & Inventory	ASN, goods receipt, quality, returns, service entry, stock, counts, reservations	ASN, goods receipt, inspection lot, stock posting, physical inventory, reservation
M10 Invoice Matching, Payables & Finance	Capture, matching, exceptions, notes, posting, payments, bank, tax, period close	Invoice, credit and debit note, payment run, payment
M11 Savings & Value Realisation	Savings pipeline, baselines, validation by finance	Savings initiative
M12 Predictive Intelligence, Agents & Copilot	AI agents, Copilot, model registry, KPI explorer	Agent run, model version
Platform & Enterprise Administration	Tenants, organisation, users, roles, rules, workflows, audit, integration, jobs, deployment	Tenant, user, role, rule version, workflow definition

The procure-to-pay chain runs through the modules in this order (BP flows F1 to F9):



Hand-off	From	To	What passes
Planned order to requisition	M02	M04	Material, quantity, need date, plant, storage location, source of supply
Free-text line to new material	M04	M03	The line becomes a material request; the requisition waits for the material
Requisition to sourcing	M04	M05	Approved lines without a source become requirement lines of an event
Award to contract and source list	M05	M07, M02	Awarded prices become a contract draft and price validity; the source list and quota arrangement are updated
Requisition to purchase order	M04	M08	Approved lines with a source of supply; price and terms are inherited from the contract (P07)
Purchase order to supplier	M08	M06 portal	Transmission by portal, e-mail or EDI; the supplier confirms or proposes a change
ASN and receipt	M08	M09	Shipped and received quantities against the PO lines
Receipt to invoice matching	M09	M10	Accepted quantities for 3-way match; quality results for exceptions
Posted invoice to spend and savings	M10	M01, M11	Spend lines (credit and debit notes as signed lines), realised savings
Payment and returns	M10	ERP, bank	Payment files, bank acknowledgements and statements, reversals

Controls that apply in every module

- **State machines.** Every document has an explicit state machine. A command that is not allowed in the current state is refused with 409 (illegal transition) and is not partly applied.
- **Segregation of duties (SoD).** The rules below are tenant rules, checked when the second action is attempted (409 `SOD_CONFLICT`, recorded as a violation). Four of them are also checked as scope pairs when roles are defined or assigned, so one person cannot hold both sides.

Rule	First action	Second action (refused for the same person)	Severity
SOD-01	Create a supplier master	Create POs for that supplier	High
SOD-02	Change supplier bank details	Approve that supplier's payments	Critical
SOD-02a	Change supplier bank details	Post that supplier's invoices	Critical
SOD-03	Create the PO	Post its goods receipt	High
SOD-04	Create the PO	Approve invoice exceptions on it	High
SOD-05	Be the sole evaluator of an event	Award it	Medium
SOD-06	Request a material	Approve it as steward	Medium
SOD-07	Own a savings initiative	Validate its savings	High
SOD-08	Capture an invoice	Approve it	High
SOD-09	Approve an invoice	Release its payment	High
SOD-10	Submit a stock posting	Approve it	High
SOD-11	Count stock	Approve the count differences	High

Scope pairs enforced on roles: SOD-02 (`supplier:bank` with `payment:run`), SOD-03 (`po:create` / `po:release` with `gr:post`), SOD-04 (`po:create` / `po:release` with `invoice:capture` / `invoice:resolve` / `invoice:post`) and SOD-11 (`inventory:count` with `inventory:approve`).

- **Attribute-based access (ABAC).** Users carry attributes such as plants, company codes, categories, cost objects and value limits. Lists and documents outside a user's attributes are not shown (404 for a document), and commands are checked against them. Where a module does not yet apply the check to a command, its section says so.
- **Rules engine.** Thresholds, bands, tolerances and policies live in effective-dated rule sets per tenant, optionally scoped by company code, plant or category. A new version is published through `POST /v1/admin/rules` and applies from its effective date; documents record the rule version they were decided under.
- **Audit.** Every state change writes an audit event with before and after values, the actor (and on-behalf-of), the reason and the rule version. Audit events are hash-chained per tenant, so a gap or an edit is detectable.
- **Idempotency and concurrent edits.** Commands accept an `Idempotency-Key`; a replay within 24 hours returns the original response. Documents return an `ETag`; commands sent with a stale `If-Match` are refused with 412 (required in production).

- **Events and the worker.** Business events go to an outbox in the same transaction as the change and are delivered at least once, in order per tenant, by the worker (on Vercel by the cron-driven drain). Consumers are listed in each module's section 7 and in `docs/PIC_Event_Catalogue.md`.
- **Period close.** Postings dated into a closed posting period are refused (`PERIOD_CLOSED`) and the refusal is audited (`POSTING_REFUSED`).

Roles

The 22 built-in roles and their permissions (scopes). Tenants can also define custom roles from the same scopes; a custom role cannot hold platform scopes or span an SoD scope pair. All internal roles also hold `pic:read` (read what their attributes allow) and `tasks:work` (work their task inbox).

Role	Scopes beyond read and tasks	Main modules
REQUESTER	`req:create`, `ses:create`	M04, M09 (service entry)
COST_OBJECT_OWNER	none (approves in the inbox)	M04 approvals
DEPT_HEAD	none (approves in the inbox)	M04, M08 approvals
MRP_CONTROLLER	`mrp:run`, `mrp:plan`, `req:create`	M02
BUYER	`req:buyer`, `po:create`, `po:release`, `po:change`, `sourcing:manage`, `req:create`	M04, M05, M08
CATEGORY_MANAGER	`sourcing:manage`, `sourcing:evaluate`, `savings:manage`, `spend:read`	M01, M05, M11
SUPPLIER_MANAGER	`supplier:manage`, `supplier:bank`	M06
CONTRACT_MANAGER	`contract:manage`, `contract:sign`	M07
LEGAL	none (clause review and approvals in the inbox)	M07
MDM_STEWARD	`mdm:request`	M03, M01 (duplicate suppliers)
RECEIVER	`gr:post`, `inventory:post`, `inventory:count`	M09
INVENTORY_CONTROLLER	`inventory:post`, `inventory:approve`	M09 inventory
QUALITY_INSPECTOR	`quality:decide`, `sourcing:evaluate`	M09, M05 (technical evaluation)
SERVICE_OWNER	`ses:approve`	M09 (service entry)
AP_CLERK	`invoice:capture`, `invoice:resolve`, `invoice:approve`, `invoice:post`	M10
FINANCE_CONTROLLER	`payment:run`, `savings:validate`, `invoice:approve`, `spend:read`, `period:close`, `inventory:approve`	M10, M11, M09 approvals
PROCUREMENT_HEAD	`spend:read`	M01, approvals
CPO	`spend:read`, `agents:manage`	M01, M12, top approval bands
CAPEX_COMMITTEE	none (approves capex in the inbox)	M04, M08 approvals
CLINICAL_LEAD	none (clinical sign-off on awards, healthcare template)	M05
TENANT_ADMIN	`admin:rules`, `admin:audit`, `admin:integration`, `admin:ops`, `admin:workflow`, `agents:manage`	Platform

Role	Scopes beyond read and tasks	Main modules
SUPPLIER_USER (external)	`supplier:portal`, `bid:submit`, `po:confirm`, `asn:create`, `invoice:submit`, `ses:create`	Supplier portal: M05, M06, M08, M09, M10

Demo tenants

The mock dataset packs load demonstration data into these tenants (all on the production Neon database, as decided for the demo):

Tenant	What it represents	Main packs
AXN	Axion, the signature-case manufacturer (Blueprint cases such as MR-1043 and RFQ-2231)	signature case, branch cases, module cases, purchase types, manufacturing controls, AI evaluation sets, plus non-signature inventory, finance, spend and sourcing data
KAC	The first client, an auto-component manufacturer, loaded from a mock SAP extract	ERP client, contracts, S/4 delta extracts, inventory, finance, spend and sourcing
CON, ITS, LOG, HLT, GEN	Industry templates: construction, IT services, logistics, healthcare, general	one industry pack each
GMC, DBY	Tenants migrated from the legacy prototype ("Global Manufacturing Corp" and "Dummy Buyer")	legacy migration, plus reference data and organisation masters

Reference data and the organisation masters (legal entities, purchasing organisations and groups) are loaded on every tenant. All mock values are labelled MOCK and must be replaced by the client's own data before go-live.

M01 Spend & Category Intelligence

1. Purpose and scope

M01 turns posted supplier invoices (and the credit and debit notes that correct them) into a classified, governed spend cube, and runs the governance work around it: the classification steward queue, maverick-spend rules and hits, duplicate-supplier clusters, data quality, the spend forecast and the category plans (Kraljic position, levers, wave plan, budget, savings target) whose budgets are then checked when requisitions are submitted (M04). Blueprint §7 (SPD-01 to SPD-09). Code: `services/spend*.py`, APIs `api/intelligence.py` (first pass) and `api/spend_ops.py` (second pass).

2. Roles

Role	What they do in M01	Scopes (from `platform/auth.py`)
Category Manager	Reads the cube, works the steward queue, disposes maverick hits it owns, generates and edits category plans, submits plans	`spend:read`, `savings:manage`, `sourcing:manage`
Master Data Steward (`MDM_STEWARD`)	Works the classification queue, taxonomy, duplicate clusters; approves supplier-merge proposals	`mdm:request` (the steward routes accept `spend:read` or `mdm:request`)
Procurement Head	Reads spend, disposes any hit, accepts exceptions above the limit, approves or rejects category plans	`spend:read`
CPO	Same as procurement head; final escalation target for maverick hits	`spend:read`, `agents:manage`
Finance Controller	Reads spend, disposes and resolves hits owned by finance (payment-control rules)	`spend:read`, `savings:validate`
Tenant Administrator	Sees cube status, refreshes the cube, versions maverick rules, runs escalation and re-evaluation; cannot decide hits or plans	`admin:rules`, `admin:integration`
Spend Classification Agent	Classifies new lines (autonomy L3); applies results at or above the threshold, sends the rest to the queue	agent principal, `spend:read`

Segregation and four-eyes controls (all checked when the action is taken, 409 `SOD_CONFLICT`):

- A reclassification of a line at or above `fourEyesAboveMinor` (default ₹25 lakh) needs two different stewards choosing the same category.
- The proposer of a supplier merge cannot approve it.
- A category plan cannot be approved or rejected by its owner or by the person who submitted it.

None of these are rows in the `SOD-xx` table; they are coded checks in `spend.py` and `spend_workflow.py`.

ABAC read scope (`services/spend_scope.py`): a user whose attributes restrict `plants`, `company_codes`, `categories`, `cost_centres` or `supplier_groups` (parent supplier codes) sees only spend inside that scope. Every total is computed after the restriction, and decisions on lines outside the scope return 403.

3. Screens

Everything is on `/pic/spend`; the view is selected with `?tab=`.

Screen ID	Route	What it shows	Actions
SPD-01	`/pic/spend?tab=overview`	Trailing 12 months: total, addressable %, under-management %, contracted %, maverick %, classification accuracy, open reviews and hits, top movers, opportunity list, open commitments, cube freshness	Deep links to the opportunities
(SPD-01)	`?tab=compare`	Fiscal-year-to-date, YoY and QoQ comparisons, run-rate	Read
SPD-02	`?tab=categories`	Treemap by taxonomy level, drill to supplier	Read
SPD-03	`?tab=suppliers`	Supplier ranking, concentration (HHI), duplicate-supplier clusters	Read
(none)	`?tab=pareto`	Pareto / ABC classes, long tail, savings potential by lever	Read
SPD-04	`?tab=cube`	Free cube by any dimension (period, category L1–L4, supplier, parent, plant, company code, cost object, material group, currency, source, ERP system, contracted, addressable, maverick, buyer, fiscal year and quarter)	Read, CSV export
(none)	`?tab=lines`	Line drill with keyset paging	CSV export (`GET /v1/spend/export`)
SPD-05	`?tab=compliance`	Contracted vs non-contracted by category, off-contract lines, price delta (leakage)	Read
SPD-06	`?tab=maverick`	Hit worklist by status, rule versions, repeat offenders	Dispose, resolve recovery, version a rule (admin)
(none)	`?tab=classification`	Steward queue (value, age, SLA), taxonomy	Accept / reclassify, bulk reclassify, teach a keyword, add or rename taxonomy nodes
(none)	`?tab=duplicates`	Duplicate-supplier clusters	Scan, propose merge, dismiss
(none)	`?tab=quality`	Data-quality score, 14 checks, history	Run the assessment
SPD-07	`?tab=strategy`	Category plans (Kraljic, levers, waves, budget, target), plan detail and pipeline links	Generate, edit, submit, approve, reject, revise
SPD-08	`?tab=pipeline` and `/pic/spend/pipeline`	Savings pipeline (owned by M11 Savings)	See M11
SPD-09	`?tab=forecast`	Spend forecast by category and plant with an 80% band, forecast accuracy	Snapshot (category manager)

4. End-to-end workflow

1. **Spend arrives (system).** When AP posts an invoice (M10), the event `invoice.posted` is consumed by `spend-cube` (`services/spend_feed.py`). It writes one `INVOICE` spend line per invoice line, net of GST, in the period of the invoice date. Each line carries:
    - the category: the material's category, else the PO line's material group, else `UNCLASSIFIED`;
    - the supplier and plant;
    - the cost object;
    - `contracted` (the PO line referenced a contract line);
    - the price variance against the contract price or the standard price, and against the last price paid;
    - the FX rate of the invoice date and the group-currency amount.

    Credit notes (`credit_note.received`, consumer `spend-cube-credit`) become negative `CREDIT_NOTE` lines. Debit notes (`debit_note.posted`, consumer `spend-cube-debit`) become `DEBIT_NOTE` lines.

    Purchase orders and goods receipts are never spend. Open PO value is a separate "commitment" view (`GET /v1/spend/commitments`). This is how the Blueprint rule "de-duplicate PO and invoice via lineage" is met. ERP history is loaded as signed lines by the cutover loader (`source = ERP_HISTORY`).

2. **Classification (agent).** Each new invoice line goes through `classify_line`:
    - **Material lines** keep the material category (confidence 1.000).
    - **Other lines** are scored on keywords from the taxonomy, with a bonus for the supplier's qualified categories. If the agent may act (L3) and the confidence is at or above `classificationThreshold` (default 0.80), the category is applied automatically.
    - **Below the threshold**, a `ClassificationReview` (status `OPEN`) is created with the proposed category, alternatives and a due date of `reviewSlaDays` (5). When the LLM gateway is configured, a low-confidence line (below 0.65) also gets an LLM suggestion. That suggestion is added to the alternatives only; it never sets the category.

    Addressability is then derived: a line is not addressable if the supplier is `INTERCOMPANY`, the category is `IC-TRANSFER`, or the taxonomy root is not addressable.

3. **Maverick evaluation (system).** `evaluate_line` applies the maverick rules in force on the document date (rule set `MAVERICK-RULES`, versioned per rule) and writes the flags on the line. For each active rule it opens a `MaverickHit`. The hit has:
    - a number (`MAV-1001` onwards);
    - an owner role (and an owner: the category plan's owner when the role is category manager);
    - a due date from the rule's SLA;
    - "savings at risk": the price variance for `OFF_CONTRACT` and `OFF_CONTRACT_PRICE`, the excess for `OVER_INVOICED`, otherwise 0.

4. **Cube refresh (job).** A write marks its period dirty. Either the `spend-cube-refresh` job (every 5 minutes) or the next read (up to 250,000 lines) rebuilds `spend_cube_summary` and `spend_commitment_summary` for the dirty periods. Analysts read SPD-01 to SPD-05 and SPD-09 only from these summaries. An admin can rebuild everything with `POST /v1/admin/spend/refresh {"full": true}`.

5. **Steward queue (Category Manager / Data Steward / Procurement Head).** Steward screen `?tab=classification`:
    - Review: `GET /v1/spend/classification-queue`, sorted by value, due date, confidence or age.

- Decide: `POST /v1/spend/classification-reviews/{id}/decision {category, comment, reason, learn_keyword}`. Choosing the proposal gives `ACCEPTED`; choosing another category gives `RECLASSIFIED`.
- Four-eyes: a reclassification at or above ₹25 lakh is first parked as `PENDING_SECOND`. A second, different steward must pick the same category. A different category reopens the review.
- Teaching: `learn_keyword` teaches the classifier a word for the chosen category. The word must be 4–40 characters and must not already belong to another category (409 `KEYWORD_CONFLICT`).
- Bulk: `POST /v1/spend/classification-reviews/bulk` decides up to `bulkReclassifyMax` (500) reviews at once. Each row is decided on its own; refused rows are reported.
- Effect: the decision moves the line and its credit and debit notes, and the cube re-materialises those periods.

6. **Maverick hits (owner).** Hit worklist on `?tab=maverick`.

   - **Dispose:** `POST /v1/spend/maverick-hits/{id}/disposition` with one of four dispositions:

Disposition	Effect
`ACCEPTED_EXCEPTION`	Hit closes as `DISPOSED`; needs a reason code
`FALSE_POSITIVE`	Hit closes as `DISPOSED`; needs a reason code
`POLICY_BREACH`	Hit closes as `DISPOSED`
`RECOVERY`	Hit moves to `RECOVERY_OPEN`

     - Every disposition needs a comment of at least 5 characters.
     - Accepting an exception above `acceptExceptionLimitMinor` (₹1 crore) needs the procurement head or CPO.
     - Who may dispose: the owner, a user holding the owner role, or the procurement head, CPO or finance controller.

   - **Resolve a recovery:** `POST /v1/spend/maverick-hits/{id}/resolve {recovered_minor, note}` sets the hit to `RESOLVED`. The recovered amount cannot exceed the hit amount.

   - **Escalation (job `spend-sla`):** an `OPEN` hit past its due date moves to the rule's escalation role (level 1), then to the CPO (level 2). The new due date is half the SLA.

7. **Duplicate suppliers (Steward / Category Manager / Supplier Manager).** Screen `?tab=duplicates`.

   - **Scan:** `POST /v1/spend/duplicates/scan` stores clusters found by same PAN or GSTIN (score 1.0), same bank account (0.95), or normalised name similarity ≥ 0.92.
   - **Propose:** `POST .../{key}/propose-merge {survivor, note}` sets the cluster to `PROPOSED`. It opens a `SPEND_SUPPLIER_MERGE` task for an `MDM_STEWARD` (SLA 3 business days).
   - **Approve:** approval (`MERGE_APPROVED`) links the other records to the survivor as their corporate parent and marks their periods dirty, so the cube rolls them up. Nothing is merged, blocked or deleted in the master or in the ERP.
   - **Dismiss:** `POST .../{key}/dismiss` sets the cluster to `DISMISSED`.

8. **Data quality (job / analyst).** The `spend-quality` job runs daily, or a user runs `POST /v1/spend/quality/run`. It runs 14 checks and stores a 0–100 score: `GET /v1/spend/quality`, `/quality/history`.

9. **Category strategy (Category Manager → Procurement Head / CPO).** Screen `?tab=strategy`.

   1. **Generate (category manager):** `POST /v1/category-plans {fiscal_year}` creates `DRAFT` plans (number `CP-101` onwards) for every category with addressable spend in the last 12 months. Each draft includes the Kraljic position, levers, dated waves, the budget (the 12-month spend forecast, or trailing spend when the forecast is thin), the savings target and the

savings split by lever. Approved, in-approval and superseded plans are left alone.
2. **Edit:** `PATCH /v1/category-plans/{ref}`.
3. **Submit:** `POST .../submit`. The plan goes to `IN_APPROVAL` and a `CATEGORY_PLAN_APPROVAL` task opens for the procurement head; the owner is excluded.
4. **Decide:** approve through the task or `POST .../approve`, or reject with `POST .../reject` (comment required). Approval re-validates the plan; the previously approved version becomes `SUPERSEDED`.
5. **Revise:** `POST .../revise` creates a new `DRAFT` version of an approved plan. Only one open revision is allowed per category and year.

`GET .../pipeline-links` lists the sourcing events and savings initiatives in the category.

10. **Hand-off.**
    - An **approved** category plan with a budget feeds the `CATEGORY-BUDGET` check when requisitions are submitted (M04 step 5).
    - Opportunities deep-link to `/pic/sourcing/new` (M05).
    - Savings initiatives and the pipeline belong to M11.
    - Maverick hits with `RECOVERY` lead to supplier credit or debit notes (M10).

Exception paths:

- A foreign-currency line with no FX rate is excluded from every total (`fxMissingLines`) until rates are loaded.
- An admin can re-run classification and rules on unprocessed lines with `POST /v1/admin/spend/reevaluate`.

5. Document states

Object	States and transitions	Command / guard
Classification review	`OPEN` → `ACCEPTED` | `RECLASSIFIED`; `OPEN` → `PENDING_SECOND` (reclassification ≥ four-eyes value) → `RECLASSIFIED` (second steward, same category) or back to `OPEN` (different category)	`.../decision`; deciding a closed review → 409 `ILLEGAL_TRANSITION`
Maverick hit	`OPEN` → `DISPOSED`; `OPEN` → `RECOVERY_OPEN` → `RESOLVED`; `OPEN` is escalated in place (level 0 → 1 → 2)	`.../disposition`, `.../resolve`; wrong state → 409
Duplicate cluster	`OPEN` → `PROPOSED` → `MERGE_APPROVED` | `REJECTED`; `OPEN` / `REJECTED` → `PROPOSED` again or `DISMISSED`	propose-merge, task decision, dismiss; wrong state → 409
Category plan	`DRAFT` → `IN_APPROVAL` → `APPROVED` | `REJECTED`; `DRAFT` → `APPROVED` directly unless `planRequiresSubmission` is true (default false); `REJECTED` → `DRAFT` on edit; `APPROVED` → (revise creates a new `DRAFT` version) → old version `SUPERSEDED` when the new one is approved	submit / approve / reject / revise; wrong state → 409; invalid plan → 422 `INVALID_PLAN`

6. Business rules and controls

Rule set key	Controls (defaults)
`SPEND-GOVERNANCE`	`classificationThreshold` 0.80; `oneTimeVendorLimitMinor` ₹5 lakh; `groupCurrency` INR; `aggregationMinSuppliers` 4; `aggregationSavingPct` 3. Defaults in code (`spend.GOV_DEFAULTS`) when not in the rule: `fiscalYearStartMonth` 4, `fourEyesAboveMinor` ₹25 lakh, `reviewSlaDays` 5, `bulkReclassifyMax` 500, `overInvoicedTolerancePct` 2, `acceptExceptionLimitMinor` ₹1 crore, `requireReasonCode` true, `staleFeedHours` 48, `fxMaxAgeDays` 10, `paretoA` / `paretoB` 80/95, `hhiHigh` / `hhiModerate` 2500/1500, `exportMaxRows` 50,000, `planMaxSavingsPct` 15, `planRequiresSubmission` false, `planWaveMinDays` 7
`MAVERICK-RULES`	Effective-dated overrides per rule (active, severity, owner role, SLA days, escalation role). A new version is created with `POST /v1/spend/maverick-rules/{code}/versions` (`admin:rules` , reason required, one version per effective date)
`CATEGORY-BUDGET`	`mode` SOFT, `warnAtPct` 90 (used at requisition submit, see M04)

Default maverick rules (`spend.MAVERICK_RULES` , severity / owner role):

Rule	Severity	Owner role
`PO_AFTER_INVOICE` (PO created after the invoice date, in the tenant time zone)	High	CATEGORY_MANAGER
`SPLIT_PO` (the PO carries the `SPLIT_PATTERN` risk factor of M08)	High	PROCUREMENT_HEAD
`NON_CATALOGUE` (free-text PO line in a material group with an active catalogue)	Medium	CATEGORY_MANAGER
`OFF_CONTRACT` (a contract valid on the PO date covers the material and plant, but the PO line does not use it)	High	CATEGORY_MANAGER
`OFF_CONTRACT_PRICE` (the PO line has a price deviation)	Medium	CATEGORY_MANAGER
`ONE_TIME_VENDOR` (addressable line above the one-time-vendor limit, supplier not qualified for the category)	High	PROCUREMENT_HEAD
`NO_GOODS_RECEIPT` (invoiced quantity above received quantity, or service invoiced above the approved SES value)	Critical	FINANCE_CONTROLLER
`PO_NOT_APPROVED` (PO not released)	Critical	FINANCE_CONTROLLER
`OVER_INVOICED` (invoiced above PO net value + 2%)	Critical	FINANCE_CONTROLLER

Hit SLA and escalation defaults:

- SLA days by severity: Critical 3, High 5, Medium 10, Low 15.
- First escalation role: category manager → procurement head; procurement head → CPO; finance controller → CPO.
- The second escalation always goes to the CPO.
- Each rule's values can be overridden by a `MAVERICK-RULES` version.

Calculations:

- **Spend** = posted invoices + debit notes − credit notes, net of tax, in the group currency (FX rate of the document date).
- **Addressable %** = addressable spend / total spend.
- **Contract coverage %** = contracted addressable spend / addressable spend.
- **Maverick %** = addressable spend with any maverick flag / addressable spend.
- **Under management %** = addressable spend that is contracted, or that is non-maverick from a Qualified/Preferred/Conditional supplier in a category with an approved category plan, / addressable spend.
- **Classification accuracy** = (auto-classified + steward-accepted) / (auto-classified + steward decisions).
- **HHI** = sum of squared supplier shares (%), per category.
- **Savings potential** (indicative, `GET /v1/spend/savings-potential`). It is never booked as realised.
  - `CONSOLIDATION` = spend outside the top 3 parent suppliers × 3%.
  - `CONTRACT_COVERAGE` = non-contracted addressable spend × 4% (`contractSavingPct`).
  - `PRICE_RECOVERY` = price paid above contract or standard price.
- **Kraljic** (plan generation):
  - Profit impact = √(category spend / largest category spend) × 10.
  - Supply risk (0–10) = criticality + concentration (HHI) + single qualified source + supplier risk tier + the twin's late-delivery probability.
  - Quadrant: impact ≥ 5 is `STRATEGIC` (risk ≥ 4) or `LEVERAGE` ; impact < 5 is `BOTTLENECK` (risk ≥ 4) or `NON_CRITICAL` .
  - Savings target = budget × 3% / 6% / 1% / 4% (by quadrant), capped at 15% of addressable spend.
  - Plan validation also checks: levers come from the catalogue, the split by lever adds up to the target, waves have dates of at least 7 days, inside the fiscal year and the year after.
- **Data-quality score** = 100 − Σ penalties. Each penalty = check weight × affected share / failure share, capped at the weight. The 14 checks are: `UNCLASSIFIED` , `CATEGORY_NOT_IN_TAXONOMY` , `SUPPLIER_UNKNOWN` , `FX_RATE_MISSING` , `FX_RATE_STALE` , `DUPLICATE_LINES` , `NON_POSITIVE_INVOICE_LINES` , `NO_LINEAGE` , `INTERCOMPANY_ADDRESSABLE` , `FUTURE_PERIOD` , `AMOUNT_OUTLIER` , `FEED_STALE` , `PERIOD_GAP` , `CUBE_TIE_OUT` .
- **Spend forecast** (`services/spend_forecast.py`):
  - The model is chosen per series by a rolling back-test (lowest MAPE) among `HOLT` (needs 6 months), `SEASONAL` (13) and `HOLT_WINTERS` (24).
  - It returns an 80% band. Series with fewer than 3 months of spend are not forecast.

Idempotency: the bulk reclassification and `POST /v1/spend/quality/run` accept an `Idempotency-Key` . Concurrent decisions on the same review, hit or plan are serialised by row locks; the second one gets 409.

7. Integrations and events

Direction	What
Inbound (consumed)	`invoice.posted` → `spend-cube` ; `credit_note.received` → `spend-cube-credit` ; `debit_note.posted` → `spend-cube-debit`
Inbound (load)	ERP spend history through the cutover loader (`tools/erp_load`), lines with `source = ERP_HISTORY` and `source_system` per ERP
Outbound	`spend.cube.refreshed` (payload `mode` , `periods` , `lines` ; stream only, no internal consumer) when a refresh is recorded. Otherwise M01 decisions emit no outbox events. Every decision writes an audit event (`CLASSIFIED` , `CLASSIFICATION_PROPOSED` , `KEYWORD_LEARNED` , `DISPOSED` , `ESCALATED` , `RECOVERY_RESOLVED` , `MERGE_PROPOSED` / `APPROVED` / `REJECTED` , `GENERATED` , `SUBMITTED` , `APPROVED` , `SUPERSEDED` , `REVISED` , …)

Direction	What
Workflow tasks	`SPEND_SUPPLIER_MERGE` (MDM steward), `CATEGORY_PLAN_APPROVAL` (procurement head); both appear in the approval inbox `/pic/tasks`

8. Scheduled jobs

Job key	Schedule (tenant time, IST default)	What it does
`spend-cube-refresh`	every 5 minutes	Rebuilds the cube periods whose lines changed; the whole cube when it was never built
`spend-sla`	hourly at :15	Escalates overdue `OPEN` maverick hits (max two levels); reports steward-queue ageing
`spend-quality`	daily 05:00	Runs the 14 data-quality checks and stores the score history
`spend-forecast-snapshot`	05:30 on the 2nd of each month	Stores the 6-month forecast as issued, so its real error can be measured later (`GET /v1/spend/forecast/accuracy`)

9. KPIs

Code	Name	Formula
`M01.SPEND_UNDER_MGMT`	Spend under management %	Under-management formula in §6 (target 75, warn 60)
`M01.CONTRACT_COVERAGE`	Contract coverage %	Contracted addressable / addressable (70 / 55)
`M01.MAVERICK_PCT`	Maverick spend %	Flagged addressable / addressable (8 / 12, lower is better)
`M01.CLASSIFICATION_ACCURACY`	Classification accuracy %	(auto + accepted) / (auto + steward decisions) (90 / 80)
`M01.SAVINGS_REALISATION_RATE`	Savings realisation rate %	Realised / negotiated savings (savings ledger)
`M01.TAIL_SPEND_PCT`	Tail spend %	Spend with class C parent suppliers / addressable
`M01.SUPPLIERS_PER_CATEGORY`	Suppliers per category	Parent suppliers with addressable spend / categories with addressable spend
`M01.SPEND_FORECAST_MAPE`	Spend forecast error %	Mean |issued forecast − actual| / actual
`M01.SAVINGS_PIPELINE`	Savings pipeline	Negotiated savings not yet realised
`M01.QUEUE_OVERDUE`	Steward queue past SLA	Open reviews past due
`M01.MAVERICK_OVERDUE`	Maverick hits past SLA	Open hits past due
`M01.DATA_QUALITY_SCORE`	Spend data-quality score	100 − penalties

10. Demo data

- **Tenant AXN (signature case, `seed/intelligence.py`)**:
  - ERP spend history from October 2024 to September 2026, with auto-classified lines, steward reviews and maverick hits (two disposed: `POLICY_BREACH` and `RECOVERY`).
  - Category plans for FY 2027-28. The `ALU-DIECAST` plan is approved by the procurement head; open it with `GET /v1/category-plans/ALU-DIECAST` (the reference may be a category).
- **Pack D16 `spend-ops` (tenants KAC and AXN, labelled MOCK)**:
  - Two fiscal years of spend (April 2025 – October 2026) on several ERPs and currencies, with credit-note netting.
  - Steward decisions in every state, including four-eyes pending, confirmed and disagreed.
  - Maverick hits in every state, with recoveries and escalations.
  - Duplicate clusters (proposed, approved, dismissed); category plans for KAC's top 8 categories in draft, in approval, approved, rejected and revised.
  - An effective-dated maverick rule version, forecast snapshots April–September 2026, and deliberate data-quality defects (DQ1–DQ8). Invoice references start `MOCK-SP/`.
- **Try:**
  - Open `/pic/spend?tab=maverick` on KAC, filter `OPEN`, and dispose a hit (numbers `MAV-1001` onwards).
  - Open `/pic/spend?tab=classification` and reclassify a line above ₹25 lakh to see `PENDING_SECOND`.

11. Known limits and open items

- Classification is keyword-based with steward learning. The optional LLM only proposes alternatives. There is no ML retraining loop.
- The cube runs on PostgreSQL summaries, not ClickHouse (a deviation from TD AD-04). Measured up to 2.5 M lines locally; 10 M and 50 M lines are not yet measured.
- Duplicate-supplier merges only set the corporate-parent link; the ERP merge stays a manual master-data task.
- `SPLIT_PO` relies on the PO risk factor, so it cannot be re-detected on ERP history lines. Goods-issue consumption is not a spend source.
- Invoice reversal after posting has no process yet, so no credit lines would be produced for it.
- The rule values above are developer defaults; the client must confirm them.
- The data-quality docstring says "twelve checks"; the code runs 14.

M02 MRP & Planning

1. Purpose and scope

M02 turns demand into planned orders and then into purchase requisitions. Demand comes from forecasts, customer orders, reservations and project demand (all held as `demand_element` rows). Planning runs:

- in daily buckets (MRP type `PD`), weekly (`PW`) or monthly (`PM`);
- by reorder point (`VB`);
- by Kanban card (`KB`, not netted by MRP).

The planner reviews the run and its exceptions, firms orders that must not change, and converts them into requisitions. Those requisitions get M04 approval and land in the buyer workbench. Tenants that run MRP in their ERP can import the ERP's planned orders instead (TD AD-05). Blueprint §8 (MRP-01 to MRP-04), flow F1 (`pic.mrp-to-pr`). Code: `services/planning.py`, `services/source.py`, `services/forecasting.py`, `services/ind_manufacturing.py` (Kanban, reorder point), `services/erp_import.py`.

2. Roles

Role	What they do in M02	Scopes
MRP Controller / Planner	Starts runs, reviews exceptions, firms and converts planned orders, overrides and releases forecasts, maintains reorder points, MRP types and Kanban cards, imports ERP planned orders	`mrp:run`, `mrp:plan`, `req:create`
Buyer	Receives the converted requisitions in the buyer workbench (M04); is notified when an order is firmed	`req:buyer`, `po:create`, `sourcing:manage`
Approvers (cost-object owner … finance controller)	Approve MRP-origin requisitions by the same DMN-01 route as manual ones	tasks
Tenant Administrator	Configures `MRP-SCHEDULE`, imports ERP planned orders, runs inbound ERP syncs	`admin:rules`, `admin:integration`
MRP Exception Agent (L2)	Ranks the exceptions of a live run (drafts only)	agent
Demand Sensing Agent (L1)	Recommends a forecast override when bias exceeds the threshold	agent

Controls:

- **Who can create MRP-origin requisitions:** only the MRP controller (or an agent principal) can create a requisition with origin `MRP` (403, principle P02).
- **Origin is fixed:** the origin of a requisition is never overwritten (enforced by a database trigger).
- **Four-eyes:** the MRP controller who converts is the requisition's requester, so DMN-01 approval excludes them.
- **No plant check on runs:** MRP runs have no ABAC plant check (there is no `MRP_RUN` action in the ABAC rule).

3. Screens

Screen ID	Route	What it shows	Actions
MRP-01	`/pic/mrp` (card "Run console")	Plant, run mode (net change / regenerative / simulation), material list; list of runs	Start run
MRP-03	`/pic/mrp` (card "Planned orders")	Planned orders by status with source of supply	Firm, Convert
MRP-02	`/pic/mrp/[ref]`	Run header, netting trace per material (gross − on-hand − open PO − in-transit − firm receipts + safety = net), exceptions by value at risk (paged), planned orders of the run	Firm, Convert
MRP-02 (stock/requirements list)	`/pic/inventory/[material]`, tab "Requirements profile" (`GET /v1/stock/materials/{material}`)	On-hand, open POs, ASNs in transit, firm planned receipts, demand incl. reservations, projected balance per bucket, below-safety and shortage flags	Read
MRP-04	`/pic/mrp/forecast`	Statistical forecast, override, actuals, bias and MAPE per material/plant; Demand Sensing recommendations	Generate, Override (reason), Release
(manufacturing controls)	`/pic/industry` (manufacturing tab)	Kanban cards, reorder-point parameters, MRP type per material	Create card, Signal empty, Refill, Suspend/Resume, Set parameters, Set MRP type
(integration)	`/pic/admin` (connectors)	ERP planned-order import	Upload JSON/CSV batch

4. End-to-end workflow

1. **Demand is in place (system / planner).** `demand_element` rows come from four sources:
    - customer orders and project demand loaded from the ERP;
    - reservations (M09);
    - released forecasts (step 9 below);
    - ERP delta syncs.

    Each change stamps `changed_at`, which is what drives net-change planning.

2. **Start the run (MRP controller, MRP-01).** `POST /v1/mrp/runs {plant, mode, materials?, run_date?}` (`mrp:run`, `Idempotency-Key` supported). The run gets a number `MR-…` (first is `MR-1043`), status `RUNNING`, and starts process instance `pic.mrp-to-pr` (steps F1-02/03).

    Alternative: the `mrp-schedule` job starts runs for the plants listed in rule `MRP-SCHEDULE` (see §8).

3. **Scope (system).** Only `ACTIVE` materials with MRP type `PD`, `PW`, `PM` or `VB` are planned; `KB` and `ND` are skipped. The scope depends on the mode:

Mode	Materials in scope
`NET_CHANGE` (default)	Only materials whose demand elements changed since the plant's last non-simulation, non-import run. On the first run, all materials.
`REGENERATIVE`	All planned materials, or the listed ones.
`SIMULATION`	The same computation as a live run, but nothing is written except the run's netting trace.

4. **Netting per material (system, F1-04)**:
    - **Supply:** on-hand `UNRESTRICTED` + `CONSIGNMENT` stock, plus open PO quantity not yet shipped (POs `RELEASED` / `TRANSMITTED` / `CONFIRMED` / `PARTIALLY_RECEIVED`), plus ASN quantity in transit, plus `FIRMED` planned orders.
    - **Buckets:** demand and receipts are put in buckets (a day for PD; Monday–Sunday for PW; the calendar month for PM).
    - **Projection:** projected stock is carried bucket by bucket. When it falls below safety stock, net = safety − projected. The quantity is lot-sized (see §6), and a proposal is made with need date = bucket date and release date = need date − (planned delivery days + GR processing days) on the plant's working-day calendar.
    - `VB` **materials** use reorder-point logic instead (see §6).
5. **Documents and exceptions (system, F1-05/06).** For a live run:
    - Unfirmed (`CREATED`) planned orders of earlier runs for the materials in scope are set to `DELETED`.
    - New planned orders `PLO-…` (first `PLO-55821`) are written with status `CREATED` and a source determination. Source determination order:

Order	Source
1	Active contract line
2	Quota arrangement
3	Source list
4	Info record
5	`NONE` (buyer queue)

      Blocked suppliers, and suppliers not Qualified/Preferred for the category, are excluded.
    - Exceptions `OPEN` are written with a value at risk (standard price × net) and a recommended action.
    - The run ends `COMPLETED`, waits at F1-07, and emits `mrp.run.completed`, which notifies the plant's MRP controllers.

   For a simulation, the run ends `SIMULATED` (F1-05 → end) and no documents are created.
6. **Review (MRP controller, MRP-02).** On `/pic/mrp/[ref]` the controller reads:
    - the netting trace (`GET /v1/mrp/runs/{ref}/results`);
    - the exceptions (`GET /v1/mrp/runs/{ref}/exceptions`, by value at risk);

- the planned orders (`GET /v1/mrp/runs/{ref}/planned-orders`).

Large runs return a summary header and paged detail.

7. **Firm (MRP controller).** `POST /v1/planned-orders/{ref}/firm` (`mrp:plan`) moves the order from `CREATED` to `FIRMED`. A firmed order is never changed or deleted by later runs. Instead, later runs raise `RESCHEDULE_IN`, `RESCHEDULE_OUT` or `CANCEL` messages on it. The event `planned_order.firmed` notifies buyers of the plant.

8. **Convert (MRP controller, F1-08).** `POST /v1/planned-orders/{ref}/convert` (`mrp:plan`, idempotent) is allowed from `CREATED` or `FIRMED`.

    1. Source determination runs again.
    2. A requisition `REQ-…` is created with origin `MRP`, purpose `MRP_REPLENISHMENT`, and one `MATERIAL` / `STOCK` line. The line inherits material, quantity, UoM, need date, plant, storage location and source; its price comes from the source allocation, else the standard price.
    3. The planned order becomes `CONVERTED` and is linked to the requisition (`converted_from`).
    4. The requisition is **submitted immediately** and follows the normal M04 controls: mandatory fields, DMN-01 route. The budget check is "not applicable" for inventory lines.
    5. When the requisition is approved, F1-09 records it in the buyer workbench. The run instance ends at F1-10 once no planned order of the run is `CREATED` or `FIRMED`.

9. **Forecast review (MRP controller, MRP-04).**

    - **Generate:** `POST /v1/forecasts/generate` refreshes actuals from goods-issue movements (201, 261, 221, 601) or the loaded ERP consumption history. It computes a 3-month statistical forecast: exponential smoothing with α from `DEMAND-SENSING` (0.3), or the registry's ML model where it back-tests better. The Demand Sensing Agent then proposes overrides.
    - **Override:** `POST /v1/forecasts/{id}/override {qty, reason, release}`. Only the MRP controller can override; the reason must be at least 5 characters, and only current and future periods can be changed.
    - **Release:** `POST /v1/forecasts/{id}/release` writes the effective quantity as a `FORECAST` demand element (`FC-YYYY-MM`, dated the 15th), so the next net-change run plans it. Statistical forecasts never reach MRP without a release.

10. **Alternative: ERP-run MRP (integration).** The integration posts the ERP's planned orders:

    - `POST /v1/integration/planned-orders/import` (or `/import/csv`) with `{system, batchRef, rows}`, scope `mrp:run` or `admin:integration`; at most 5,000 rows per batch.
    - Each row is validated (active stock material, known plant within the caller's plants, qty > 0, dates).
    - Accepted rows become planned orders on a run with mode `ERP_IMPORT`, one per plant. Firm and convert then work exactly as above.
    - Resending the same batch reference returns the original result; the same reference with different rows is refused.
    - Rejected rows trigger a notification (`integration.batch.imported`).

11. **Alternative: Kanban (`KB`).**

    1. The controller creates cards: `POST /v1/kanban/cards {material, plant, binQty, supplier?}` (status `FULL`).
    2. An empty bin is signalled with `POST /v1/kanban/cards/{ref}/signal` : the card goes `EMPTY`, then `REPLENISHING`. This raises a requisition for one bin: origin `MRP`, purpose `KANBAN_REPLENISHMENT`, need date = today + lead time (at least 2 days). The requisition is submitted for approval at once.
    3. `.../refill` sets the card back to `FULL` and counts a cycle. Refill is manual; goods receipt does not refill a card.
    4. `.../suspend` and `.../resume` need a reason.
    5. Card actions take `If-Match` (row version) and `Idempotency-Key`. The rule `KANBAN` must exist (manufacturing template), otherwise `CONTROL_NOT_ENABLED`.

12. **Hand-off.** Approved MRP requisitions appear in the buyer workbench (`GET /v1/buyer-workbench?origin=MRP` , REQ-04). The buyer then creates the PO (M08) or bundles the lines into a sourcing event (M05).

5. Document states

Object	States and transitions	Command / guard
MRP run	`RUNNING` → `COMPLETED` (live, ERP import) | `SIMULATED`	`POST /v1/mrp/runs` ; invalid mode → 422 `INVALID_MODE`
Planned order	`CREATED` → `FIRMED` → `CONVERTED` ; `CREATED` → `CONVERTED` ; `CREATED` → `DELETED` (replaced by a later run for the same material)	`/firm` only from `CREATED` ; `/convert` only from `CREATED` / `FIRMED` ; otherwise 409 `ILLEGAL_TRANSITION`
MRP exception	`OPEN` (no command closes it)	none
Forecast period	statistical → overridden (reason) → released as demand element	override only for current/future periods (409 `PERIOD_CLOSED`)
Kanban card	`FULL` → `EMPTY` (signal) → `REPLENISHING` (requisition raised) → `FULL` (refill); `FULL` ↔ `SUSPENDED`	wrong state → 409

6. Business rules and controls

Netting formula (summary per material, shown in the trace):

```
Net = max(0, Gross − (On-hand + Open PO + In-transit + Firm planned receipts) + Safety stock)
```

Signature case `MR-1043` , material DC-330: 12,000 − (1,400 + 1,200 + 600 + 0) + 200 = **9,000**.

- **Supply definition:** on-hand counts `UNRESTRICTED` plus `CONSIGNMENT` stock. This was a product decision of 2 Nov 2026 and extends TD §9.1. Quality, blocked and subcontractor stock are not counted.
- **Lot sizing** (`material.lot_size_policy` , then minimum lot, then rounding up to the pack size):
  - `FIXED` : [net / lot] × lot.
  - `EOQ` : max(net, √(2 × annual demand × ordering cost / (standard price × holding rate))).
  - `MINMAX` : max(net, max lot − projected).
  - `PERIOD` : see §11.
  - `EXACT` (default): net.
- **Reorder point (`VB`)**:
  - Arrival = run date + lead time (working days).

- Projected = on-hand + open PO + in-transit + firm receipts − demand due by arrival.
- If projected ≤ reorder point: order up to the maximum stock (or reorder point + safety stock when no maximum is kept). One proposal: need = arrival, release = run date.
- No parameters → exception `NO_REORDER_PARAMS`.
- Parameters are set with `PUT /v1/planning/parameters`; the maximum must be ≥ the reorder point.

- **Exception codes:**

Code	Meaning
`BELOW_SAFETY`	On-hand below safety stock
`PAST_DUE`	Release date before the run date
`RESCHEDULE_IN`	A firmed order is needed earlier
`RESCHEDULE_OUT`	A firmed order is needed later
`CANCEL`	A firmed order is not needed
`NO_REORDER_PARAMS`	A `VB` material has no reorder point at the plant

- **MRP type** (`PUT /v1/materials/{number}/mrp-type`): `PD`, `PW`, `PM`, `VB`, `KB`, `ND`. It is audited.
- **Conversion inherits** material, quantity, date, plant, storage location and source of supply (Blueprint §8.4). The requisition then follows M04 rules: P01 purpose code, `MANDATORY-FIELDS`, DMN-01 (for example, ₹36.9 lakh goes through the 4-step chain).
- **Rule sets:**

Key	Controls (defaults)
`MRP-SCHEDULE`	`plants: []`, `mode: NET_CHANGE`, `overlapMinutes: 120`
`DEMAND-SENSING`	`alpha` 0.3, `biasThreshold` 0.15, `horizon` 3
`KANBAN`	Manufacturing template switch; it carries `purposeCode` and `autoRequisition`, which the code does not read (see §11)

- **Idempotency:** `POST /v1/mrp/runs`, `/planned-orders/{ref}/convert`, the planning-parameter, MRP-type and Kanban commands take `Idempotency-Key`. Planned orders are row-locked on firm and convert.
- **Scale:**
  - Up to 2,000 materials are loaded by material id, more by plant.
  - Runs with more than 5,000 orders write the run→order link lazily (on firm or convert).
  - The TD target is 50,000 materials in less than 10 minutes.

7. Integrations and events

Event / message	Direction	Consumer / effect
`mrp.run.completed`	emitted (live run, ERP import)	`notify-mrp-run`: in-app notification to the plant's MRP controllers
`planned_order.firmed`	emitted	`notify-planned-order-firmed`: notification to the plant's buyers
`kanban.signalled`	emitted	`kanban-replenishment`: raises the replenishment requisition when the signal did not already do it
`integration.batch.imported`	emitted by the ERP import	`notify-batch-imported`: MRP controllers and admins are told about rejected rows
`requisition.submitted` / `requisition.approved`	from the converted requisition	see M04
ERP planned orders	inbound	`POST /v1/integration/planned-orders/import[/csv]`; or the inbound sync kind `PLANNED_ORDERS` (`POST /v1/admin/erp-sync/PLANNED_ORDERS/run`), which uses the same import
ERP stock, open POs, demand	inbound	ERP sync kinds `STOCK`, `OPEN_POS` (on demand, `admin:integration`)
Requisition to ERP	not sent	A `PURCHASE_REQUISITION` mapping exists in `sap_mapping.py`, but no service queues it: MRP requisitions stay in PIC until the PO

8. Scheduled jobs

Job key	Schedule	What it does
`mrp-schedule`	daily 02:00	Runs MRP for each plant (or `{plant, mode, materials}`) listed in `MRP-SCHEDULE`. Skips a plant with a run still `RUNNING` inside `overlapMinutes`. Unknown plants are reported. Does nothing while the list is empty (the default). Supports dry run

There is no scheduled job for forecast generation or for inbound ERP syncs (see §11).

9. KPIs

Code	Name	Formula
`M02.FORECAST_MAPE`	Forecast accuracy (MAPE) %	Mean |effective forecast − actual| / actual over closed periods (target 20, warn 30)
`M02.EXCEPTION_BACKLOG`	Open MRP exceptions	Count of `mrp_exception` with status `OPEN` (20 / 50)
`M02.PO_CONVERSION_LEAD`	Planned-order conversion lead time (days)	Mean (requisition created − planned order created) for converted orders (3 / 7)
`M02.STOCKOUTS`	Stock-out incidents	Stock materials with open demand and no unrestricted or consignment stock (0 / 3)

Code	Name	Formula
`M02.DAYS_OF_COVER`	Inventory days of cover	Unrestricted stock value / (annual demand value / 365) (45 / 75)

10. Demo data

- **Signature case, tenant AXN (`seed/axion.py`), user `mrp.controller`, plant P100:**
  - Run `MR-1043` (net change, 6 Oct 2026). DC-330 nets to 9,000 and gives `PLO-55821`; DC-331 raises `BELOW_SAFETY`.
  - `PLO-55821` is firmed and converted to `REQ-2311` (source: info record), then approved through the 4-step DMN-01 chain.
  - Open `/pic/mrp/MR-1043`.
- **D1 operating cases (`seed/datasets/branches.py`, AXN, April 2027):**
  - Case 11: a `SIMULATION` run (DC-345, no orders).
  - Case 12: `PAST_DUE` on SO-90412, firm and convert.
  - Case 13: the next run deletes unfirmed orders and raises `RESCHEDULE_OUT`.
  - Case 14: `RESCHEDULE_IN` from the urgent order SO-90437.
- **D8 `axn-manufacturing-controls` (AXN, July 2027):**
  - Three new C-part materials, created through F9.
  - A washer on Kanban: three cards of 5,000 (`KB-…`); two bins signalled, giving two approved requisitions.
  - A cable tie on reorder point 400 / maximum 2,000: a regenerative run proposes an order up to the maximum, which is firmed and converted.
  - A gasket on `PW` weekly buckets: one planned order per week.
- **D18 `erp_delta_kac` (tenant KAC):** a post-cutover delta sync of all 8 kinds, including `PLANNED_ORDERS`, which arrive as `ERP_IMPORT` runs.

11. Known limits and open items

- **MRP exceptions are never closed.** There is no resolve or close command, so `M02.EXCEPTION_BACKLOG` counts exceptions of every past run.
- **No cancel or unfirm command for planned orders.** A `CANCEL` message can only be acted on by converting the order or leaving it.
- **Net-change scope is driven only by demand-element changes.** Stock or PO changes alone do not bring a material into a net-change run. The MRP-01 screen label "materials with changed demand or supply" overstates this.
- **The `PERIOD` lot-size policy currently falls back to lot-for-lot.** The run never passes a period quantity. Use MRP types `PW` / `PM` for period buckets.
- **The material change request (M03) accepts MRP types `PD`, `ND`, `VB`, `VM`.** It cannot set `PW`, `PM` or `KB`, and `VM` is not planned by the engine. Use `PUT /v1/materials/{number}/mrp-type` instead.
- **No ABAC plant check on MRP run, firm or convert.** Any holder of `mrp:run` / `mrp:plan` can plan any plant. The ERP import does check the caller's plants.
- **The `KANBAN` rule flags `purposeCode` and `autoRequisition` are not read.** The purpose is fixed and the requisition is always raised automatically.
- **Inbound ERP syncs (`services/erp_sync.py`) are not registered as scheduled jobs**, although the module docstring says the scheduler registers them. They run on demand through `/v1/admin/erp-sync/{kind}/run`.
- **Older docs are outdated.** `docs/PIC_Process_Flows.md` (27 Oct) still says "only MRP type PD", "daily buckets", and lists only three exception codes. `PIC_Pending_Workflows.md` lists scheduled MRP, Kanban/reorder point and weekly buckets as not built. `PIC_Module_Gap_Analysis.md` lists the ERP planned-order import as open. All of these are now in the code.

M03 Material & Master Data (MDM)

1. Purpose and scope

M03 governs the creation and change of material masters so that what is bought again is bought against one clean golden record. It covers:

- new material requests (direct, or promoted from a free-text requisition line);
- AI duplicate scoring with a block or warn gateway;
- parallel approval by four steward domains;
- creation of the material and its synchronisation to the ERP;
- change requests on existing materials, with impact analysis, field-level ownership by material type and version history.

Supplier-master governance (registration, bank, tax and address changes, Blueprint MDM-04) is described with the supplier lifecycle (M06). Blueprint §9 (MDM-01 to MDM-04), flow F9 (`pic.material-request`). Code: `services/mdm.py` , `services/material_changes.py` , APIs in `api/plan_request.py` and `api/wave1_c.py` .

2. Roles

Role	What they do in M03	Scopes
Requester / Buyer / MRP Controller	Raise a new material request (`POST /v1/materials/requests` needs `req:create`); a requester promotes their own free-text line; merge their own blocked request into the proposed existing material	`req:create`
Master Data Steward (`MDM_STEWARD`) with attribute `steward_domains` (PURCHASING, QUALITY, FINANCE, PLANT)	Approve or reject the domain task of a request or change request; override a duplicate block (with reason); merge a request in stewardship; request material changes in their own domain	`mdm:request`
ERP (system)	Creates or updates the material, confirms with a document number	integration
Material Dedup Agent (L2)	Scores duplicates and drafts the candidate list	agent

SoD:

- **SOD-06** (`MATERIAL_REQUEST` vs `MATERIAL_STEWARD_APPROVE` , Medium): whoever requested a material, or a change to it, may not approve it as steward. It is also enforced on the duplicate override.
- **Four-eyes:** the requester is excluded from every steward task.
- **Change requests:** the domain approvals must come from different people (`distinctApprovers`).

3. Screens

Screen ID	Route	What it shows	Actions
MDM-01	`/pic/materials`	Materials list, material requests by status, change requests in approval	New material request (guided form)

Screen ID	Route	What it shows	Actions
MDM-02 / MDM-03	`/pic/materials/[ref]` (`ref` = `MRQ-…`)	Request, duplicate check (score, parts, up to 5 candidates incl. in-flight requests), steward tasks with domain and due date, linked PR line	Approve / Reject (steward), Steward override (blocked), Merge into candidate
(labelled "MDM-04" in the UI)	`/pic/materials/[ref]` (`ref` = material number or `MCR-…`)	Material master, version history, field ownership for the type, change request with before/after, impact and approvals	Request change (with preview), Approve / Reject, Withdraw
REQ-03	`/pic/tasks`	Steward tasks (`MDM_STEWARD`, `MATERIAL_CHANGE`) next to other approvals	Approve / Reject (bulk)

Promoting a free-text PR line has an API (`POST /v1/requisitions/{ref}/lines/{n}/promote-material`) but no button in the UI. The `reusable` checkbox at requisition creation is the UI path.

4. End-to-end workflow

1. **Need a reusable item (requester / buyer, F9-01/02).** The request is raised in one of three ways:
   - `POST /v1/materials/requests` with `{description, type (STOCK / NON_STOCK / SERVICE), material_group, category, base_uom, valuation_class?, hsn_code?, mpn?, attributes?}`;
   - automatically from a requisition line created as free text with `reusable: true` (M04 step 2). The PR does not wait for it;
   - by promoting a live free-text or service line: `POST /v1/requisitions/{ref}/lines/{n}/promote-material`. Only the requester can do this, only for lines without a material, catalogue item or contract, and only once per line (409 `ALREADY_PROMOTED`).

   The request gets `MRQ-…` (first `MRQ-7731`). The requester is recorded for SOD-06, and process `pic.material-request` starts.

2. **Duplicate check (system, F9-03).** Rule `MDM-DUP` is resolved for the category. Every `ACTIVE` material, and every other request still in flight without a material, is scored:

   ```
   score = 0.5 × description + 0.3 × MPN + 0.1 × attributes + 0.1 × classification
   ```

   - description = max(text sequence ratio, token overlap);
   - MPN = 1 if the manufacturer part numbers are equal;
   - attributes = share of equal attribute values;
   - classification = 1 if both material group and category are equal.

   Candidates scoring ≥ warnAt − 0.25 are listed (top 5), and the Material Dedup Agent logs the run.

3. **Gateway (F9-04):**

Top score	Status	Next
≥ 0.92 (`blockAt`)	`DUPLICATE_BLOCKED`	The process waits. Either the requester (or a steward) merges into an existing material (`POST .../merge {material}`, which must be an `ACTIVE` listed candidate) and the status becomes `MERGED`, or a steward overrides with a reason (`POST .../override`, SOD-06 checked), which goes to stewardship

Top score	Status	Next
≥ 0.75 (`warnAt`)	`IN_STEWARDSHIP`	Shown as a warning; the stewards see the candidates
< 0.75	`IN_STEWARDSHIP`	Normal path

4. **Stewardship (stewards, F9-05).** Four parallel `MDM_STEWARD` tasks are opened, one per domain in `MDM-DUP.domains` (PURCHASING, QUALITY, FINANCE, PLANT).
    - Each task can only be worked by a steward whose `steward_domains` include that domain. The requester is excluded and SOD-06 is checked.
    - Decide with `POST /v1/materials/requests/{ref}/decision {APPROVE|REJECT, comment}`, or in the inbox. A reject needs a comment.
    - One rejection cancels the other tasks and the request becomes `REJECTED`.
    - While a request is in stewardship, a steward can still merge it into a listed candidate (status `MERGED`).
5. **Approval (system).** When all four domains approve:
    1. The request becomes `APPROVED`.
    2. A material `M-…` (first `M-400001`) is created `ACTIVE` with `erp_synced = false`, and version 1 is recorded in `material_version`.
    3. The event `material_request.approved` notifies the requester.
    4. A `MATERIAL_MASTER` message (idempotency key `MAT|<number>`) is queued to the tenant's ERP target (rule `ERP-TARGET`, default SAP S/4HANA: OData `API_PRODUCT_SRV` / IDoc `MATMAS05`).
6. **ERP confirmation (F9-06).** When the ERP posts the message, the material is marked `erp_synced`, the request becomes `SYNCED_TO_ERP`, and the ERP document is recorded.
7. **Link (F9-07/08).** If the request came from a PR line, `linked_material_id` is set on that line. The line stays free text (P04: a material is never substituted into an approved line). The process ends with the golden record live.
8. **Change an existing material (requester or steward):**
    1. **Preview:** `POST /v1/materials/{number}/change-requests/preview {changes}` returns the classification and impact. Impact covers open POs, live contracts, catalogue items, info records, open planned orders and requisition lines, live sourcing events, stock, forecasts, source lists and quotas.
    2. **Submit:** `POST /v1/materials/{number}/change-requests {changes, reason}` (scope `mdm:request` or `req:create`). It is refused in these cases:

Case	Response
Reason shorter than 5 characters	422
Material not `ACTIVE`	409 `MATERIAL_NOT_ACTIVE`
Another change request already `IN_APPROVAL`	409 `CHANGE_PENDING`
Field not changeable, or not applicable for the material type	422 `FIELD_NOT_CHANGEABLE`
Values unchanged	422 `NO_CHANGES`

Case	Response
UoM change while there is stock or open documents	422 `IMPACT_BLOCKED`

The request `MCR-…` is created `IN_APPROVAL`. One parallel `MATERIAL_CHANGE` task opens per owning domain, plus the extra domains the type requires (for example, a STOCK base-UoM change also needs QUALITY and FINANCE). The event `material.change.requested` is emitted.

3. **Apply:** on the last approval the material is row-locked. If its version changed since the request, the request becomes `REJECTED` (stale). Otherwise the fields are applied, the version is raised, a snapshot is kept in `material_version`, and the request becomes `APPLIED`. A `MATERIAL_MASTER` message with key `MAT|<number>|<version>` is queued and `material.changed` is emitted. ERP confirmation sets `erp_synced` and the ERP document.
4. **Reject or withdraw:** one rejection gives `REJECTED` (`material.change.rejected`). The requester can withdraw: `POST /v1/material-change-requests/{ref}/cancel` gives `CANCELLED`.

9. **Inbound master sync (admin).** `POST /v1/admin/erp-sync/MATERIALS/run` (or `/payload`) applies changed ERP materials (MARA/MAKT/MARC/MBEW) through the staged loader, with change detection by content hash and a delta cursor.

5. Document states

Object	States and transitions	Command / guard
Material request	`DRAFT` → `DUPLICATE_CHECK` (both transient, same call) → `DUPLICATE_BLOCKED` | `IN_STEWARDSHIP`; `DUPLICATE_BLOCKED` → `IN_STEWARDSHIP` (override) | `MERGED`; `IN_STEWARDSHIP` → `APPROVED` → `SYNCED_TO_ERP`; `IN_STEWARDSHIP` → `REJECTED` | `MERGED`	override only from `DUPLICATE_BLOCKED`; merge only from `DUPLICATE_BLOCKED` / `IN_STEWARDSHIP`; else 409 `ILLEGAL_TRANSITION`
Material change request	`IN_APPROVAL` → `APPLIED` | `REJECTED` (incl. stale version) | `CANCELLED`	decision only while `IN_APPROVAL` (409); withdraw by requester only
Material	`ACTIVE`, version n → n+1 per applied change; `erp_synced` false until the ERP confirms	

The TD state list (Draft → Duplicate Check → In Stewardship → Approved → Synced | Rejected | Merged) is implemented. The code adds `DUPLICATE_BLOCKED` as the waiting state of the duplicate gateway.

6. Business rules and controls

Rule set key	Controls (defaults)
`MDM-DUP`	`blockAt` 0.92, `warnAt` 0.75, weights description 0.5 / mpn 0.3 / attributes 0.1 / classification 0.1, `domains` PURCHASING, QUALITY, FINANCE, PLANT
`MATERIAL-FIELD-OWNERSHIP`	The owning domain per field. PURCHASING: description, group, category, MPN, attributes. PLANT: base UoM and all planning fields. FINANCE: valuation class, HSN, GST rate, standard price. QUALITY: quality-managed. `byType`: fields not applicable to SERVICE / NON_STOCK, and extra approving domains for STOCK. `outsideOwnership` `ROUTE` (or `REFUSE`); `distinctApprovers` true

Rule set key	Controls (defaults)
`ERP-TARGET`	ERP that receives the master messages (default `SAP_S4`)

Further rules:

- **Field validation:** HSN must be 4, 6 or 8 digits. The GST rate must be one of the legal slabs. MRP type must be one of `PD`, `ND`, `VB`, `VM`. Lot policy must be one of `EXACT`, `FIXED`, `EOQ`, `PERIOD`, `MINMAX`.
- **Impact warnings (not blockers):** HSN/GST changes do not alter open POs. Price or valuation changes with stock on hand need an ERP revaluation. Planning changes apply from the next MRP run. Category changes affect live sourcing and contracts.
- **ERP is updated only after final approval**, exactly once per material version (Blueprint §9.4).
- **Bank-detail changes** (dual finance approval, call-back) are in M06 (`SUPPLIER-QUAL.bankApprovers` 2, `coolingOffHours` 72).

7. Integrations and events

Event / message	Direction	Consumer
`material_request.approved`	emitted	`notify-material-approved` (requester)
`material.change.requested`, `material.change.rejected`, `material.changed`	emitted	stream only (no internal consumer)
`MATERIAL_MASTER` (create and change)	outbound to ERP	SAP: OData `API_PRODUCT_SRV` `A_Product`, IDoc `MATMAS05` / `MATMAS`; posted confirmation triggers `SYNCED_TO_ERP` / `erp_synced`
ERP `MATERIALS` kind	inbound sync (on demand)	Staged loader; sensitive vendor bank and tax differences from the `VENDORS` kind become MDM-04 change requests instead of being applied

8. Scheduled jobs

None for M03. Steward tasks use the workflow SLA timers and the `ESCALATION` rule like every approval:

- level 1: reminder;
- level 2: the manager role is added as a candidate;
- level 3: the task is reassigned.

9. KPIs

Code	Name	Formula
`M03.REQUEST_CYCLE`	Material request cycle time (days)	Mean time from material request to the created material (target 3, warn 5)
`M03.DUPLICATE_RATE`	Duplicate rate %	Requests blocked as duplicates or merged / requests (5 / 10)

Code	Name	Formula
`M03.FIRST_TIME_RIGHT`	First-time-right %	Requests never returned or rejected by a steward / submitted requests (90 / 75)
`M03.BANK_FRAUD_BLOCKED`	Bank-change fraud attempts blocked	Supplier bank-change requests rejected at call-back or approval

10. Demo data

- **Signature case, AXN:**
  - On 10 Oct 2026 the requester raises `REQ-2312`: 2 × "Custom fixture for DC-330 line 3", free text, ₹92,500 each, cost centre 4100, with one quote (HTS004), flagged reusable. This starts `MRQ-7731`.
  - The four domain stewards approve it (`mdm.purchasing`, `mdm.quality`, `mdm.finance`, `mdm.plant`), giving a new material that is synced to the ERP and linked to REQ-2312 line 10.
  - The buyer then requests "Aluminium die-cast housing DC-330" with MPN `AXN-DC-330`. It is blocked as a duplicate and merged into DC-330.
- **D1 operating cases (AXN, April 2027):**
  - Case 17: the QUALITY steward rejects a request (hydraulic oil).
  - Case 18: score in the 0.75–0.92 warning band, approved.
  - Case 19: score ≥ 0.92, steward override, approved.
- **D8 (AXN, July 2027):** three C-part materials are created through F9.
- **Material change requests:** no dataset seeds one. Try `/pic/materials/DC-330` and request a FINANCE change (for example the HSN code) as a steward of another domain to see routing.

11. Known limits and open items

- **No withdraw command for a material request.** A `DUPLICATE_BLOCKED` request that nobody merges or overrides stays open.
- **New-material approval always uses all four domains.** Field-level ownership by material type applies only to change requests.
- **No UI button for "promote free-text line to material request"** (API only).
- **The UI label "MDM-04" differs from the Blueprint.** The UI uses it for the material change request, while Blueprint MDM-04 is the supplier master change.
- **The change-request MRP-type list (`PD`, `ND`, `VB`, `VM`) is out of line with the planning engine** (`PD`, `PW`, `PM`, `VB`, `KB`, `ND`); see M02 §11.
- **Older docs are outdated.** `PIC_Module_Gap_Analysis.md` (27 Oct) says tax-ID and address changes are not built; `services/master_changes.py` now implements them with dual approval. `PIC_Pending_Workflows.md` items 17 and 18 (material change requests, field-level ownership) are now built.

M04 Requisition & Guided Buying

1. Purpose and scope

M04 lets any employee request what they need through the right channel. Search runs in the order contract item → catalogue item → material master → free text. The channels are:

- hosted catalogue and punch-out (cXML 1.2 / OCI 4.0);
- free-text and service requests, with request templates and category SOW forms.

Budget, quote policy, mandatory fields and the DMN-01 approval route are applied at submit. Approved lines (manual and MRP alike) reach the buyer workbench, which converts them to a purchase order (M08) or a sourcing event (M05). Blueprint §10 (REQ-01 to REQ-04), flow F2 (`pic.requisition`). Code: `services/requisition.py` , `guided_buying.py` , `punchout.py` , `request_templates.py` , `mandatory.py` , APIs in `api/plan_request.py` and `api/master_ops.py` .

2. Roles

Role	What they do in M04	Scopes
Requester	Searches, creates, edits, submits, cancels own requisitions; punch-out; promotes free-text lines	`req:create` , `ses:create`
Cost-Object Owner	Approves for their cost objects (attribute `cost_objects` , else plant)	tasks
Department Head	Second approver from ₹2 lakh	tasks
Procurement Head, Finance Controller	Third and fourth approvers above ₹10 lakh	tasks
Capex Committee	Added for asset purchases	tasks
Buyer	Works the buyer workbench; may change or cancel an APPROVED requisition; creates the PO or the sourcing event	`req:buyer` , `po:create` , `sourcing:manage` , `req:create`
MRP Controller	Creates MRP-origin requisitions (M02)	`mrp:plan` , `req:create`
Supplier Manager	Configures a supplier's punch-out endpoint	`supplier:manage`
Guided Buying Agent (AG-03, L2)	Adds its best match and an account-assignment suggestion to the search; re-checks free-text lines at creation; learns from the requester's choice	agent

Controls:

- **Four-eyes:** the requester and earlier approvers in the chain are excluded from later approval tasks.
- **ABAC:**
  - `REQ_SUBMIT` checks the submitter's `company_codes` and `plants` .

    - `REQ_APPROVE` checks the approver's company code and plant, and the value limit (`value_limit_minor` / `approval_limits`).
    - Rule `ABAC`, mode `ENFORCE`.
- **Only the requester** can submit (403) and add lines.

3. Screens

Screen ID	Route	What it shows	Actions
REQ-01	`/pic/requisitions/new` (card "Find what you need")	"What do you need?" search across contracts, catalogue, materials; recommended channel and agent hint; request templates and Guided SOWs (Professional services, IT, Marketing, Facilities); punch-out suppliers	Add line from a result, Punch out, Open a template/SOW form
REQ-02	`/pic/requisitions/new`, `/pic/requisitions/[ref]`	Header (plant, company code, purpose code/text), lines (type, purchase type, account assignment, need date, price, quotes, reusable/MPN), budget and policy results, approval chain, process history	Create (draft), Submit (justification, "request a sourcing event instead of quotes"), Edit line, Cancel
(list)	`/pic/requisitions`	Worklist by status/origin, "mine"	Open
REQ-03	`/pic/tasks`	Approval inbox (all task kinds) with evidence (total, budget result)	Approve, Return, Reject (comment required), bulk
REQ-04	`/pic/buyer`	Approved / partially ordered lines with open quantity, origin filter (MRP, MANUAL, …), source of supply, sourcing status	Create purchase order; sourcing bundling is on `/pic/sourcing/new` (M05)

4. End-to-end workflow

1. **Search (requester, REQ-01, F2-02/03).** `GET /v1/guided-buying/search?q=&plant=` (`req:create`).
    - **Results:** active contract lines (contract `ACTIVE` / `EXPIRING`, valid today, with today's price), active hosted catalogue items, and active materials, each with a text score of 0.6 or more.
    - **Recommendation:** the response names the recommended channel, the first non-empty one in the order contract → catalogue → material. `freeTextAllowed` is true only when nothing was found. The UI shows the hint "Use the … result before adding free text (P03)".
    - **Agent:** when the Guided Buying Agent may act, it adds its best match and a suggested account assignment, with an `aiRef`.
2. **Create (requester, REQ-02).** `POST /v1/requisitions` with origin `MANUAL`, `PROJECT` or `CATALOGUE`, plant, company code, optional purchasing org and group, purpose code and text, and lines (`Idempotency-Key` supported). The requisition is created `DRAFT` with number `REQ-…` and process `pic.requisition` starts. Each line is validated:

Line type	Must have
`CATALOGUE`	A catalogue item or a punch-out cart
`CONTRACT`	The contract line from the search, matching the material, on an executed contract; the price is set from the contract
`MATERIAL`	A material number

Line type	Must have
`FREE_TEXT`	No material

Every line also needs:

- a description, a material group and an estimated price;
- quantity > 0 and price ≥ 0;
- quotes recorded as `{supplier, reference, date}` (dates not in the future); a bare "quotes attached" count does not count;
- active organisation records (company code, plant, purchasing org/group, storage location, cost object; deactivated ones are refused).

Sales-order account assignments are checked against the order (`SALES-ORDER-ACCOUNT`). A free-text line marked `reusable` starts an MDM request in parallel (M03); the PR does not wait.

- **Templates:** a line that names a request template gets the template's defaults (type, purchase type, material group, category, UoM, default account type).
- **SOW forms:** the category SOW forms (`category-sow-forms.tsx`) build a structured SOW and send it as the line description.

3. **Punch-out alternative (requester).**

    1. `GET /v1/punchout/suppliers` , then `POST /v1/punchout/sessions {supplier}` . The session `PUN-…` is `OPEN` for `PUNCHOUT.sessionMinutes` (30), with BuyerCookie `<tenant>.<random>` . cXML posts a PunchOutSetupRequest; OCI returns the HOOK_URL form. A blocked supplier gives 409.
    2. The supplier posts the cart back to `POST /v1/punchout/return` . The session becomes `RETURNED` and `punchout.cart.returned` is emitted.
    3. `POST /v1/punchout/sessions/{cookie}/convert` (only by the user who punched out) creates a new `CATALOGUE` -origin requisition or adds lines to the user's own `DRAFT` / `RETURNED` requisition.
        - Items matching the hosted catalogue become `CATALOGUE` lines. A returned price more than `priceTolerancePct` (default 2%) above the catalogue price is refused.
        - Unmatched items become `FREE_TEXT` lines, and the cart counts as that supplier's quote.
        - Non-INR items are refused. The session becomes `CONVERTED` .

4. **Submit (requester, F2-07).** `POST /v1/requisitions/{ref}/submit {justification?, request_sourcing?}` . Allowed only from `DRAFT` or `RETURNED` , and only by the requester. Checks run in this order:

    1. ABAC `REQ_SUBMIT` .
    2. At least one line.
    3. P01 purpose code.
    4. `MANDATORY-FIELDS` for `REQUISITION` (422 `MANDATORY_FIELDS_MISSING` lists every gap).
    5. Per line:

Check	Rule
Account assignment	Needed for `NON_STOCK` , `SERVICE` , `EXPENSE` , `ASSET`

Check	Rule
Asset assignment	`ASSET` lines must be assigned to `ASSET` or `WBS`
Material master	Needed for `STOCK`, `CONSIGNMENT`, `SUBCONTRACTING`
Template fields	The template's `requiredFields`
Sales order	Re-checked
Quote policy	On lines without material, catalogue or contract: distinct-supplier quotes ≥ the required count, or, when sourcing is allowed for the value and `request_sourcing` is set, the line is marked `SOURCING_REQUIRED`

All violations come back together (422 `RULE_VIOLATION`).

6. Category-plan budget (`CATEGORY-BUDGET`, see §6).
7. Cost-object budget (`BUDGET`, see §6). The commitment is booked.
8. DMN-01 route.

The rule versions used are stored on the requisition (`rule_set_version`). The status becomes `SUBMITTED` and `requisition.submitted` is emitted.

5. **Route (system).**
   - **Empty chain** (≤ ₹25,000 and every line catalogue, or every line contract): the requisition goes `APPROVED` immediately (auto-approval, step F2-08).
   - **Otherwise** it goes `IN_APPROVAL` and `REQ_APPROVAL` tasks open: sequential, SLA 2 business days, scoped to the cost object (else the plant).
   - **Several cost objects:** each cost-object role (owner, department head) becomes a parallel stage with one task per cost object; the other roles run in sequence between those stages.
6. **Approve (approvers, REQ-03, F2-08).** `GET /v1/tasks`, then `POST /v1/tasks/{id}/complete {APPROVE|REJECT|RETURN, comment}`, or `/v1/tasks/bulk-complete`. Reject and return need a comment.

Decision	Result
Last approval	The requisition becomes `APPROVED` (see step 7)
`REJECT`	`REJECTED`; budget commitments released; process ends
`RETURN`	`RETURNED`; budget released; the requester edits and resubmits (back to step 4)

Unanswered tasks escalate per rule `ESCALATION`. Delegation of authority applies.

7. **Approved (system, F2-09).** On approval the system:
   - stores an approved snapshot (value, account, supplier per line);
   - re-runs source determination on material lines (`sourcing_status` `SOURCED` when a source is found);

- emits `requisition.approved`, which notifies the requester;
- ends the process. For MRP origin it records F1-09 (M02).

8. **Change after approval (requester or buyer).** `PATCH /v1/requisitions/{ref}/lines/{n}`.
   - The quantity cannot go below the already ordered quantity.
   - If value, account, account type or preferred supplier changed, the requisition goes back to `IN_APPROVAL`. Budget and category budget are re-booked (a soft stop takes the change's justification), DMN-01 is recomputed (at least the cost-object owner), and a new approval round starts.
   - Other changes (date, description) need no re-approval.
9. **Cancel (requester, or buyer once approved).** `POST /v1/requisitions/{ref}/cancel {reason}` is allowed from `DRAFT`, `RETURNED`, `IN_APPROVAL` or `APPROVED`. Budget is released, open tasks and timers are cancelled, and `requisition.cancelled` is emitted. It is refused in these cases:

Case	Response
On a PO	409 `REQUISITION_ORDERED`
Partly ordered	409 `REQUISITION_ORDERED`
Bundled into sourcing	409 `REQUISITION_IN_SOURCING`

10. **Buyer workbench and conversion (buyer, REQ-04).** `GET /v1/buyer-workbench?origin=` lists `APPROVED` and `PARTIALLY_ORDERED` lines with open quantity, oldest need date first.
    - **To PO (M08):** `POST /v1/purchase-orders` with source `REQUISITION`. Ordered quantity is booked per line, and the requisition goes `PARTIALLY_ORDERED` then `ORDERED`.
    - **To sourcing (M05):** bundle lines into a sourcing event from `/pic/sourcing/new` (`GET /v1/sourcing/bundle-candidates`). Lines go `IN_SOURCING`.
    - **Closing and reopening:** when all its POs are closed the requisition is `CLOSED`. A cancelled PO gives its quantity back (`ORDERED` / `PARTIALLY_ORDERED` → `APPROVED` or `PARTIALLY_ORDERED`).

5. Document states

From	Command	To	Who / guard
`DRAFT`, `RETURNED`	submit	`SUBMITTED`	requester; all submit checks pass
`SUBMITTED`	route / auto_approve	`IN_APPROVAL` / `APPROVED`	DMN-01 chain empty → auto
`IN_APPROVAL`	approve / reject / return	`APPROVED` / `REJECTED` / `RETURNED`	task completion; reject and return release budget
`APPROVED`	reapprove	`IN_APPROVAL`	line change of value, account or supplier
`APPROVED`, `PARTIALLY_ORDERED`	order_partially / order	`PARTIALLY_ORDERED` / `ORDERED`	PO creation (M08)
`ORDERED`	close	`CLOSED`	all POs closed
`ORDERED`, `PARTIALLY_ORDERED`	reopen / reopen_partially	`APPROVED` / `PARTIALLY_ORDERED`	PO cancelled

From	Command	To	Who / guard
`DRAFT`, `RETURNED`, `IN_APPROVAL`, `APPROVED`	cancel	`CANCELLED`	reason; not ordered, not in sourcing

Any other command returns 409 `ILLEGAL_TRANSITION`.

- **Punch-out session:** `OPEN` → `RETURNED` → `CONVERTED`; `OPEN` past its expiry is `EXPIRED` (409 `PUNCHOUT_SESSION_CLOSED`).
- **PR line `sourcing_status`:** `OPEN` / `SOURCED` / `SOURCING_REQUIRED` / `IN_SOURCING` / `ORDERED`.

6. Business rules and controls

Rule set key	Controls (defaults, INR)
`DMN-01`	See the approval bands below. Asset adds `CAPEX_COMMITTEE`; `slaBusinessDays` 2. Scope: company code, plant, dominant category by value, purchase type (ASSET if any line is an asset)
`QUOTE-POLICY`	See the quote counts below
`BUDGET`	`hardFor` ASSET, WBS (hard stop); `softFor` COST_CENTER, ORDER (soft stop with justification)
`CATEGORY-BUDGET`	`mode` SOFT, `warnAtPct` 90
`REQUEST-TEMPLATES`	`SVC-PROFESSIONAL`, `IT-HARDWARE-SOFTWARE`, `MKT-CAMPAIGN-EVENT`, `FAC-MAINTENANCE`: line defaults, `requiredFields`, `quotePolicy` (min quotes 2 / 2 / 3 / 1 from ₹25,000; sourcing event allowed from ₹10 L / ₹10 L / ₹5 L / ₹10 L), approver hints
`MANDATORY-FIELDS`	`REQUISITION`: header `purpose_code`; lines `description`, `material_group`, `qty`, `uom`, `need_date` (scoped versions replace the default for that scope)
`PUNCHOUT`	`sessionMinutes` 30, `protocols` CXML, OCI; `priceTolerancePct` default 2 in code
`SALES-ORDER-ACCOUNT`	Sales order must exist as demand, material must match, 0% over-assignment; budget only if one exists
`ABAC`	`REQ_SUBMIT` (company code, plant), `REQ_APPROVE` (+ value limit)
`ESCALATION`	SLA escalation of approval tasks (remind → add manager → reassign)

DMN-01 approval bands. The upper bound of each band is inclusive.

Requisition value	Approvers
≤ ₹25,000, all lines catalogue or all lines contract	Auto-approved
≤ ₹25,000, other channels	COST_OBJECT_OWNER
≤ ₹2 lakh	COST_OBJECT_OWNER
≤ ₹10 lakh	COST_OBJECT_OWNER, DEPT_HEAD

Requisition value	Approvers
> ₹10 lakh	COST_OBJECT_OWNER, DEPT_HEAD, PROCUREMENT_HEAD, FINANCE_CONTROLLER

Quote policy. Applies to free-text and free-service lines. Quotes are counted per distinct supplier, and each must have a reference and date.

Line value	Quotes required
< ₹25,000	0
≤ ₹2 lakh	1
> ₹2 lakh	3, or a sourcing event (`request_sourcing`)

A template can only tighten this policy, never relax it.

Budget calculations:

- **Cost-object budget:** per cost object and fiscal year (April–March), available = budget − committed − consumed.
  - Hard type over budget → 422 `BUDGET_EXCEEDED` .
  - Soft type over budget without justification → 422 `BUDGET_JUSTIFICATION_REQUIRED` .
  - No budget row → 422.
  - A sales order without a budget settles to the order.
  - Inventory lines without an account assignment: "not applicable".
- **Category budget:** applies only to categories with an `APPROVED` category plan (M01) that has a budget for the fiscal year.
  - committed = PO lines (not DRAFT/CANCELLED) created in the fiscal year + unordered value of SUBMITTED/IN_APPROVAL/APPROVED/PARTIALLY_ORDERED requisition lines.
  - used % = (committed + this requisition) / plan budget.
  - At or above `warnAtPct` : warning.
  - Above 100%: SOFT mode needs a justification (422 `CATEGORY_BUDGET_JUSTIFICATION_REQUIRED`); HARD mode blocks (422 `CATEGORY_BUDGET_EXCEEDED`).
  - Plan rows are locked, so concurrent submits are serialised.

Change after approval restarts approval only if value, account or supplier changes (Blueprint §10.4).

Principles enforced:

- **P01:** purpose code.
- **P02:** origin immutable; only MRP creates MRP-origin requisitions.
- **P03:** search order, and catalogue/contract lines need the item from the search.
- **P04:** free text never references a material, and is never auto-converted to one.
- **P08:** purchase-type rules.

7. Integrations and events

Event	Producer	Consumer
`requisition.submitted`	submit	stream only (the approval tasks carry it)
`requisition.approved`	approval	`notify-requisition-approved` (requester)
`requisition.cancelled`	cancel	`notify-requisition-cancelled` (requester)
`punchout.cart.returned`	cart return	stream only
`material_request.approved`	M03 (reusable line)	requester notification; the PR line gets `linked_material_id`

Punch-out protocols:

- **cXML:** PunchOutSetupRequest / PunchOutOrderMessage (`cxml-urlencoded` / `cxml-base64`).
- **OCI:** `NEW_ITEM-*` fields.
- The shared secret is field-encrypted. A built-in simulator supplier (`url = "builtin"`) runs the whole flow without an external catalogue.

ERP: requisitions are not posted to the ERP. A `PURCHASE_REQUISITION` mapping exists in `sap_mapping.py` / `connectors.py` but nothing queues it; the first ERP document is the PO (M08).

8. Scheduled jobs

No M04-specific job. Approval SLAs run on workflow timers (`ESCALATION` rule). Expired punch-out sessions are evaluated when they are read or returned; no job sweeps them.

9. KPIs

Code	Name	Formula
`M04.CATALOGUE_CHANNEL`	Catalogue / contract channel %	Requisition lines from a catalogue or contract / lines (target 60, warn 40)
`M04.APPROVAL_CYCLE`	PR approval cycle time (hours)	Mean time from submit to final approval (24 / 48)
`M04.FREE_TEXT_PCT`	Free-text %	Free-text lines / lines (15 / 25, lower is better)
`M04.BUDGET_BLOCKED`	Budget-blocked PRs	Requisitions held or stopped by a budget check (0 / 5)

10. Demo data

- **Signature case, AXN (users `requester` , `plant.controller` , `ops.head` , `buyer`):**
  - `REQ-2311` (origin MRP, ₹36.9 lakh) is approved through the 4-step chain.
  - `REQ-2312` (free-text fixture, 2 × ₹92,500, cost centre 4100, one quote from HTS004, reusable) passes the budget check, is approved by the cost-object owner, and starts `MRQ-7731` .
  - Open `/pic/requisitions/REQ-2312` and `/pic/buyer` .

- **D1 operating cases (AXN, April 2027):**
  - Case 9: an approver returns, the requester resubmits with a shorter chain, and a value change after approval restarts approval with the budget re-booked.
  - Case 10: the department head rejects and the budget is released.
- **D2 `purchase_types` (AXN, June 2027):** every purchase type from requisition to settlement.
- **D7 `industry_general` (tenant GEN):** a laptop bought by cXML punch-out against the built-in simulator supplier LVIT01, ₹78,500, with a single approval.
- **D17 `sourcing_ops` (KAC, AXN):** requisitions bundled into sourcing events.

11. Known limits and open items

- **No maintenance path for hosted catalogue items.** There is no API, screen or loader to create or maintain `catalogue_item` rows outside the seeds and datasets.
- **Category SOW form details are not stored as data.** The forms send structured `attributes`, but the requisition line has no attributes column, so the SOW survives only as the composed line description. That description is stored in a 200-character column; the API does not limit its length, so long SOW texts risk a database error (not verified at runtime).
- **Quotes are references only** (supplier, reference, date). There is no quote-document store.
- **The search is simple.** It is a text-similarity scan of all contract lines, catalogue items and materials. The catalogue search ignores the plant, and there is no search index (TD names OpenSearch).
- **Requisitions are never sent to the ERP** (see §7). Tenants that need ERP PRs would need that message switched on.
- **Older docs are outdated.** `PIC_Module_Gap_Analysis.md` (27 Oct) says punch-out is not built and the Guided Buying Agent does not log runs; both are now implemented. `PIC_Pending_Workflows.md` items 11 and 12 (sales-order account assignment, request templates) are now built.

M05 Strategic Sourcing

1. Purpose and scope

M05 runs RFIs, RFQs, RFPs and reverse auctions from approved requirements to an authorised, explainable award (Blueprint §11, flow F3, Technical Document §6/§9.2/§9.3). It covers event building and bundling, invitations of qualified suppliers, clarifications, sealed bids with an optional two-person opening, scoring with a technical gate, TCO, should-cost and benchmarks, award scenarios and optimisation, award governance, regret letters and the hand-off to contracts (M07), source determination and savings (M11). Code: `services/sourcing*.py`, `auction.py`, `should_cost.py`, `source.py`; API `api/source_contract.py`, `api/source_ops.py`, `api/wave1_c.py`; workflow process `pic.sourcing-award` (steps F3-02 to F3-10).

2. Roles

Role	What they do in M05	Scopes
CATEGORY_MANAGER	Builds, publishes, closes, opens and evaluates events of their categories; proposes the award; first approver in every award band	`sourcing:manage`, `sourcing:evaluate`, `savings:manage`
BUYER	Can build an event and manage the events they own	`sourcing:manage`
QUALITY_INSPECTOR	Evaluator (technical/quality criteria)	`sourcing:evaluate`
CPO, FINANCE_CONTROLLER	Award approvers in the upper band of `AWARD-APPROVAL`	`tasks:work`
CLINICAL_LEAD	Extra award sign-off in the healthcare template (`AWARD-APPROVAL.clinicalSignOff`)	`tasks:work`
SUPPLIER_USER	Asks questions, bids, withdraws a bid, bids in a live auction (supplier portal)	`bid:submit`, `supplier:portal`
TENANT_ADMIN	Maintains rule sets `SOURCING-POLICY`, `SOURCING-TCO`, `AWARD-APPROVAL`, `AWARD-RISK`, `AUCTION`	`admin:rules`

Commands that manage an event (publish, close, open, disqualify, new round, scenarios, award, cancel) are allowed only to the **event owner or a CATEGORY_MANAGER of the event's category** (`NOT_EVENT_MANAGER`, 403). SoD: **SOD-05** (`SOURCING_AWARD` vs `BID_SCORE`, condition SOLE_EVALUATOR): the sole evaluator of an event cannot award it; the award proposer never approves it (four-eyes on the approval chain). An evaluator who declared a conflict of interest cannot score or propose the award.

3. Screens

Screen (BP)	Route under /pic	What it shows	Actions
SRC-01 Sourcing Project Cockpit	`/sourcing`	Worklist of events by status	Open event, new event

Screen (BP)	Route under /pic	What it shows	Actions
SRC-02 Event Builder	`/sourcing/new`	Steps: 1 Event, 1b Schedule/scoring/rules, 2 Requirement lines (approved requisitions, items and lots), 3 Scoring model, 4 Invitees, 5 Baseline/should-cost/award constraints	Save draft, Save and publish
SRC-02 (templates)	`/sourcing/templates`	Event templates (type, criteria, constraints, scoring/baseline method, invitee filter)	New template, new version
SRC-03 Bid Comparison & TCO (event page)	`/sourcing/[ref]`	Header, lines; tabs Live auction, Criteria, Invitations, Clarifications, Bids and opening, Evaluation, Comparison, TCO simulator, Award scenarios (SRC-04), Award (SRC-05), Hand-off, History	Publish, Close, Open bids, Score, Declare, Disqualify, New round, Build scenarios, Award, Cancel event, Clone
SRC-03 (tools)	`/sourcing/should-cost`, `/sourcing/benchmarks`, `/sourcing/tco`	Should-cost models, price benchmarks, TCO simulator	New model/version, Evaluate, Record a price
SRC-05 Award Approval	`/tasks` (task kind `AWARD_APPROVAL`)	Scenario, allocations, rationale, exceptions	Approve, reject
SLM-05 Supplier portal (RFQs tab)	`/portal`	Invited events, questions, own bid	Ask, bid, withdraw, auction bid

4. End-to-end workflow

1. **Demand arrives.** Approved requisition lines (manual, catalogue or MRP origin) with no source of supply wait in the buyer queue (`/buyer`, REQ-04). The Sourcing Strategy Agent (L1, `sourcing_strategy.py`, `/v1/agents/SOURCING_STRATEGY/runs`) may recommend event type, invitees and weights by Kraljic quadrant.
2. **Build the event (F3-02).** Category manager or buyer, SRC-02, `POST /v1/sourcing/events` (or `POST /v1/sourcing/templates/{code}/events`, or `POST /v1/sourcing/events/{ref}/clone`). Checks:
   - type RFI, RFQ, RFP or REVERSE_AUCTION; title and category; close time; `clarification_close_at` before the close; `open_at` before the close and not for an auction;
   - **P06**: every line comes from an APPROVED or PARTIALLY_ORDERED requisition; one company code per event; same material/plant/UoM when sources are merged into one line; at most 50 lines; lots (`lot`) with codes `[A-Z0-9_-]`; an auction and a should-cost model are single-line only;
   - bundling (`bundle.kind` PR_LINES, MRP_DEMAND, CATEGORY_PLAN): only OPEN requirement lines; MRP_DEMAND only MRP-origin requisitions; CATEGORY_PLAN must match the category. A line of another category is refused (`SOURCING-POLICY.lineCategoryCheck` ENFORCE, always for bundles) or audited (WARN);
   - criteria: unique codes, whole-number weights 0-100, data source MANUAL/SCORECARD/ESG/RISK, optional `min_score` (technical gate);
   - invitees: no duplicate; category status QUALIFIED or PREFERRED, or CONDITIONAL with a `waiver_reason` (`P06/SRC-INVITE`). Candidates with segment and risk filters: `GET /v1/sourcing/invitee-candidates`;
   - baseline per line by `baseline_method` (LAST_PRICE, WEIGHTED_AVG, SHOULD_COST, MARKET_INDEX, BUDGET, MANUAL); the TCO rule values are copied to the event (`tco_params`);
   - ABAC `SOURCING_CREATE` (category, plant, company code). Result: event `DRAFT`, number from the `RFQ-` series (all types use it), requirement lines set to `IN_SOURCING`, audit `CREATED`.
3. **Publish (F3-02).** Owner/category manager, `POST .../publish` (Idempotency-Key, If-Match with the event ETag). Guards: weights sum to 100 (`SRC-WEIGHTS`) and lock; RFQ/auction need `PRICE_TCO` > 0; at least one invitee; close in the future; purpose code (**P01**); mandatory fields per type (`need_date` for RFQ/RFP/auction); `minOpenHours`; clarification deadline in the future; close before the need date (`SRC-NEEDDATE`); category budget (`budgetCheck` SOFT needs `budget_justification`, HARD refuses); fewer invitees than `minBidders` is recorded. An auction

freezes its `AUCTION` settings. Status `PUBLISHED` then immediately `OPEN`, unless `open_at` is in the future (stays `PUBLISHED`; timer `SOURCING_OPEN`). Timer `SOURCING_CLOSE` is created. Event `sourcing.event.published` (savings initiative IDENTIFIED).

4. **Clarifications.** Supplier asks (`POST .../clarifications`) until the clarification deadline (`CLARIFICATION_WINDOW_CLOSED`, 409); owner answers (`.../{seq}/answer`) or posts a notice; supplier may withdraw its own question. Answers go to every invitee without the asker's name (`sourcing.clarification.answered`, notification CLARIFICATION_ANSWERED).
5. **Bid (F3-03).** Supplier, portal, `POST .../bids` (lines, optional lots with `discount_pct` / `all_or_nothing`, `valid_until`). Gate: event OPEN, before close, invited, not disqualified, still eligible; plausibility (`priceBand` vs baseline and absolute max, lead time, payment days, validity), one currency per bid with an FX rate on the reference date, offer valid beyond the close. Refusals are logged (`LATE_BID_REFUSED`, `UNINVITED_BID_REFUSED`, `INELIGIBLE_BID_REFUSED`, `DISQUALIFIED_BID_REFUSED`). The bid is stored encrypted with only its SHA-256 readable; re-submitting replaces it (edit counter); `POST .../bids/withdraw` destroys the content. Events `sourcing.bid.submitted` (feeds the responsiveness scorecard), `sourcing.bid.withdrawn`.
    - **Reverse auction:** bids via `POST .../auction/bids` (price only). Each bid must be below the supplier's last bid by `minDecrementPct`, at or below `ceilingMinor`; soft close extends by `extensionMinutes` when a bid arrives in the last `triggerMinutes`, at most `maxExtensions` (12) times; rank/best price shown per `showRank` / `showBestPrice`. At the end each supplier's last price becomes a sealed bid.
6. **Close and open (F3-03).** Timer at close time (or owner `POST .../close`; `CLOSE_TIME_NOT_REACHED` before). Without two-person opening the bids are unsealed at once (`BIDS_UNSEALED`), status `CLOSED` -> `UNDER_EVALUATION`, event `sourcing.bids.unsealed`, `competitive` = valid bidders >= `minBidders`. With `SOURCING-POLICY.twoPersonOpening` or `constraints.two_person_opening` the event waits in `CLOSED`; two different managers each call `POST .../open` (`ALREADY_OPENED_BY_YOU`, 409). Any earlier read of bids answers **423 BIDS_SEALED** and writes `BID_ACCESS_DENIED`.
7. **Evaluate (F3-04).** Evaluators, Evaluation tab: `POST .../declarations` first (conflict of interest, at least 10 characters; a declared conflict cannot be withdrawn), then `POST .../scores` (0-100 per manual criterion; SCORECARD/ESG/RISK criteria are scored from data at the close). `GET .../comparison` and `comparison.csv` show the TCO stack, price score, weighted score, technical gate, outlier and collusion screens, ranking. Disqualify with a written reason: `POST .../disqualify` (bid stays on record, scenarios marked superseded, supplier notified, `sourcing.bid.disqualified`).
8. **Another round? (F3-05).** `POST .../rounds` with a new close time: status back to `OPEN`, round + 1, new close timer; when the new round is unsealed, a supplier's earlier-round bid becomes SUPERSEDED by its new bid (manual scores stay with the supplier).
9. **Scenarios (F3-06).** Owner, `POST .../scenarios`: LOWEST_COST, BEST_VALUE, CAPACITY_CONSTRAINED (optimiser), or `POST .../scenarios/custom` (CUSTOM split by % or monthly quantity, must total 100 %). Bidders failing the technical gate or excluded by `AWARD-RISK` caps are left out. Each scenario has weighted price/TCO, annual savings vs baseline and a risk score.
10. **Propose the award (F3-06/07).** Owner, `POST .../award` with scenario, rationale and conflict declaration. `award_gate` blocks (422): `BID_NOT_EVALUATED`, `OFFER_EXPIRED`, `TECHNICAL_GATE`, `UNSCORED_CRITERIA`, `LEAD_TIME_AFTER_NEED` (ENFORCE). Exceptions recorded on the decision: NOT_LOWEST, LIMITED_COMPETITION, INCOMPLETE_TCO, UNVERIFIED_CAPACITY, NEGATIVE_SAVINGS, LEAD_TIME_AFTER_NEED; those needing a rationale require `minRationaleChars` (15). ABAC `AWARD_APPROVE` (category, value limit); SoD-05 guard. Status `AWARD_PENDING`.
11. **Approve (F3-07/08).** If annual value >= `approvalAboveMinor` (50 lakh) or there is a rationale-type exception, `AWARD_APPROVAL` tasks follow the band chain (default: up to 50 lakh CATEGORY_MANAGER; above: CATEGORY_MANAGER -> CPO -> FINANCE_CONTROLLER). Otherwise the award is immediate. A rejection returns the event to `UNDER_EVALUATION` (decision REJECTED, scenario kept as evidence).
12. **Award and hand-off (F3-09/10).** Status `AWARDED`; requirement lines `SOURCED`; regret letter per non-awarded bidder (`sourcing.award.regret`, notification SOURCING_NOT_AWARDED with its own rank, never a competitor's data); `sourcing.event.awarded` triggers: `contract-from-award` (one contract draft per awarded supplier: SCHEDULING_AGREEMENT for a material, VALUE_CONTRACT for free text; multi-line: one contract per supplier with all its lines; validity from the 1st of next month for 12 months; tolerances and LME index from `CLM-PLAYBOOK.awardDefaults`) -> M07; `savings-negotiated` (initiative NEGOTIATED with the baseline method) -> M11; `notify-sourcing-awarded`. On contract activation: price validities published, quota arrangement created, source-list entry added (`sourcing-source-list`) -> M08 source determination. `GET .../handoff` shows contracts, prices, quota counters, source list and `poReleaseEligible`.
13. **Cancel (any state before AWARDED).** `POST .../cancel` with reason: open tasks/timers closed, pending award withdrawn, requirement lines not sourced elsewhere back to OPEN, invitees told (once published), `sourcing.event.cancelled` (savings LOST). An awarded event cannot be cancelled.

5. Document states

From	Command (route)	To	Guard
DRAFT	publish (`POST .../publish`)	PUBLISHED, then OPEN at once	publish checks (§4 step 3)
PUBLISHED	open (timer `SOURCING_OPEN`)	OPEN	`open_at` reached
OPEN	close (timer `SOURCING_CLOSE` or `POST .../close`)	CLOSED	close time (auction: extended end) reached
CLOSED	evaluate (automatic at close, or second `POST .../open`)	UNDER_EVALUATION	two different openers when required
UNDER_EVALUATION	new_round (`POST .../rounds`)	OPEN	future close time
UNDER_EVALUATION	propose_award (`POST .../award`)	AWARD_PENDING	award gate, SoD-05, ABAC
AWARD_PENDING	award (auto or last approval)	AWARDED	approver != proposer
AWARD_PENDING	reject_award (approval rejected)	UNDER_EVALUATION	
DRAFT, PUBLISHED, OPEN, CLOSED, UNDER_EVALUATION, AWARD_PENDING	cancel (`POST .../cancel`)	CANCELLED	reason required

Bid statuses: SEALED, WITHDRAWN, UNSEALED, SUPERSEDED, DISQUALIFIED, INVALID. Award decision: PENDING, APPROVED, REJECTED, CANCELLED. Any other command answers 409 (`illegal_transition`).

6. Business rules and controls

Rule set	Controls
`SOURCING-POLICY`	`minBidders` (2), `limitedCompetitionNeedsRationale` , `requireAllCriteriaScored` , `minRationaleChars` (15), `incompleteTcoNeedsRationale` / `unverifiedCapacityNeedsRationale` (off by default; on in the KAC versions), `unverifiedCapacity` FLAG/BLOCK, `twoPersonOpening` , `tieBreak` , `minOpenHours` , `clarificationCloseHours` , `leadTimeCheck` ENFORCE/WARN/OFF, `budgetCheck` HARD/SOFT/OFF, `mandatoryFields` , `lineCategoryCheck` , `maxExtensions` , `priceBand` , max lead/payment/validity days, `outlier` , `collusion` , `baselineMethods`
`SOURCING-TCO`	`carryingRatePct` 18, `costOfCapitalPct` 12, `copqPerDefectMinor` , `includeSafetyStock` / `maxSafetyDays` , `defaultPpm` / `ppmFromScorecardPeriods` , freight estimates by incoterm, `dutyRates` by HS prefix, `taxCreditable` , `missingInput` FLAG/REJECT
`AWARD-APPROVAL`	value bands and chains, `approvalAboveMinor` 50 lakh

Rule set	Controls
`AWARD-RISK`	`maxRiskScore`, `maxSingleSourcePct`, `maxConditionalSharePct`, `noDataScore` (event constraints can override)
`AUCTION`	`minDecrementPct` 0.5, `triggerMinutes` 2, `extensionMinutes` 5, `showRank`, `showBestPrice`, optional `ceilingMinor` (per event: `PUT .../auction` while DRAFT)

Calculations:

- **TCO per base unit (paise):** `price + freight + duty + packaging + carrying + quality - payment-term value`; carrying = price x carrying rate x (lead time days + safety days)/365; quality = PPM/1,000,000 x CoPQ; payment value = price x cost of capital x payment days/365; duty = (price + freight) x duty % of the HS prefix; freight estimated as % of price for EXW/FCA/FAS/FOB when not quoted. Prices are normalised first (FX on the reference date, UoM factor, GST stripped when creditable). Each component has a status (BID, ESTIMATED, TWIN, SCORECARD, RULE_DEFAULT, INCLUDED, NOT_QUOTED, NOT_APPLICABLE, MISSING); MISSING makes the line `tcoIncomplete`. Try it: `POST /v1/sourcing/tco/simulate`.
- **Price score** (`scoring_method`): LOWEST_RATIO = lowest TCO / this TCO x 100; MIN_MAX = (highest - this)/(highest - lowest) x 100; RANK = 100 x (n - rank + 1)/n; value-weighted over lines. **Weighted score** = sum(weight x score)/100; unscored criteria count as 0 and are listed.
- **Ranking**: by TCO, then weighted score, earliest bid, supplier code.
- **Optimiser (CAPACITY_CONSTRAINED)**: minimise sum(x x TCO x Q) subject to sum x = 1, x x Q <= min(bid capacity, twin free capacity), `min_share` <= x <= `max_share`, at most `max_suppliers`, MOQ (default constraints: 2 suppliers, max 60 %, min 20 %); risk caps from `AWARD-RISK`.
- **Savings**: (baseline - awarded price) x annual quantity; an award above baseline is a NEGATIVE_SAVINGS exception, not a negative saving.
- **Should-cost models** (`POST /v1/sourcing/should-cost/models`, versioned): components QTY_RATE, TIME_RATE, FIXED, PCT_OF, margin, index-linked rates. **Benchmarks**: `GET/POST /v1/sourcing/benchmarks`.

Controls: Idempotency-Key on create/publish/award and other commands; If-Match/ETag on event commands (412 when stale); ABAC actions `SOURCING_CREATE`, `SOURCING_MANAGE`, `SOURCING_EXPORT` (category) and `AWARD_APPROVE` (category + value limit); CSV export carries `X-Content-SHA256` and audit `COMPARISON_EXPORTED`.

7. Integrations and events

Event	Consumers
`sourcing.event.published`	`savings-identify`
`sourcing.bid.submitted`	`scorecard-responsiveness` (M06)
`sourcing.bid.withdrawn`, `sourcing.bids.unsealed`	stream only
`sourcing.clarification.answered`	`notify-clarification`
`sourcing.bid.disqualified`	`notify-bid-disqualified`
`sourcing.award.regret`	`notify-award-regret`
`sourcing.event.awarded`	`contract-from-award` (M07), `savings-negotiated` (M11), `notify-sourcing-awarded`
`sourcing.event.cancelled`	`notify-sourcing-cancelled`, `savings-lost`

Event	Consumers
`contract.activated` (M07)	`sourcing-source-list` (source list entry), `savings-contracted`

No ERP message is sent by M05 itself; the award reaches the ERP later through contracts/POs. The Supplier Capability Digital Twin (`twin.py` , `PIC_TWIN_*`) supplies free capacity, PPM and late-delivery probability; AI: Bid Analysis agent run (L2) at opening. Notifications: SOURCING_AWARDED, SOURCING_NOT_AWARDED, SOURCING_DISQUALIFIED, SOURCING_CANCELLED, CLARIFICATION_ANSWERED, TASK_ASSIGNED.

8. Scheduled jobs

No cron job. Workflow timers drive the flow: `SOURCING_OPEN` (scheduled opening), `SOURCING_CLOSE` (close and unseal, per round), task SLA timers on `AWARD_APPROVAL` . Timers fire on the worker tick (`python -m app.pic.worker` , Vercel Cron `/internal/worker/tick` , or `POST /v1/admin/ops/tick`); closing is idempotent.

9. KPIs

Code	Name	Formula
`M05.SOURCING_CYCLE`	Sourcing cycle time (days)	mean time from publication to the approved award
`M05.COMPETITIVE_EVENTS`	Competitive events %	awarded non-RFI events with at least `minBidders` valid bidders / awarded non-RFI events
`M05.NEGOTIATED_SAVINGS_PCT`	Negotiated savings %	negotiated savings / (baseline price x annual volume)
`M05.BIDDERS_PER_EVENT`	Bidders per event	distinct bidders evaluated or disqualified / published events with a valid bid
`M05.SUPPLIER_PARTICIPATION`	Supplier participation %	invited suppliers that bid / invitations of published events
`M05.AWARD_TO_CONTRACT`	Award-to-contract lead time (days)	mean time from approved award to activation of its first contract
`M05.SAVINGS_LINK`	Sourced savings realised %	realised / negotiated savings of initiatives from a sourcing event
`M05.NON_LOWEST_AWARDS`	Awards to a non-lowest bid %	approved awards with a NOT_LOWEST exception / approved awards

`GET /v1/kpis?module=M05` ; `GET /v1/sourcing/pipeline` lists events per stage with value, age and next step.

10. Demo data

- **AXN (Axion signature case, `seed/axion.py`): RFQ-2231**, category ALU-DIECAST: Kovai Diecast (KD002) 412 then 405 in round 2, Trichy Forge (TF001) 398, Salem Light Alloys (SLA003) 405; should-cost 391; baseline last price 410; CAPACITY_CONSTRAINED award TF001 60 % @ 398 / KD002 40 % @ 405 (weighted 400.80), approved CATEGORY_MANAGER -> CPO -> FINANCE_CONTROLLER; contract drafts `SA-5520` onward.
- **D17 `sourcing-ops` (KAC and AXN)**: KAC has 40 MOCK events (titles `MOCK K01` to `MOCK K40` , plan in `backend/data/erp_mock/event_plan_kac.json`) in every type and state: e.g. K08/K38 multi-round, K07 lot bidding, events waiting for the two-person opening, scheduled (PUBLISHED), auctions; templates RFQ-DIRECT-MATERIAL, RFP-SERVICES, AUCTION-STANDARD. AXN gets six extra events A01-A06 (Aug-Sep 2027).
- **D1 `axn-module-cases`** : DC-350 reverse auction with clarification board, eight live bids, two soft-close extensions.

11. Known limits and open items

- Not built: Dutch and sealed-then-auction formats, alternate offers per line, two-envelope (technical before price) evaluation, local content/ESG caps in the allocation (`PIC_Strategic_Sourcing_Analysis.md`).
- Offer validity is checked at bid and award; no job expires offers automatically.
- All event types share the `RFQ-` number series (an RFP is numbered RFQ-nnnn).
- No sourcing data is sent to the ERP; outcome reaches the ERP only through contracts and POs.
- Default TCO rule has an empty `dutyRates` table and `defaultPpm` null: imports and suppliers without PPM show MISSING until the tenant fills them.

M06 Supplier Lifecycle & Performance

1. Purpose and scope

M06 onboards, qualifies, segments, monitors and develops suppliers, with the Supplier Capability Digital Twin as data foundation (Blueprint §12, flow F8 for onboarding, MDM-04 for bank and master changes). Qualification is held **per category**: a supplier can be QUALIFIED for one category and REGISTERED or EXITED for another. Code: `services/supplier.py` (process `pic.supplier-onboarding`, steps F8-01 to F8-09), `invitations.py`, `master_changes.py`, `risk.py`, `risk_feeds.py`, `sanctions_feed.py`, `supplier_mgmt.py`, `twin.py`.

2. Roles

Role	What they do in M06	Scopes
SUPPLIER_MANAGER	Invites/registers suppliers, starts and decides qualification, disposes screening, blocks/unblocks, requests and call-back-verifies bank changes, requests tax-ID/address/contact changes and is one of the two master-change approvers, segments, development plans, risk mitigation	`supplier:manage`, `supplier:bank`
FINANCE_CONTROLLER	Approves bank changes (two different controllers) and is the other master-change approver; maintains supplier finance data (MSME, TDS)	`tasks:work`, `payment:run`
QUALITY_INSPECTOR	Verifies certificates, owns and verifies CAPA	`quality:decide`
CATEGORY_MANAGER	Sets PREFERRED / EXITED per category	(role check)
SUPPLIER_USER	Self-registers; submits profile, certificates and bank details during onboarding; requests its own tax-ID, address and contact changes (portal Company details); answers CAPA and ESG/cyber questionnaires	`supplier:portal`
TENANT_ADMIN	Sanctions list sync (`POST /v1/admin/sanctions/sync`), risk feed configuration, rule sets	`admin:integration`, `admin:rules`

SoD and four-eyes (supplier-master governance, Blueprint MDM-04):

Rule	Conflict	Where recorded	Where enforced
SOD-01 (High)	`SUPPLIER_CREATE` vs `PO_CREATE` on the same supplier	`supplier.register`	PO creation (`order._create_po`, `sod.guard`)
SOD-02 (Critical)	`SUPPLIER_BANK_CHANGE` vs `PAYMENT_APPROVE`	`supplier.request_bank_change` (the requester)	payment proposal and payment run (`payables.py`)
SOD-02a (Critical)	`SUPPLIER_BANK_CHANGE` vs `INVOICE_POST`	same	invoice and debit-note posting (`payables.py`, `debit_notes.py`)

Critical rules can never be mitigated through the SoD exception path (`SOD-MITIGATION`). Further four-eyes checks in code: the bank call-back is done by someone other than the requester (`SOD_CONFLICT`); the requester and the call-back verifier are excluded from the bank approval tasks; the registrant cannot clear or merge its own screening; the uploader cannot verify a certificate; a master change is never approved by its requester (action `SUPPLIER_MASTER_CHANGE` is logged and checked in the task guard) and its two approvals must come from two different people.

3. Screens

Screen (BP)	Route under /pic	What it shows	Actions
SLM-02 Onboarding & Qualification (list)	`/suppliers`	Supplier base with status per category, segment filter; invitations (F8-01)	Invite supplier, revoke invitation
(register)	`/suppliers/new`	Company, tax IDs, commercial defaults	Register supplier
(self-registration)	`/register?token=...`	Public page opened from the invitation e-mail	Register (creates the portal login)
SLM-01 Supplier 360	`/suppliers/[code]`	Twin summary, categories and qualifications, certificates, screening, bank accounts and pending bank changes, master data card (masked GSTIN, address, contact; change history with before/after and approvers), segment, assessments, development plans, finance (MSME, TDS), audit, onboarding process	Start qualification, record assessment, decide, verify certificate, screening disposition, request bank change, verify call-back, request master change (TAX / ADDRESS / CONTACT), block/unblock
MDM-04 Supplier Master Change (approval)	`/tasks`	Task kinds `BANK_CHANGE` and `SUPPLIER_MASTER_CHANGE` with masked before/after, reason, suspected duplicate GSTINs	Approve, reject
SLM-03 Performance Scorecard	`/suppliers/performance`	Latest scorecard per supplier (OTIF, PPM, invoice accuracy), CAPA list	Open CAPA, verify
SLM-04 Supplier Risk Monitor	`/suppliers/risk`	Risk heat map and signals by category	Run Supplier Risk Agent, mitigate
SLM-05 Supplier Portal	`/portal`	Company details (the same master data card, portal mode), RFQs, orders, contracts, invoices, service sheets, CAPA	Request master change, and the supplier-side actions of every flow

4. End-to-end workflow

1. **Invite or register (F8-01).**
   - Invitation: supplier manager, `/suppliers`, `POST /v1/supplier-invitations` (e-mail, legal name, categories). One open invitation per e-mail (`INVITATION_PENDING`); token stored as SHA-256 only; valid `SUPPLIER-INVITATION.validDays` (14). Event `supplier.invitation.sent`. The supplier opens `/register`, `POST /v1/public/invitations/{token}/register` (tax IDs, password policy); expired/used/revoked links answer 410. Result: supplier registered, qualification started for the invited categories, portal login (SUPPLIER_USER) created, `supplier.registered` (notifies supplier managers).
   - Direct: supplier manager, `/suppliers/new`, `POST /v1/suppliers`. Checks: GSTIN format/state/check digit, PAN inside GSTIN, payment-terms key (`N60`, `2/10 N45`). Supplier `REGISTERED`, every category `REGISTERED`, SOD-01 recorded, onboarding process starts and waits at F8-02.
2. **Profile, bank, certificates (F8-02).** Supplier portal or manager, `POST /v1/suppliers/{code}/profile`: bank details (account, IFSC, holder: all three) only while onboarding runs; certificates any time (expiry timer `CERTIFICATE_EXPIRY` armed). Bank details create a bank change request (`PENDING_CALLBACK`) and trigger screening.

3. **Screening (F8-03).** Automatic: duplicates by GSTIN/PAN blind index and similar normalised legal name; sanctions by fuzzy match against the local list and, when configured, the licensed provider (`PIC_SANCTIONS_PROVIDER/URL/API_KEY`). Outcome CLEAR, REVIEW (duplicate suspected or provider unreachable) or BLOCKED (sanctions hit: supplier is blocked at once). REVIEW is disposed by a supplier manager other than the registrant: `POST /v1/suppliers/{code}/screening-disposition` CLEAR, MERGE (into the duplicate: status MERGED, categories EXITED) or RESCREEN.
4. **Bank details and bank changes (MDM-04, F8-03/04).**
   - *Request.* During onboarding the bank details come with the profile (step 2). After onboarding they change only through `POST /v1/suppliers/{code}/bank-changes` (`supplier:bank` , i.e. a supplier manager; the portal answers `BANK_CHANGE_PROCESS_REQUIRED`), which starts its own `BANK_CHANGE` process. Checks: supplier not BLOCKED; no other bank change pending (`BANK_CHANGE_PENDING`); an Indian account needs the IFSC; a foreign account (country other than IN) needs SWIFT/BIC and may carry intermediary BIC, charge bearer (DEBT, CRED, SHAR) and purpose code. The new account is stored encrypted and inactive, request number `BCR-nnn` , status `PENDING_CALLBACK` ; **SOD-02/02a** recorded for the requester.
   - *Call-back.* A different person phones the supplier on a known number and records it: `POST /v1/bank-changes/{number}/callback` with a note (status `PENDING_APPROVAL`). Requester = caller answers `SOD_CONFLICT` .
   - *Dual approval.* `BANK_CHANGE` tasks for `SUPPLIER-QUAL.bankApprovers` (2) FINANCE_CONTROLLERs, excluding requester and caller. Any rejection: REJECTED, the old account stays active.
   - *Activation.* On the last approval the new account becomes active (`verified_on` today) and every other account of the supplier is deactivated; `payable_from` = now + `coolingOffHours` (72). The payment proposal and payment run skip the supplier until then ("bank change cooling-off until ..."). The risk scan raises a FRAUD signal when an invoice arrives within `bankChangeInvoiceDays` (30) of a bank change. For an ACTIVE supplier the change goes to the ERP (`SUPPLIER_MASTER` , key `{code}\|BANK\|{BCR number}`); a supplier not yet in the ERP carries its account in the vendor-create message.
5. **Qualification per category (F8-05).** Supplier manager, Supplier 360: `POST /v1/suppliers/{code}/qualifications` (category `UNDER_QUALIFICATION`); `POST /v1/qualifications/{qid}/assessment` (questionnaire score, audit score, sample approval such as `PPAP_APPROVED`); certificates verified by a supplier manager or quality inspector (`POST /v1/certificates/{cid}/verify`).
6. **Decision (F8-06/07).** `POST /v1/qualifications/{qid}/decision` QUALIFIED, CONDITIONAL (written waiver) or REJECTED (category EXITED). QUALIFIED requires, from `SUPPLIER-QUAL` for the category: mandatory certificates present, valid and verified; questionnaire >= `minQuestionnaireScore` (70); audit >= `minAuditScore` (75); PPAP for `sampleApprovalRequired` categories. Any non-rejected decision also requires an active verified bank account and a passing screening gate. First non-rejected decision: supplier `ACTIVE` and ERP vendor creation (`SUPPLIER_MASTER` , idempotency key `{code}\|CREATE` ; `twin_ref` set when posted).
7. **Monitoring on (F8-08/09).** Certificate expiry timers set; onboarding ends when the last invited category is decided (F8-09).
8. **Performance.** Scorecards per supplier and month update on `goods_receipt.posted/reversed` (OTIF), `quality.usage_decision` (PPM), `invoice.posted` (first-pass invoice accuracy) and `sourcing.bid.submitted` (responsiveness); ESG/CAPA measures via `POST /v1/suppliers/{code}/scorecard-measures/refresh` ; monthly recalculation job. Scorecards are written back to the twin.
9. **CAPA and development.** A rejected quantity at the usage decision opens a CAPA automatically (`capa-on-rejection`); manual `POST /v1/capas` . Supplier responds (`/capas/{ref}/response` : root cause + actions, status RESPONDED); quality verifies (`/capas/{ref}/verify` : CLOSED or back to OPEN). Development plans (`/suppliers/{code}/development-plans` , actions with owners and due dates, reminders, review task before closing).
10. **Risk (SLM-04).** Supplier Risk Agent run (`POST /v1/supplier-risk/scan` or agent run) creates signals from sanctions screening, legacy compliance, financial tier HIGH, certificates expiring within `certificateWarnDays` (60), OTIF < `minOtifPct` (90), PPM > `maxPpm` (1000), twin late probability >= `maxLateProb` (0.30), invoice within 30 days of a bank change (FRAUD), plus CONCENTRATION, GEOGRAPHIC, SINGLE_SOURCE and CYBER (these open mitigation tasks). External feeds: `POST /v1/risk-feeds/{source}/pull` or signed webhook. `POST /v1/risk-signals/{id}/mitigation` records the action.
11. **Status changes.** PREFERRED (`POST /v1/suppliers/{code}/categories/{category}/status`): only from QUALIFIED, average OTIF >= `preferredOtifPct` (95) and worst PPM <= `maxPpm` over the last three scorecards. EXITED per category (open POs audited). Expired mandatory certificate without renewal: category QUALIFIED/PREFERRED -> CONDITIONAL automatically.
12. **Block / unblock.** `POST /v1/suppliers/{code}/block` (or automatic on a sanctions hit): supplier and all categories BLOCKED, unpaid invoices get a payment block, each open PO gets a `BLOCKED_SUPPLIER_PO` buyer review task; `supplier.status.changed` flags open POs and sourcing events and notifies their owners. Unblock (supplier manager, reason) restores the previous status and lifts the blocks it set; refused while a sanctions match stands.

13. **Tax-ID, address and contact changes (MDM-04, `services/master_changes.py`).**

- *Request.* Supplier manager on Supplier 360, or the supplier's own portal user on Company details: `POST /v1/suppliers/{code}/master-changes` with `fieldGroup` TAX, ADDRESS or CONTACT, the new values and a reason (history: `GET .../master-changes`). Checks: field group allowed by `SUPPLIER-MASTER-CHANGE.fieldGroups`; reason given; supplier not BLOCKED; no other change of the same group IN_APPROVAL (`CHANGE_PENDING`); a portal user only for its own supplier.
- *Validation.* TAX: GSTIN format, check character, state code and PAN embedded in the GSTIN; the PAN; optional new address (needed when the state changes); refused when the GSTIN is unchanged; other suppliers with the same GSTIN are listed as suspected duplicates in the approval evidence. ADDRESS: lines, city, PIN (6 digits), state code; a move to another GST state is refused here and must be a TAX change with the new GSTIN. CONTACT: e-mail and/or phone (9-18 digits, optional +country code). Unchanged values are refused.
- *Storage.* New values are encrypted (`after_enc`, per-tenant key); only masked before/after values are stored in clear, shown and audited. Number `SMC-nnn`, status `IN_APPROVAL`, audit `MASTER_CHANGE_REQUESTED`.
- *Approval.* Parallel `SUPPLIER_MASTER_CHANGE` tasks to the roles in `SUPPLIER-MASTER-CHANGE.approvers` (FINANCE_CONTROLLER and SUPPLIER_MANAGER), requester excluded, two different people. Any rejection: REJECTED, master unchanged.
- *Apply.* After the last approval the values are written to the supplier (TAX: encrypted GSTIN, mask, blind indexes for GSTIN and PAN, state code from the GSTIN, address if given; ADDRESS; CONTACT: e-mail, phone). A TAX change re-runs duplicate and sanctions screening; a sanctions hit blocks the supplier. Status APPLIED, audit `MASTER_CHANGE_APPLIED`, event `supplier.master.changed` (notifies the supplier managers and the supplier), and for a supplier already in the ERP a `SUPPLIER_MASTER` update message (key `{code}|{fieldGroup}|{SMC number}`).

14. **Hand-off.** Category status gates M05 invitations and award, M07 contract activation (blocked supplier refused) and M08 PO routing/release (one-time-vendor gate). Bank account and cooling-off feed the M10 payment run; SOD-02/02a bind M10 invoice posting and payment approval.

5. Document states

Category status (per supplier and category): `REGISTERED -> UNDER_QUALIFICATION -> QUALIFIED | CONDITIONAL | EXITED`; `QUALIFIED -> PREFERRED`; `QUALIFIED/PREFERRED -> CONDITIONAL` (certificate expiry); any -> `BLOCKED` (block) -> previous (unblock); any -> `EXITED`.

Supplier status: `REGISTERED -> ACTIVE` (first non-rejected qualification), `-> BLOCKED` (block/sanctions) `-> previous` (unblock), `REGISTERED -> MERGED` (duplicate merged).

Object	States
Bank change (`BCR-`)	PENDING_CALLBACK -(`POST /v1/bank-changes/{n}/callback`)-> PENDING_APPROVAL -(2 x `BANK_CHANGE` approval)-> APPROVED, or REJECTED; a second callback answers 409
Supplier master change (`SMC-`)	IN_APPROVAL -(both `SUPPLIER_MASTER_CHANGE` approvals)-> APPLIED, or REJECTED; approving a closed change answers 409
Qualification	OPEN -> DECIDED (QUALIFIED, CONDITIONAL, REJECTED)
Certificate	VALID -> EXPIRED (timer); `verified` flag
Invitation	open -> USED, REVOKED or expired
CAPA	OPEN -> RESPONDED -> CLOSED (or back to OPEN)
Development plan	DRAFT -> ACTIVE -> COMPLETED (after the review task) or CANCELLED; actions OPEN -> DONE or CANCELLED

Illegal actions answer 409 (e.g. deciding a DECIDED qualification, unblocking a supplier that is not BLOCKED).

6. Business rules and controls

Rule set	Controls
`SUPPLIER-QUAL`	`mandatoryCertificates` per category (e.g. ALU-DIECAST: IATF16949, ISO14001), `minQuestionnaireScore` 70, `minAuditScore` 75, `sampleApprovalRequired`, `bankApprovers` 2, `coolingOffHours` 72
`SUPPLIER-RISK`	`minOtifPct` 90, `maxPpm` 1000, `maxLateProb` 0.30, `certificateWarnDays` 60, `bankChangeInvoiceDays` 30, `preferredOtifPct` 95
`SUPPLIER-RISK-EXT`	risk points (Critical 40, High 25, Medium 10, Low 3), `highTierPoints` 15, `maxScore` 100, concentration, single-source and cyber thresholds
`SUPPLIER-SEGMENTATION`	12-month spend share >= 5 % and risk >= 40 -> STRATEGIC/LEVERAGE/BOTTLENECK/ROUTINE
`SUPPLIER-ASSESSMENT`	ESG/cyber questionnaire validity 365 days, minimum accept score 40
`SUPPLIER-INVITATION`	`validDays` 14
`SUPPLIER-MASTER-CHANGE`	`approvers` FINANCE_CONTROLLER + SUPPLIER_MANAGER (parallel, both required), `fieldGroups` TAX, ADDRESS, CONTACT
`MANDATORY-FIELDS`	`SUPPLIER_REGISTER` : legal_name, gstin, pan, state_code, contact_email

Field ownership of the supplier master (who may change what, and how):

Fields	Change path	Requested by	Approved by
GSTIN, PAN, state code (TAX; with the address when the GST state changes)	master change, group TAX, re-screened after apply	supplier manager or the supplier's portal user	FINANCE_CONTROLLER + SUPPLIER_MANAGER
Address within the same GST state (ADDRESS)	master change, group ADDRESS	same	same
Contact e-mail and phone (CONTACT)	master change, group CONTACT	same	same
Bank account, IFSC / SWIFT, holder, charge bearer, purpose code	bank change (call-back + 2 finance controllers + 72 h cooling-off); during onboarding via the profile	supplier manager (`supplier:bank`); supplier only during onboarding	2 different FINANCE_CONTROLLERs
Category status, segment, PREFERRED/EXITED	qualification decision, `PUT /v1/suppliers/{code}/segment`, category status command	supplier / category manager	no approval task (audited)
MSME class, TDS data	`PUT /v1/suppliers/{code}/finance`	supplier manager or finance (`payment:run`)	no approval task (audited)

Legal name: the TAX group carries GSTIN, PAN and address only; no command changes the legal name, payment terms or incoterms of an existing supplier (not found at HEAD).

Calculations: OTIF % = on-time-in-full receipts / receipts x 100; PPM = rejected qty / received qty x 1,000,000; invoice accuracy % = first-pass invoices / invoices x 100. Supplier risk score = sum of severity points of open signals (+15 for financial tier HIGH), capped at 100; BLOCKED = 100; levels LOW <15, MEDIUM 15-39, HIGH 40-69, CRITICAL >=70. Tax IDs and bank accounts are field-encrypted per tenant with blind indexes for duplicate checks; only masked values are shown, audited and put on the integration queue. ABAC: the master-change request calls action `MASTER_CHANGE` and bank tasks map to `SUPPLIER_CHANGE`, but see §11 (not effective for suppliers).

7. Integrations and events

- **ERP (`SUPPLIER_MASTER`, outbound, exactly-once integration queue, target from rule `ERP-TARGET`)**:
  - vendor create at the first non-rejected qualification (key `{code}|CREATE`; name, masked GSTIN, bank, terms); the posting stores the ERP vendor as `twin_ref` (`ERP-VENDOR-<no>`);
  - bank change of an ACTIVE supplier (key `{code}|BANK|{BCR}`);
  - tax-ID, address or contact change of a supplier already in the ERP (key `{code}|{fieldGroup}|{SMC}`); a REGISTERED supplier is not in the ERP yet, so its create message carries the current master. The queue payload holds masked values only; the SAP S/4HANA connector decrypts the approved change itself and applies it to the business partner found by search term (OData `A_BusinessPartner`, tax type `IN3` for the GSTIN; IDoc transport also supported). A failed posting is retried by the integration queue; a dead-lettered message is replayed by the integration administrator (M13).
- **Sanctions**: licensed provider over HTTPS (`PIC_SANCTIONS_*`, timeout/retries/size cap in `sanctions_feed.py`); list sync to the local table by admin; provider outage gives REVIEW, never a silent clear.
- **Risk feeds**: generic REST/JSON adapters per provider (`PIC_RISKFEED_<NAME>_URL/API_KEY/MAP/WEBHOOK_SECRET`), matched by code, GSTIN/PAN blind index or name; unmatched events kept as UNMATCHED.
- **Digital twin**: `PIC_TWIN_URL/TOKEN`; capability, evidence level, free capacity, trust, late probability; cached in `twin_snapshot`.
- **Events**: `supplier.status.changed` -> `supplier-status-flags`; `supplier.registered` -> `notify-supplier-registered`; `supplier.master.changed` -> `notify-supplier-master` (CHANGE_REQUEST notification to internal supplier managers and to the supplier's portal users); `supplier.invitation.sent`, `supplier.segment.changed`, `supplier.assessment.accepted`, `supplier.development_plan.completed` stream only. Consumed: `goods_receipt.*`, `invoice.posted`, `quality.usage_decision`, `sourcing.bid.submitted`.

8. Scheduled jobs

Job / timer	Schedule	What it does
`scorecard-monthly`	`0 3 1 * *` (1st of month, 03:00)	Recomputes the previous month's scorecards from receipts and invoices; idempotent; manual run with `{"period": "YYYY-MM"}`
timer `CERTIFICATE_EXPIRY`	day after expiry, 00:05 IST	Certificate EXPIRED; mandatory categories to CONDITIONAL unless renewed
timer `DEV_PLAN_RECHECK` and action reminders	per plan	Development plan reminders and review
task SLA timers	per task	Escalation of bank-change and mitigation tasks

The Supplier Risk Agent scan and sanctions list sync are on demand (no cron job at HEAD).

9. KPIs

Code	Name	Formula
`M06.ONBOARDING_CYCLE`	Onboarding cycle time (days)	mean time from registration to the first qualification decision
`M06.QUALIFIED_COVERAGE`	Qualified supplier coverage %	qualified or preferred supplier-category pairs / pairs not exited
`M06.SUPPLIER_OTIF`	Supplier OTIF %	receipts on time and in full / receipts
`M06.SUPPLIER_PPM`	Supplier PPM	rejected qty / received qty x 1,000,000
`M06.HIGH_RISK_SUPPLIERS`	High-risk suppliers %	active suppliers with risk tier HIGH / active suppliers
`M03.BANK_FRAUD_BLOCKED`	Bank-change fraud attempts blocked	bank-change requests rejected at call-back or approval

10. Demo data

- **AXN signature case**: TF001, KD002, SLA003 qualified for ALU-DIECAST with bank accounts and certificates.
- **D1 `axn-branch-cases` (AXN)**: case 15 invitation -> self-registration -> profile -> bank dual approval -> qualification REJECTED (Vellore Heat Treaters); case 16 ISO 14001 expiry -> ALU-DIECAST CONDITIONAL (AAC202); case 33 bank change rejected by the finance controller (AAC202, IFSC UTIB0003912); case 28 rejected lot -> CAPA -> response -> verified.
- **D1 `axn-module-cases` (AXN)**: invitation and onboarding of Krishnagiri Returnable Packaging; MDM-04 TAX change with a state move (new GSTIN and address) approved, ADDRESS change of APC007 rejected, CONTACT change raised by APC007's own portal user and approved (`SMC-` numbers on each supplier's master data card).
- **KAC** (`erp-client-kac`): suppliers migrated from the mock S/4HANA extract.

11. Known limits and open items

- No scheduled risk scan or sanctions re-screening: both run on demand.
- Licensed sanctions, risk feed and twin providers are not connected; the local sanctions list is fictitious sample data (`SAMPLE-LIST`).
- Onboarding questionnaires are scores and free JSON; there is no questionnaire builder.
- Supplier-level exit is per category (no supplier-wide EXITED command).
- ABAC does not restrict supplier master governance in practice: `MASTER_CHANGE` checks company code and plant, which a supplier record does not carry, and the `BANK_CHANGE` / `SUPPLIER_MASTER_CHANGE` tasks are not mapped to a document in `abac._document`, so the `SUPPLIER_CHANGE` category check never runs.
- After onboarding a supplier cannot request its own bank change from the portal (`supplier:bank` only); the supplier manager raises it.
- No change path for legal name, payment terms or incoterms of an existing supplier.
- `docs/PIC_Pending_Workflows.md` still lists monthly scorecards, segmentation, development plans, responsiveness/ESG measures and geographic/single-source/cyber signals as not built; the code has them (doc outdated).

M07 Contract Lifecycle Management

1. Purpose and scope

M07 authors, approves, executes and tracks buy-side contracts and makes executed contracts the source of price and terms for POs (Blueprint §13, flow F4). Types: SCHEDULING_AGREEMENT, QUANTITY_CONTRACT, VALUE_CONTRACT, RATE_CARD, SOW (call-off types with lines) and MASTER_AGREEMENT, NDA (framework types without lines or prices). Code: `services/contract.py` (process `pic.contract`, steps F4-02 to F4-10), `amendments.py`, `rebates.py`, `clause_extract.py`, `seed/clause_library.py`; API `api/source_contract.py`, `api/source_ops.py`, `api/governance.py`, `api/wave1_c.py`.

2. Roles

Role	What they do in M07	Scopes
CONTRACT_MANAGER	Creates manual contracts, runs the review, reviews supplier redlines, submits, signs as buyer, activates, index resets, obligations, amendments, termination, rebates; approver below the threshold; decides renewal tasks	`contract:manage`, `contract:sign`
LEGAL, FINANCE_CONTROLLER, CPO	Approvers above the threshold or for non-standard clauses; amendment and termination approvers	`tasks:work`
CATEGORY_MANAGER / event owner	Owner of contracts born from an award	
SUPPLIER_USER	Views own non-draft contracts, proposes redlines, signs in the portal	`supplier:portal`
FINANCE_CONTROLLER	Settles rebates (also contract manager)	`payment:run`

Four-eyes: the contract owner and the submitter are excluded from `CONTRACT_APPROVAL` tasks; the amendment requester cannot approve it (`NOT_YOUR_TASK`). ABAC `CONTRACT_SIGN` (company code + value limit) on signing and on contract approval tasks.

3. Screens

Screen (BP)	Route under /pic	What it shows	Actions
CLM-01 Contract Workspace (repository)	`/contracts`	Contracts by status, type, supplier	Open, new
CLM-01 (manual contract)	`/contracts/new`	Type, header (supplier, company code, validity, terms, jurisdiction, category overlay, master agreement), lines	Create draft
CLM-01/02/03 contract page	`/contracts/[ref]`	Header, lines, tabs Clauses (CLM-02 review and redlines), Signatures, Consumption & obligations (CLM-03), Amendments, Rebates, Clause extraction	Review, redline review, Submit for approval, Sign as buyer, Activate, Index reset, Add obligation, Amend, Terminate

Screen (BP)	Route under /pic	What it shows	Actions
CLM-04 Obligation Tracker	`/contracts/obligations`	Buyer/supplier obligations and milestones (open, due soon, overdue, completed, on-time %), renewal calendar	Mark complete
Approvals	`/tasks`	`CONTRACT_APPROVAL`, `CONTRACT_AMENDMENT`, `CONTRACT_RENEWAL`, `TERMINATED_CONTRACT_REVIEW`	Approve, reject, return
SLM-05 portal (Contracts tab)	`/portal`	Supplier's contracts	Propose redline, sign

4. End-to-end workflow

1. **Trigger (F4-01).** Either the award consumer `contract-from-award` (from M05) or a contract manager, `/contracts/new`, `POST /v1/contracts` (idempotent). Checks: `valid_to` after `valid_from`; call-off types need lines, framework types refuse lines, prices, index and framework reference (`CLM-FRAMEWORK`); a call-off under a master agreement must be with the same supplier and company code, inside its validity, and the master must be ACTIVE/EXPIRING; index reference needs index, base and band.
2. **Draft (F4-02).** Clauses from the `CLM-PLAYBOOK` library for the type, overlaid by category clauses and the governing-law (jurisdiction) variant; standard obligations per type with due dates from `valid_from`; each line gets a price-validity period (reason AWARD or INITIAL, not yet published). Status `DRAFT`, number from the `SA-` series. Templates: `GET /v1/contract-templates`.
3. **Review (F4-03).** Contract manager, `POST /v1/contracts/{ref}/review`: Contract Review agent (L2, deterministic against the playbook) scores every clause (STANDARD 0, FALLBACK 30, NON_STANDARD 70; reworded STANDARD counts as fallback) and suggests fallbacks. Status `IN_NEGOTIATION`. `GET .../clause-review` (CLM-02) shows each clause vs standard text, pending supplier redlines and the approval route.
4. **Redline (F4-04).** Supplier, portal, `POST .../redlines`: always NON_STANDARD and pending review. Contract manager redlines with a position. Contract manager reviews each supplier redline, `POST .../redlines/{clause}/review` ACCEPT (with a position) or RESTORE. Optional LLM suggestion: `POST .../clauses/{clause}/redline-suggestion`.
5. **Submit (F4-05/06).** `POST .../submit` (If-Match). Refused while a supplier redline is unreviewed (`REDLINE_REVIEW_PENDING`). Route from `CONTRACT-APPROVAL`: value >= `thresholdMinor` (1 crore) or any NON_STANDARD clause -> LEGAL -> FINANCE_CONTROLLER -> CPO; otherwise CONTRACT_MANAGER. Status `IN_APPROVAL`. Approved -> `OUT_FOR_SIGNATURE`; rejected -> back to `IN_NEGOTIATION`.
6. **Sign (F4-07).** Buyer (`contract:sign`, `POST .../signatures` party BUYER) and supplier (portal user, party SUPPLIER) each sign once; the signature stores the SHA-256 of the document (number, supplier, validity, terms, clause texts, line quantities). With a provider: `POST .../signature-envelopes` creates an envelope (DocuSign, Adobe Sign, Leegality or eMudhra via `PIC_ESIGN_PROVIDER/URL/API_KEY/WEBHOOK_SECRET`); the HMAC-signed webhook `POST /v1/esign/webhook` records each signer. `MOCK` provider only outside production, labelled "Demo signature - not legally binding".
7. **Activate (F4-08).** Contract manager, `POST .../activate` (If-Match). Checks: still OUT_FOR_SIGNATURE; validity not ended; supplier not BLOCKED/EXITED/REJECTED/MERGED; framework still valid; both signatures with the same hash equal to the current text. Result: `ACTIVE`, price validities **published** to source determination, quota arrangement line per awarded contract (share = target qty / total of the award's contracts), timers `CONTRACT_RENEWAL` (valid_to - `renewalNoticeDays`) and `CONTRACT_EXPIRY`; event `contract.activated` (source list entry, savings CONTRACTED).
8. **Consume (F4-09).** Every `purchase_order.released/changed/cancelled/closed` recomputes released quantity/value per contract line (RELEASED to INVOICED count in full; CLOSED counts what was delivered). Thresholds 80/90/100 % (`consumptionThresholds`) emit `contract.consumption.threshold` once per line (notification CONTRACT_CONSUMPTION). `GET .../consumption` (CLM-03) shows released vs target, off-contract buying and price leakage, compliance rate and obligations.
9. **Index reset.** `POST .../index-resets` (If-Match) or `/index-resets/from-feed`: always a new price-validity period after the last reset; within the band the price is unchanged (INDEX_RESET_ABSORBED; on an award contract the avoided increase is booked as cost avoidance in M11), outside it the price moves by the excess (INDEX_RESET_ADJUSTED). Never overwrites a period.

10. **Obligations and milestones.** `POST .../obligations` (party BUYER/SUPPLIER, kind OBLIGATION or milestone), `POST /v1/obligations/{id}/complete` with evidence note; on time if completed by the due date.
11. **Amend (CLM-AMEND).** `POST .../amendments/preview` then `POST .../amendments` (kind VARIATION, EXTENSION, TERMS or TERMINATION), submit/withdraw. Approval chain from `CONTRACT-AMENDMENT`: TERMINATION -> `termination` chain (LEGAL, CPO); any NON_STANDARD clause -> LEGAL + `above`; |value delta| >= 1 crore -> LEGAL, FINANCE_CONTROLLER; else CONTRACT_MANAGER. Approval is stale if the contract version moved (`AMENDMENT_STALE`). Applied on the effective date (immediately or timer `CONTRACT_AMENDMENT_EFFECTIVE`): contract `version` + 1, new price periods (old ones never re-priced), event `contract.amended`.
12. **Renew / expire / terminate (F4-10).** At the renewal notice date the contract becomes `EXPIRING` and a `CONTRACT_RENEWAL` task goes to CONTRACT_MANAGER; approving drafts an EXTENSION amendment, which once approved returns the contract to `ACTIVE` with a later end date. Expiry timer: `EXPIRED`, pending amendments lapse, open renewal task cancelled. Termination: `POST .../terminate` (reason, `for_cause`; default effective date = today + `noticeDays` 90, today for cause) creates a TERMINATION amendment; when applied: `TERMINATED`, prices and quota end, open POs get a `TERMINATED_CONTRACT_REVIEW` buyer task, `contract.terminated`.
13. **Rebates.** `POST .../rebates` (basis VALUE or VOLUME, period MONTHLY/QUARTERLY (Indian FY)/ANNUAL, rising tiers, RETROACTIVE or SLAB, scope CONTRACT or SUPPLIER). Accrual recomputed from posted invoices net of credit notes (consumer on `invoice.posted`, `POST /rebates/{ref}/accrue`). Settlement after the period ends (`POST /rebate-accruals/{id}/settle`): vendor credit memo to the ERP once, event `rebate.settled`.
14. **Migrated contracts.** `POST /v1/clause-extractions` extracts clauses from signed contract text; items at or above `CLAUSE-EXTRACTION.autoApplyAbove` (0.85) apply directly during migration, the rest are reviewed (`review`, `apply`, `reject`).

5. Document states

From	Command (route)	To	Guard
DRAFT	review (`POST .../review`)	IN_NEGOTIATION	
IN_NEGOTIATION	submit (`POST .../submit`)	IN_APPROVAL	no unreviewed supplier redline
IN_APPROVAL	approve (last `CONTRACT_APPROVAL` task)	OUT_FOR_SIGNATURE	four-eyes, ABAC CONTRACT_SIGN
IN_APPROVAL	reject	IN_NEGOTIATION	
OUT_FOR_SIGNATURE	activate (`POST .../activate`)	ACTIVE	both signatures, same hash, supplier not blocked
ACTIVE	expiring (timer `CONTRACT_RENEWAL`)	EXPIRING	renewal notice date
EXPIRING	extend (approved EXTENSION amendment)	ACTIVE	
ACTIVE, EXPIRING	expire (timer `CONTRACT_EXPIRY`)	EXPIRED	valid_to passed
ACTIVE, EXPIRING	terminate (approved TERMINATION amendment)	TERMINATED	effective date reached
EXPIRING	renew	RENEWED	defined, no caller at HEAD

Amendment: DRAFT -> IN_APPROVAL -> APPROVED -> EFFECTIVE; rejected or withdrawn/lapsed -> REJECTED; returned -> DRAFT. Rebate agreement ACTIVE/CANCELLED; accrual OPEN -> SETTLED; settlement PENDING_ERP -> POSTED. Illegal transitions answer 409.

6. Business rules and controls

Rule set	Controls
`CLM-PLAYBOOK`	clause templates per type, `categoryClauses`, jurisdiction variants, `positionRisk` (STANDARD 0, FALLBACK 30, NON_STANDARD 70), `fallbacks`, obligations per type, `awardDefaults` (tolerances over 2 %, under 5 %, price 0.5 %; index LME_AL base 100, band 3 %, quarterly)
`CONTRACT-APPROVAL`	`thresholdMinor` 1 crore, `above` LEGAL/FINANCE_CONTROLLER/CPO, `below` CONTRACT_MANAGER, `renewalNoticeDays` 120, `consumptionThresholds` [80, 90, 100]
`CONTRACT-AMENDMENT`	`thresholdMinor` 1 crore, `above`, `below`, `termination` LEGAL/CPO, `noticeDays` 90
`CLAUSE-EXTRACTION`	`autoApplyAbove` 0.85

- **Only executed contracts are referenced**: a PO with `contractRef` needs the contract ACTIVE or EXPIRING, a line for the material and plant, a published price valid on the need date, and committed quantity (released + open drafts + this order) within target x (1 + over-tolerance %) (`CLM-PO`).
- **Price tolerance**: a PO price override on a contract line deviating more than `price_tolerance_pct` needs a deviation reason and is flagged (`P07`).
- **Contract risk** = highest clause risk; band HIGH >= 70, MEDIUM >= 30.
- **Consumption** % = released qty / target qty x 100 per line. **Compliance rate** = released on contract / (on contract + off-contract) for the contracted items; **price leakage** = paid above contract price on off-contract and flagged contract-priced lines.
- **Index reset**: change % = (index / base - 1) x 100; adjustment = 0 within the band, else change minus band (signed); new price = negotiated price x (1 + adjustment %).
- **Rebate**: RETROACTIVE applies the reached tier rate to the whole basis; SLAB applies each tier's rate to the part inside it.
- If-Match required on submit, activate, terminate, index-resets and amendments (`preconditions.py`; enforced when `PIC_REQUIRE_IF_MATCH` is on, which it is in production).

7. Integrations and events

- **E-signature**: provider envelopes and HMAC webhook (`ESIGN_SIGNATURE_INVALID` 401, `DOCUMENT_CHANGED` 409 when the signed hash differs); internal signatures otherwise.
- **ERP**: rebate settlement `SUPPLIER_CREDIT_MEMO` (idempotency key `REBATE|{agreement}|{period}`). Contracts themselves are not posted to the ERP; ERP outline agreements arrive with the client migration (`erp-client-kac`).
- **Events emitted**: `contract.activated` -> `sourcing-source-list`, `savings-contracted`; `contract.consumption.threshold` -> `notify-contract-consumption`; `contract.amended` -> `notify-contract-amended`; `contract.terminated` -> `notify-contract-terminated`; `rebate.settled` stream only. **Consumed**: `sourcing.event.awarded` (`contract-from-award`), `purchase_order.released/changed/cancelled/closed` (`contract-consumption`), `invoice.posted` (`rebate-accrual`).

8. Scheduled jobs

No cron job. Timers: `CONTRACT_RENEWAL` (09:00 IST on valid_to - 120 days), `CONTRACT_EXPIRY` (00:05 IST the day after valid_to), `CONTRACT_AMENDMENT_EFFECTIVE` (00:01 IST on the effective date), task SLA timers on approval and renewal tasks. Rebate accrual is event-driven and recomputed on read (`accrue_all` has no scheduler).

9. KPIs

Code	Name	Formula
`M07.CONTRACT_CYCLE`	Contract cycle time (days)	mean time from contract draft to activation
`M07.CONTRACT_LEAKAGE`	Contract leakage (minor units)	value of open off-contract and off-contract-price hits (spend cube)
`M07.EXPIRING_90D`	Contracts expiring in 90 days	active contracts with valid-to within 90 days
`M07.OBLIGATION_ON_TIME`	Obligation on-time %	obligations met on time / obligations due

10. Demo data

- **AXN signature case**: `SA-5520` and the next contract, scheduling agreements born from RFQ-2231 (TF001 60 %, KD002 40 %), approved LEGAL -> FINANCE_CONTROLLER -> CPO, signed, active, quota 60/40, index reset 1 Jan 2027 absorbed; PO-77410 is a release from SA-5520.
- **D12 `contracts-kac` (KAC)**: 60 MOCK contracts from `backend/data/erp_mock/KAC_CLM`: master agreements with call-offs (e.g. `KAC-MA-0001`), NDAs, rate cards, SOWs, obligations, rebates, amendments, renewals; `KAC-RC-0007` and `KAC-SOW-0001` in negotiation with redlines, `KAC-VC-0004` terminated.
- **D1 `axn-module-cases`**: C1 variations below/above 1 crore, C2 renewal -> EXTENSION, C3 termination with an open PO flagged, C4 extension. **D1 `axn-branch-cases`**: case 20 approval rejected -> redlines -> resubmitted; case 21 consumption alerts 80/90/100 %.
- **D17 `sourcing-ops`**: 22 award contracts in KAC (`SA-55nn`), 5 active.

11. Known limits and open items

- `RENEWED` has no caller: renewal extends the same contract; successor-contract renewal is a product decision.
- No cancel/withdraw state for a contract that never reached signature.
- Several live contracts for one material and plant: source determination takes the first row of an unordered query (no precedence rule).
- PO quantity against the contract is checked at PO creation, not again at release; value contracts consume by quantity x price.
- The e-signature path was exercised only with the MOCK provider; the clause library is an illustrative sample until legal supplies the real one.
- `NOTIFICATIONS` lists kind `CONTRACT_EXPIRING`, but nothing produces it; renewal is signalled through the `CONTRACT_RENEWAL` task.
- `docs/PIC_Module_Gap_Analysis.md` still lists the external e-signature provider and clause extraction as missing, and `docs/PIC_Pending_Workflows.md` lists master agreement/NDA, jurisdiction templates, rebates and clause extraction as not built; the code has them (docs outdated).

M08 Purchase Orders & Supplier Collaboration

1. Purpose and scope

M08 creates, validates, approves, transmits and manages POs that inherit price and terms decided upstream (Blueprint §14, flow F5, Technical Document §9.2 source determination). It covers creation from requisitions (manual, catalogue or MRP origin), releases against contracts, source determination, price and supplier checks, risk scoring and touchless release, approval by value and deviation, transmission (portal, EDI, e-mail), supplier confirmation, change orders with versions, delivery schedules, expediting, cancellation/closing and the hand-off to ASN and goods receipt (M09). Code: `services/order.py` (process `pic.po`, steps F5-02 to F5-10), `po_controls.py`, `source.py`, `transmission.py`; API `api/order_pay.py`, `api/order_ops.py`, `api/governance.py`.

2. Roles

Role	What they do in M08	Scopes
BUYER	Converts requisition lines to POs, overrides price with a reason, changes/cancels/closes, decides supplier change requests and confirmation reviews, expedites; approver in every PO band; buyer-lead deviation approver	`po:create`, `po:release`, `po:change`, `req:buyer`
PROCUREMENT_HEAD	Second approver above 5 lakh; one-time-vendor approver	`tasks:work`
SUPPLIER_USER	Confirms (with dates per schedule line), rejects, requests changes, sends ASN	`po:confirm`, `asn:create`, `supplier:portal`
TOUCHLESS_PO agent	Releases a PO that meets DMN-02 (autonomy L4, value limit, SoD checked)	`po:release`
EXPEDITING agent	Drafts follow-up text when the confirmation SLA is missed (L2)	

SoD: **SOD-01** (supplier creator may not create its POs; guarded at PO creation); **SOD-03** (PO creator may not post the GR) and **SOD-04** (PO creator may not approve invoice exceptions) are recorded here and enforced in M09/M10. Four-eyes on approval chains. ABAC: PO approval tasks check `PO_APPROVE` (company code, plant, value limit).

3. Screens

Screen (BP)	Route under /pic	What it shows	Actions
PO-01 PO Workbench	`/buyer` (REQ-04 buyer workbench) and `/purchase-orders`	Approved requisition lines waiting for a PO (MRP and manual); PO worklist	Create purchase order
PO-02 PO Detail & Validation	`/purchase-orders/[ref]`	Lines with inherited references, risk score and factors, touchless reasons, deviations; tabs Schedule, Transmissions, Retention, Subcontracting	Change order, Cancel order, Close order; supplier: Confirm, Confirm with dates, Send change, Send ASN
PO-03 Confirmations & Changes	`/purchase-orders/changes`	Awaiting confirmation, confirmations, change requests, version history	Open PO, decide via tasks

Screen (BP)	Route under /pic	What it shows	Actions
PO-04 Expediting Cockpit	`/purchase-orders/expediting`	Open schedule lines ranked by days late and twin late-delivery probability	Expedite (drafted message)
Approvals	`/tasks`	`PO_APPROVAL`, `PO_CHANGE_APPROVAL`, `PO_CHANGE_REVIEW`, `PO_CHANGE_REQUEST`, `EXPEDITE`, `BLOCKED_SUPPLIER_PO`, `TERMINATED_CONTRACT_REVIEW`	Approve, reject
SLM-05 portal (Orders tab)	`/portal`	Supplier's POs	Confirm, Confirm dates, Request change, Send ASN

4. End-to-end workflow

1. **Create (F5-01/02).** Buyer, `/buyer`, `POST /v1/purchase-orders` with `source` {type REQUISITION, id, lines}, optional `supplierId`, `contractRef`, `release` {qty, needDate}, `unitPriceMinor` + `deviationReason`, `oneTimeVendorReason`, `retentionPct` (Idempotency-Key). MRP demand arrives as requisitions from the MRP run (M02, process `pic.mrp-to-pr`) and is converted the same way. Checks: requisition APPROVED/PARTIALLY_ORDERED and release qty <= open qty (**P07**); organisation records active.
   - **With `contractRef`** (release order against a contract): see M07 §6 (`CLM-PO`); price = published validity on the need date.
   - **With `supplierId`**: manual or catalogue price.
   - **Neither**: **source determination** (`source.determine`): 1 single active contract line -> 2 quota arrangement (quantity split by share, prices from each supplier's contract, allocated counters updated) -> 3 source list (fixed first) -> 4 info record -> 5 none: `NEEDS_SOURCING` (422, start a sourcing event, M05). Suppliers BLOCKED or not QUALIFIED/PREFERRED for the material's category are excluded and listed. A free-text line needs a supplier or a sourcing event (`P03`). One PO per supplier is created (`DRAFT`, number `PO-` series), with lines inheriting price, payment terms, incoterms and tolerances (contract tolerances, else `RCV-TOLERANCE`), purchase-type flags from `PURCHASE-TYPE-MATRIX` (GR, IR, SES, ERS, receipt-less expense below the limit), default schedule line, lineage links. SoD-01 guard.
2. **Price checks.** Contract-priced override outside `price_tolerance_pct` needs `deviationReason` (`P07`). A line not priced by a contract is checked by `PO-PRICE-CHECK`: above the contract price of the material by more than 2 % or above the last price paid by more than 5 % (12-month lookback) needs a deviation reason and adds the buyer-lead approver.
3. **Validate (F5-03).** Automatic (`POST .../validate` re-runs it): PO risk score from `PO-RISK` (late-delivery probability from the twin, price deviation, supplier not qualified, value above band, split pattern, account assignment changed); split detection by cost object (`PO-RISK` window/band) and by supplier + material (`PO-SPLIT`: 3 orders below 2 lakh within 7 days). PO Validation agent run (L3). Audit `VALIDATED`.
4. **Touchless decision (F5-04).** `DMN-02`: price sources only CONTRACT/CATALOGUE, supplier QUALIFIED/PREFERRED, value <= 25 lakh, no split, late probability < 0.30, risk < 30, account assignment unchanged; and the `TOUCHLESS_PO` agent is allowed (L4, value, SoD). If yes: released by the agent, step F5-05 skipped.
5. **Route and approve (F5-05).** Blocked supplier refused (`PO_SUPPLIER_BLOCKED`, SLM-BLOCK); mandatory fields at release (`MANDATORY-FIELDS.PO_RELEASE`: payment terms, purpose code; lines GST rate, need date, plant); **one-time-vendor gate**: a supplier not QUALIFIED/PREFERRED for the category is refused unless `ONE-TIME-VENDOR` allows it (value <= 5 lakh, reason >= 10 characters, approver PROCUREMENT_HEAD added; BLOCKED/MERGED never; never touchless). Status `IN_APPROVAL`; `PO_APPROVAL` tasks by `PO-APPROVAL` band (<= 5 lakh BUYER; above BUYER + PROCUREMENT_HEAD) plus `deviationChain` for deviations. Rejection -> `DRAFT`; buyer corrects and `POST .../resubmit` (reason, If-Match).
6. **Release and transmit (F5-06).** On the last approval: `RELEASED` (event `purchase_order.released`: ERP posting `PURCHASE_ORDER`, contract consumption), then immediately `TRANSMITTED` over the supplier's channels (`order_channel`): PORTAL always, EDI (EDIFACT D.96A ORDERS, ORDCHG for later versions) and e-mail when configured. Each transmission stores the document and its SHA-256 (`GET .../transmissions`, resend). Event `purchase_order.transmitted` (notification PO_TRANSMITTED, delivery consumer). Confirmation due = now + `confirmationSlaHours` (24); timer `PO_CONFIRMATION_SLA`.
7. **Supplier confirmation (F5-07/08).** Supplier, portal (`POST .../confirmations`) or inbound EDI ORDRSP (`POST /v1/edi/inbound`). Confirmed dates per schedule line. Within tolerance -> `CONFIRMED`, schedule lines get confirmed dates, `purchase_order.confirmed`. Rejection, later date, quantity outside the line tolerance or a price change -> confirmation `CHANGE_REQUESTED` and a `PO_CHANGE_REVIEW` task for the buyer: accept (becomes a change order) or decline. SLA missed -> `EXPEDITE` task for the buyer with an agent-drafted message.

8. **Change orders (versions).** Buyer, `POST .../versions` (If-Match): quantity (never below received), price, need date, schedule. A price outside the contract tolerance or the price-check band needs a reason; with `PO-APPROVAL.changeApproval` the change waits for `PO_CHANGE_APPROVAL` (price deviation always; band increase when enabled) before version n+1 exists. Applying: snapshot of version n kept (`po_version`), version + 1, confirmed dates cleared, re-sent (ORDCHG on EDI), new SLA timer, `purchase_order.changed` (ERP update, consumption), status back to `TRANSMITTED` from CONFIRMED. Supplier or buyer change requests on qty/date: `POST .../change-requests` -> `PO_CHANGE_REQUEST` task -> accepted request becomes a change order.
9. **Delivery schedule.** `PUT .../lines/{line_no}/schedule` (If-Match): edited in place before confirmation or while transmissions are still queued; afterwards it is a new version that the supplier re-confirms.
10. **Hand-off to receiving (F5-09).** Supplier sends the ASN (`POST /v1/asns` or EDI DESADV) with batches and, above `EWAY-BILL.requiredAboveMinor` (50,000), an e-way bill -> M09 goods receipt -> `PARTIALLY_RECEIVED` / `RECEIVED` -> M10 invoice -> `INVOICED` -> `CLOSED`. Service POs (all lines SES) end at F5-10 after confirmation; performance is confirmed by service entry sheets.
11. **Cancel / close.** `POST .../cancel` (reason) from DRAFT, IN_APPROVAL, RELEASED, TRANSMITTED, CONFIRMED when nothing was shipped, received or invoiced and no service entry is open: tasks cancelled, requisition quantity re-opened, cancellation sent over the same channels, ERP cancellation, `purchase_order.cancelled` . `POST .../close` from INVOICED/RECEIVED; `POST .../short-close` from PARTIALLY_RECEIVED (`purchase_order.closed`).
12. **Exceptions from other modules.** Supplier blocked -> `BLOCKED_SUPPLIER_PO` buyer review; supplier status change -> PO audit flag and notification; contract terminated -> `TERMINATED_CONTRACT_REVIEW` task.

5. Document states

From	Command	To
DRAFT	route (creation, not touchless)	IN_APPROVAL
DRAFT, IN_APPROVAL	release (touchless or last approval)	RELEASED
IN_APPROVAL	reject	DRAFT
RELEASED	transmit (automatic)	TRANSMITTED
TRANSMITTED	confirm (`POST .../confirmations`)	CONFIRMED
RELEASED, TRANSMITTED	change (`POST .../versions`)	same status
CONFIRMED	change	TRANSMITTED (re-confirmation)
CONFIRMED, TRANSMITTED, PARTIALLY_RECEIVED	receive_partially / receive (GR, M09)	PARTIALLY_RECEIVED / RECEIVED
PARTIALLY_RECEIVED	change, confirm	PARTIALLY_RECEIVED
RECEIVED; CONFIRMED (receipt-less expense)	invoice (M10)	INVOICED
INVOICED, RECEIVED	close (`POST .../close`)	CLOSED
PARTIALLY_RECEIVED	short_close (`POST .../short-close`)	CLOSED
DRAFT, IN_APPROVAL, RELEASED, TRANSMITTED, CONFIRMED	cancel (`POST .../cancel`)	CANCELLED
RECEIVED, PARTIALLY_RECEIVED	unreceive / unreceive_unconfirmed (GR reversal 102)	CONFIRMED, PARTIALLY_RECEIVED or TRANSMITTED

Confirmation record: CONFIRMED or CHANGE_REQUESTED per version. Transmission: QUEUED, SENT, ACKNOWLEDGED, FAILED. Illegal transitions answer 409.

6. Business rules and controls

Rule set	Controls
`DMN-02`	touchless: `priceSources` CONTRACT/CATALOGUE, `supplierStatuses` QUALIFIED/PREFERRED, `touchlessBandMinor` 25 lakh, `maxLateProb` 0.30, `maxRisk` 30
`PO-RISK`	additive points: late prob >= 0.15 -> 18, >= 0.30 -> 35; price deviation 18; supplier not qualified 40; value above 25 lakh 15; split 25 (window 7 days, band 2 lakh); account assignment changed 12
`PO-APPROVAL`	bands (<= 5 lakh BUYER; above BUYER, PROCUREMENT_HEAD), `deviationChain` BUYER, `confirmationSlaHours` 24, `changeApproval` (`onPriceDeviation`, `onBandIncrease`)
`PO-PRICE-CHECK`	`contractBandPct` 2, `lastPriceBandPct` 5, `lastPriceBasis` SUPPLIER_THEN_MATERIAL, `lookbackMonths` 12
`PO-SPLIT`	`windowDays` 7, `minOrders` 3, `perOrderBelowMinor` 2 lakh
`ONE-TIME-VENDOR`	`maxValueMinor` 5 lakh, `minReasonLength` 10, `approverRoles` PROCUREMENT_HEAD, `blockedStatuses` BLOCKED/MERGED
`MANDATORY-FIELDS`	`PO_RELEASE` header and line fields
`PURCHASE-TYPE-MATRIX`, `RCV-TOLERANCE`	GR/IR/SES/ERS flags per purchase type; default receipt and price tolerances

Calculations: PO risk = sum of the triggered points (max 100); a deviation = |new price - reference| > reference x tolerance %; quota allocation part = qty x share/total share (last supplier takes the rest). Idempotency-Key on PO create and ASN; If-Match required on `versions`, `resubmit` and schedule changes; ETag on `GET /v1/purchase-orders/{ref}`. A supplier user sees only its own POs (404 otherwise).

7. Integrations and events

Integration	Direction	Detail
ERP `PURCHASE_ORDER`	outbound	on release and every change (idempotency key `{company}
EDI EDIFACT D.96A	outbound ORDERS/ORDCHG, cancellation; inbound ORDRSP (confirmation), DESADV (ASN)	gateway `PIC_EDI_GATEWAY_URL/TOKEN`; without it transmissions stay QUEUED and can be resent
E-mail	outbound	`platform.mailer` to `order_channel.email`; without SMTP stays QUEUED
Supplier portal	both	always used; confirmations, change requests, ASN

Events emitted: `purchase_order.released` -> `erp-po-posting`, `contract-consumption`; `purchase_order.changed` -> same; `purchase_order.transmitted` -> `notify-po-transmitted`, `po-transmission-delivery`; `purchase_order.confirmed` -> `notify-po-confirmed`; `purchase_order.cancelled` -> `erp-po-posting`, `contract-consumption`, `notify-po-cancelled`, `po-transmission-

`delivery`; `purchase_order.closed` -> `contract-consumption`. Consumed: `supplier.status.changed` (PO flags). Notifications: PO_TRANSMITTED, PO_CANCELLED, CHANGE_REQUEST, TASK_ASSIGNED.

8. Scheduled jobs

No cron job. Timer `PO_CONFIRMATION_SLA` per PO version (fires at the due time; opens an `EXPEDITE` task if the current version is still unconfirmed and unanswered); task SLA timers on approval tasks. ERP postings of POs retry through the integration queue (the nightly `erp-reconciliation` job covers invoices, payments and credit memos, not POs).

9. KPIs

Code	Name	Formula
`M08.TOUCHLESS_PO`	Touchless PO %	POs released without human touch / POs released
`M08.PO_CYCLE`	PO cycle time (hours)	mean time from PO creation to release
`M08.CONFIRMATION_SLA`	Confirmation rate within SLA %	confirmations received before the due time / confirmations
`M08.PO_CHANGE_RATE`	PO change rate %	released POs with more than one version / released POs

All four can be read per tenant and per plant.

10. Demo data

- **AXN signature case**: **PO-77410**, release from `SA-5520` to TF001 (5,400 @ 398), and the KD002 order (3,600 @ 405) from the 60/40 quota, both touchless (PO risk 18/100 per the Blueprint design note) and confirmed; ASN, GR and invoice INV-TF-8812 follow.
- **D1 `axn-branch-cases` (AXN)**: case 25 supplier rejects -> PO CANCELLED -> re-sourced, confirmation SLA breach -> Expediting task; case 26 later date accepted (v2, re-confirmed) and price change declined; case 32 PO approval rejected -> DRAFT -> cancelled and re-raised; case 34 supplier change request after confirmation rejected.
- **D1 `axn-module-cases` (AXN)**: APC007 DC-352 PO: approval, transmission PORTAL + EDI + e-mail (queued), inbound EDI ORDRSP, schedule change -> version 2 + ORDCHG -> re-confirmed, inbound DESADV -> ASN with e-way bill.

11. Known limits and open items

- Only `source.type = REQUISITION` is accepted on `POST /v1/purchase-orders`: there is no direct PO from a contract or MRP planned order without a requisition, and no free manual PO without one.
- Scheduling-agreement "schedule lines" are delivery schedules on each release PO; there is no standing scheduling-agreement schedule document.
- ABAC actions `PO_CREATE` and `PO_RELEASE` exist in the policy but the PO create/release commands do not call them at HEAD; ABAC applies to PO approval tasks (`PO_APPROVE`).
- Purchasing organisation is fixed to `1000` and currency to INR on PO creation.
- Contract quantity is checked at creation, not at release (see M07).
- `docs/PIC_Module_Gap_Analysis.md` says M08 transmission is "portal transmission only"; the code sends EDI ORDERS/ORDCHG and e-mail and reads ORDRSP/DESADV (doc outdated). `docs/PIC_Pending_Workflows.md` still lists the one-time-vendor hard block and the +5 % last-price check as partial; both are built.

M09 Receiving, Quality & Inventory

1. Purpose and scope

M09 (Blueprint §15, flow F6 "Goods receipt / service entry", process key `pic.receipt`) confirms that what was ordered actually arrived or was performed: the supplier's advance shipping notice (ASN), the goods receipt (GR) with tolerances, e-way bill, batches, serials and expiry, quality inspection and the usage decision, returns and replacements, receipt reversal, service entry sheets (SES), and the special stocks (consignment, subcontracting). Every receipt updates stock, GR/IR and the supplier scorecard and is sent to the ERP. The inventory part (no stand-alone module in the Blueprint; built on top of M09, BP §22-23, MRP-02, TD §9.1) is the stock ledger and everything that reads or changes it: stock and movement reads, alerts, governed stock postings (transfer, block, unblock, scrap), physical inventory counts and reservations.

2. Roles

Role	What they do here	Key scopes	SoD / notes
SUPPLIER_USER (portal)	Creates ASNs with e-way bill data; enters service entry sheets	`asn:create`, `ses:create`	Only on their own supplier's orders (`SUPPLIER_SCOPE`, 403)
RECEIVER (stores clerk)	Posts goods receipts and reversals, provides subcontracting components, withdraws consignment stock, creates stock postings and reservations, enters counts	`gr:post`, `inventory:post`, `inventory:count`	SOD-03: may not post the GR of a PO they created. SOD-11: `inventory:count` and `inventory:approve` cannot be held by one person, so a RECEIVER cannot also be INVENTORY_CONTROLLER or FINANCE_CONTROLLER
QUALITY_INSPECTOR	Takes the usage decision on inspection lots, requests replacements, recalls lots	`quality:decide`	
SERVICE_OWNER	Approves service entry sheets	`ses:approve`	Approver must differ from the supplier user who entered the SES and from the PO creator
INVENTORY_CONTROLLER	Creates/submits/posts stock postings, approves postings and count differences, orders recounts	`inventory:post`, `inventory:approve`	SOD-10 (per document): the maker or submitter of a posting never approves it; SOD-11: a counter never approves the count
FINANCE_CONTROLLER	Second approver in higher value bands of postings and counts	`inventory:approve`	
CPO	Third approver in the top value band (default rule)	(task chain role)	
BUYER / TENANT_ADMIN	Maintain receipt tolerances per material/supplier	`po:change` or `admin:rules`	
REQUESTER	May withdraw consignment stock (route accepts `req:create`)	`req:create`	

All internal users can read stock, alerts, movements and receipts (`pic:read`), restricted to the plants in their `plants` attribute (ABAC; absent or `*` = all plants).

3. Screens

Screen (BP ID)	Route under /pic	What it shows	Actions
RCV-01 Inbound & ASN Monitor	`/receiving` tab **Inbound**	ASNs in transit with PO, expected date, documents; confirmed POs awaiting receipt	Receive (opens the GR dialog; RECEIVER only)
RCV-04 Returns & Quality Results	`/receiving` tab **Quality**	Open inspection lots with lot quantity and sample size	Usage decision (QUALITY_INSPECTOR)
RCV-03 Service Entry Sheet	`/receiving` tab **Service entries**	SES by PO, method, period, value, status	Approve (SERVICE_OWNER, status SUBMITTED)
RCV-02 Goods Receipt	`/receiving` tab **Goods receipts**, detail `/receiving/[ref]`	Receipts with status, lines, lots, warnings (e.g. expired e-way bill), lineage, audit	Reverse, request replacement on a return
(special stocks)	`/receiving` tab **Consignment**	Consignment stock, withdrawals, self-billing	Withdraw, settle (ERS)
(lot trace)	`/receiving` tab **Lot trace**, `/receiving/lots/[material]/[batch]`	Where a batch went (receipts, movements, units used)	Recall lot
(tolerances)	`/receiving` tab **Tolerances**	Over/under-delivery tolerance per material, supplier or both	Set / delete (TENANT_ADMIN, BUYER)
(supplier portal)	`/portal` tabs **Orders**, **Service sheets**	The supplier's confirmed orders and service sheets	Create ASN, create SES
Inventory (no BP ID; the page header reads "INV-01 · Inventory", which collides with the Blueprint's INV-01 Invoice Inbox)	`/inventory` tabs **Stock**, **Alerts**, **Movements**, **Postings**, **Counts**, **Reservations**	Stock by plant / location / stock type / batch; typed alerts; signed movement ledger; inventory documents	New posting, new count, new reservation; CSV export of movements
Material stock card (MRP-02 profile)	`/inventory/[material]`	Balances, open reservations, per-bucket projected stock	
Inventory documents	`/inventory/postings/[ref]` , `/inventory/counts/[ref]` , `/inventory/reservations/[ref]`	Lines, approvals, documents, lineage, audit tabs	Submit, approve/reject, post, cancel, enter counts, recount, issue

4. End-to-end workflow

A. Goods (F6-01 → F6-09)

1. **Supplier ships (portal, `POST /v1/asns`, scope `asn:create`).** The PO must be `CONFIRMED` or `PARTIALLY_RECEIVED`. Checks: at least one line, quantities positive; shipped quantity per line ≤ ordered × (1 + over-tolerance %) (`RCV-TOLERANCE`); e-way bill rule `EWAY-BILL` : if the consignment value incl. GST exceeds `requiredAboveMinor` (default ₹50,000) the e-way bill number (12 digits, rule `format`), vehicle number (Indian state or BH series) and validity (or `distance_km` , validity = ship date + ⌈km ÷ `kmPerDay` (200)⌉ days, to 23:59:59 IST) are mandatory (422 `EWAY_BILL_REQUIRED`). Result: ASN `IN_TRANSIT` ; event `asn.received` (consumer `notify-asn-received`); PO process steps F5-09/F5-10.
2. **Receiver posts the GR (`/receiving` → Receive, `POST /v1/goods-receipts` , scope `gr:post` , `Idempotency-Key` honoured).** The PO must be `TRANSMITTED` , `CONFIRMED` or `PARTIALLY_RECEIVED` ; an ASN of another PO is refused (422 `ASN_PO_MISMATCH`). System checks, in order:
    - SoD `SOD-03` (PO creator may not post its GR); ABAC `GR_POST` on the plant of every received line;
    - mandatory fields rule `MANDATORY-FIELDS` for document `GR_POST` (by company code, plant, category, purchase type, supplier group);
    - the company code, plant and storage location are active (422 `ORG_RECORD_INACTIVE`); the posting period of both the posting date and today is open (409 `PERIOD_CLOSED`);
    - per line: service lines are refused (they use SES); receipt-less lines are refused (2-way matched); over-delivery beyond the line's over-tolerance % is blocked (`RCV-TOLERANCE`); lot capture (`RCV-LOT`): batch if the material is `batchManaged` , expiry if `expiryManaged` , exactly one serial per unit if `serialManaged` , no expired goods, at least `minShelfLifeDays` remaining, serials unique and not already received on a non-reversed receipt.
    - Stock type per line: `QUALITY` if the line is quality-managed, `CONSIGNMENT` for a consignment line, otherwise `UNRESTRICTED` . Stock moves only for purchase types `STOCK` , `CONSIGNMENT` , `SUBCONTRACTING` (movement **101**). A subcontracting receipt consumes the components at the subcontractor (**543**).
    - Result: GR `POSTED` then immediately `QUALITY_HOLD` (any quality-managed line; one inspection lot per such line, sample size from ISO 2859-1 level II) or `ACCEPTED` . ASN lines are reduced; the ASN becomes `RECEIVED` when fully received. A receipt after the e-way bill validity is allowed but flagged (`EWAY_BILL_EXPIRED` warning, audited). Event `goods_receipt.posted` (consumers `kanban-refill` , `supplier-scorecard`). Healthcare template: a cold-chain excursion reported on the ASN quarantines the goods.
3. **No quality hold:** the receipt completes at once: ERP message `GOODS_RECEIPT` (key `GR\|<number>` ; asset lines carry capitalisation data from rule `ASSET-CAPITALISATION`), F6-08 "stock, GRIR and scorecard updated", F6-09 end.
4. **Quality inspector decides (`/receiving` → Quality, `POST /v1/inspection-lots/{ref}/usage-decision` , scope `quality:decide`).** Checks: lot `OPEN` ; accepted + rejected = lot quantity; a rejection needs a defect code and a reason; ABAC `STOCK_MOVE` on the plant; period open. Accepted quantity moves quality → unrestricted (or back to consignment) with **321**; rejected quantity moves quality → blocked (**350**), a return delivery `RD-…` is created and the blocked stock leaves with **122**. When the last open lot of the receipt is decided the GR becomes `ACCEPTED` , `PARTIALLY_REJECTED` or `REJECTED` , one `quality.usage_decision` event is emitted (consumers `capa-on-rejection` opens a CAPA, `supplier-scorecard`), and the ERP `GOODS_RECEIPT` message is sent (step 3).
    - **Credit or replacement (F6-07).** By default a return expects a credit note (handled in M10). With `replacement: true` on the decision, or later via `POST /v1/return-deliveries/{ref}/replacement` (return must be `CREATED` and expecting a credit note), the returned quantity is reopened on the PO for a new ASN and receipt; return status `REPLACEMENT_DUE` ; event `return_delivery.replacement_requested` .
5. **Hand-off to M10:** the accepted (4-way) or received-less-rejected (3-way) quantity is what invoices may be matched against; received-not-invoiced value appears as GR/IR accrual.

B. Reversal (movement 102). `POST /v1/goods-receipts/{ref}/reverse` (scope `gr:post` , reason required). Refused with 409 `RECEIPT_INVOICED` while any non-rejected invoice references the PO lines; refused once a lot was rejected and returned (use a return delivery instead). ABAC `GR_POST` and period checks apply. Stock is taken back from where it is now (quality stock if its lot is open, else released stock), open lots are cancelled, the ASN goes back to `IN_TRANSIT` , the PO quantities are reopened, and if the original GR was posted to the ERP a reversal `GOODS_RECEIPT` message (key `GR\|<number>\|102`) is enqueued. Event `goods_receipt.reversed` .

C. Services (F6-02 "service" → F6-04). The supplier (or an internal user with `ses:create`) enters an SES on a service line (`POST /v1/service-entries` , method `MILESTONE` , `TIME_MATERIAL` , `UNIT_RATE` or `PROGRESS`). Check `RCV-SES` : approved + in-approval + this sheet ≤ PO line value. Submission starts an `SES_APPROVAL` task for `SERVICE_OWNER` , excluding the supplier who entered it and the PO creator. Approval (`POST /v1/service-entries/{ref}/approve` or the task inbox) re-checks the cap under a row lock, adds the value to the PO line's approved service value, emits `service_entry.approved` (consumers `notify-service-entry-approved` , `accessorial-approved`). Rejection returns it to `REJECTED` ; it can be resubmitted.

D. Special stocks.

- Subcontracting: `POST /v1/purchase-orders/{ref}/components` (scope `gr:post`) moves components from own stock to the subcontractor (**541**, storage location = supplier code); receipt of the finished part consumes them (**543**). Event `subcontracting.components_provided`.
- Consignment: goods are received into supplier-owned `CONSIGNMENT` stock (no liability). `POST /v1/consignment/withdrawals` issues to a cost object (**201K**, purpose `CONSUMPTION`) or into own stock (**411K**, `TRANSFER`), first-expiry-first-out unless batch/serials are named; it creates the liability at the PO price; optional replenishment requisition. Event `consignment.withdrawn` (consumer `par-level-replenishment`). Settlement is self-billing (ERS) in M10.
- Lot recall: `POST /v1/lots/{material}/{batch}/recall` (scope `quality:decide`, notice reference and reason required) quarantines every unit on hand and returns it (**122**) with a return delivery per receipt line; event `lot.recalled`.

E. Inventory documents.

1. **Stock posting `STP-` (`/inventory` → Postings).** Types `TRANSFER` (311 within a plant, 301 plant to plant, same company code), `BLOCK` (344), `UNBLOCK` (343), `SCRAP` (551). Release from quality stock is refused (422 `USE_USAGE_DECISION`). Create (`inventory:post`, maker recorded for SOD-10) → submit: ABAC `INVENTORY_POST`, active org, period open, stock available. Rule `INVENTORY-POSTING`: a `TRANSFER` / `BLOCK` / `UNBLOCK` below `autoPostBelowMinor` (default ₹25,000) is approved by rule and posted at once; everything else, and every `SCRAP`, goes to the approval chain of its value band, excluding the maker and submitter. After approval a user with `inventory:post` posts it (`POST .../post`): stock re-checked under lock (422 `INSUFFICIENT_STOCK`), period re-checked (409 `PERIOD_CLOSED`), ERP message `STOCK_POSTING` (key `STP|<number>`). Events `stock_posting.submitted/approved/rejected/posted`.
2. **Physical inventory `PHI-` (`/inventory` → Counts).** Create for a plant and storage location, optionally by material group, named materials or ABC class (value issued in the last 12 months); book quantities are frozen (`frozenAt`). Receivers count blind (`PUT .../counts`, `inventory:count`; first entry moves `PLANNED` → `COUNTING`). A line outside `qtyTolerancePct` (2 %) or `valueToleranceMinor` (₹5,000) must be recounted by a different person; a controller can order a recount (`POST .../recount`). Submit → approval chain by the value of the differences (approver is never a counter, SOD-11) → post (`inventory:post`): difference = counted − book at freeze, posted as **701** (gain) / **702** (loss) on unrestricted stock, so receipts during counting are not wiped out; a loss larger than the stock now on hand is refused. ERP message `STOCK_POSTING` (key `PHI|<number>`); event `physical_inventory.posted`.
3. **Reservation `RSV-` (`/inventory` → Reservations).** Create (`inventory:post`, ABAC `INVENTORY_RESERVE`): availability = unrestricted stock at the location less other open reservations; rule `INVENTORY-RESERVATION` `blockWhenUnavailable` (default false) decides warning or refusal; a cost object is required (`requireCostObject`). The open quantity is `RESERVATION` demand for MRP. Issue (`POST .../issue`) posts **201** (first expiry first out) and reduces the demand; ERP `STOCK_POSTING` key `RSV|<number>|<cumulative issued>`. Cancel by the requester or `inventory:post`; expiry by job. Events `stock_reservation.created/issued/cancelled`.

F. Reads. `GET /v1/stock`, `/v1/stock/summary`, `/v1/stock/alerts`, `/v1/stock/materials/{material}`, `/v1/stock/movements` and `/v1/stock/movements/export` (CSV, max 50,000 rows, hashed and audited); `GET /v1/inventory/rules`.

5. Document states

Document	States and transitions (command)	Guards
ASN	`IN_TRANSIT` → `RECEIVED` (all lines received); back to `IN_TRANSIT` on GR reversal	PO `CONFIRMED` / `PARTIALLY_RECEIVED` to create
Goods receipt	`POSTED` → `hold` → `QUALITY_HOLD` / → `accept` → `ACCEPTED`; `QUALITY_HOLD` → `ACCEPTED` / `PARTIALLY_REJECTED` / `REJECTED` (last usage decision); `POSTED`, `ACCEPTED`, `QUALITY_HOLD` → `reverse` → `REVERSED`	Reverse refused if invoiced or a lot was returned
Inspection lot	`OPEN` → `DECIDED` (usage decision `ACCEPTED` / `PARTIALLY_ACCEPTED` / `REJECTED`); `OPEN` → `CANCELLED` on GR reversal	accepted + rejected = lot qty
Return delivery	`CREATED` → `REPLACEMENT_DUE` (replacement)	must still expect a credit note

Document	States and transitions (command)	Guards
Service entry	`DRAFT` →submit→ `SUBMITTED` →approve→ `APPROVED`; `SUBMITTED` →reject→ `REJECTED` →submit→ `SUBMITTED`; `APPROVED` →revoke→ `REVOKED`	value cap `RCV-SES`
Stock posting	`DRAFT` → `SUBMITTED` → `APPROVED` → `POSTED`; `SUBMITTED` → `REJECTED`; `DRAFT` / `SUBMITTED` / `APPROVED` → `CANCELLED`	SOD-10, stock under lock, period
Physical inventory	`PLANNED` →start→ `COUNTING` →complete→ `COUNTED` →submit→ `PENDING_APPROVAL` →approve→ `APPROVED` →post→ `POSTED`; `COUNTED` / `PENDING_APPROVAL` →reopen/reject→ `COUNTING`; any open state → `CANCELLED`	recount by another person, SOD-11
Reservation	`OPEN` → `PARTIALLY_ISSUED` → `ISSUED`; `OPEN` / `PARTIALLY_ISSUED` → `CANCELLED` or `EXPIRED`	

Any command not allowed from the current state returns 409 (`ILLEGAL_TRANSITION`).

6. Business rules and controls

Rule set / control	What it controls
`RCV-TOLERANCE`	Default over/under/price receipt tolerance without a contract (default 0 %). Overridden per material, supplier or both (`PUT /v1/receipt-tolerances` ; most specific wins at PO creation) and per PO line
`EWAY-BILL`	`requiredAboveMinor` 5,000,000 paise, `kmPerDay` 200, `format` `^[0-9]{12}$`
`MANDATORY-FIELDS` (document `GR_POST`)	Header and line fields required at receipt, scoped by company code, plant, category, purchase type, supplier group
`ASSET-CAPITALISATION`	Asset class (by valuation class, material group, default), cost centre and capitalise flag sent with asset-line receipts
`INVENTORY-POSTING`	`autoPostBelowMinor` 2,500,000; `autoPostTypes` TRANSFER/BLOCK/UNBLOCK (SCRAP never); `approvalBands` ≤ ₹2 L INVENTORY_CONTROLLER, ≤ ₹10 L + FINANCE_CONTROLLER, above + CPO; reason required for BLOCK and SCRAP; `scrapReasons` , `blockReasons` ; `crossPlantTransfer` ; `maxLines` 100; SLA 2 business days
`INVENTORY-COUNT`	`qtyTolerancePct` 2, `valueToleranceMinor` 500,000, `recountDifferentCounter` true, bands ≤ ₹50,000 / ≤ ₹5 L / above, `maxLines` 2000, SLA 3 days
`INVENTORY-RESERVATION`	`blockWhenUnavailable` false, `graceDays` 7, `requireCostObject` true, purposes, `maxOpenDays` 365
`INVENTORY-ALERTS`	`qualityHoldDays` 7, `blockedDays` 30, `expiryWarnDays` 90, `negativeDrift`
ABAC actions	`GR_POST` , `STOCK_MOVE` , `INVENTORY_POST` , `INVENTORY_APPROVE` (with value limit), `INVENTORY_COUNT` , `INVENTORY_RESERVE` (company code + plant)
SoD	SOD-03 (PO creator vs GR), SOD-10 (posting maker vs approver, per document), SOD-11 (counter vs count approver; also blocked at role assignment)
Cut-off	GR, usage decision, reversal, stock postings, counts, consignment, components, recall and expiry write-off all refuse a closed posting period (409 `PERIOD_CLOSED`)

Ledger controls (`stock_ledger.post_movement`): one primitive for every movement; the balance row is locked; a balance below zero is refused (422 `NEGATIVE_STOCK` ; the database also has `ck_stock_qty_not_negative`); quantities at most 3 decimals; sign checked per movement type; value = quantity × standard price at posting. Movement types in use: 101, 102, 122, 201, 201K, 301, 311, 321, 343, 344, 344K, 350, 411K, 541, 543/544, 551, 561 (opening), 701/702, `ERP` (sync). Alerts: `STOCK_OUT` , `BELOW_SAFETY` , `BELOW_REORDER` , `EXPIRING` , `EXPIRED` , `QUALITY_HOLD_AGED` , `BLOCKED_AGED` ,

`NEGATIVE_DRIFT`. "Available" stock for MRP, the stock card and stock-out alerts = unrestricted + consignment (product decision of 2 Nov 2026, extends TD §9.1); quality, blocked and subcontractor stock are excluded; own-stock value and days of cover use unrestricted only.

Idempotency: `Idempotency-Key` on ASN, GR and SES creation; inventory commands are idempotent per document via their ERP key.

7. Integrations and events

ERP message (outbound)	When	Key	SAP mapping
`GOODS_RECEIPT`	Receipt complete (after last usage decision)	`GR	`
`GOODS_RECEIPT` (102)	Reversal of a posted receipt	`GR	
`STOCK_POSTING`	Posted stock posting, count difference, reservation issue	`STP	… `, PHI

Events: `asn.received`, `goods_receipt.posted`, `goods_receipt.reversed`, `quality.usage_decision`, `return_delivery.replacement_requested`, `service_entry.approved`, `subcontracting.components_provided`, `consignment.withdrawn`, `lot.recalled`, `stock.expired_written_off` (healthcare), `stock_posting.*`, `physical_inventory.posted`, `stock_reservation.*`. Inbound: ERP stock levels can be synced (`STOCK` kind of the ERP sync, see Platform); the opening position is loaded with `tools/stock_opening_load` (movement 561, reference `OPENING_BALANCE`, dry run and reconciliation report).

8. Scheduled jobs

Job key	Schedule	What it does
`inventory-reservations`	daily 02:45	Expires open reservations whose need date + `graceDays` has passed (demand set to zero, `EXPIRED`); removes zero-quantity reservation demand older than 7 days. Dry run supported
`scorecard-monthly`	1st of month 03:00	Recomputes supplier scorecards from receipts and invoices (receipt quality feeds it)

9. KPIs

Code	Name	Formula
`M09.RECEIPT_ACCURACY`	Receipt-to-PO accuracy %	received PO lines inside the PO tolerance / received PO lines
`M09.DOCK_TO_STOCK`	Dock-to-stock time (h)	mean time from GR to the usage decision
`M09.QUALITY_REJECTION`	Quality rejection rate %	rejected qty / received qty
`M09.SES_APPROVAL`	SES approval time (h)	mean time to approve an SES
`M09.INVENTORY_TURNS`	Inventory turns	value issued (201/201K, annualised) / average own stock value at standard price
`M09.STOCK_COUNT_ACCURACY`	Stock count accuracy %	counted lines inside tolerance at the first count / counted lines of posted counts

Code	Name	Formula
`M09.STOCK_ADJUSTMENT_VALUE`	Stock adjustment value	absolute value of 701/702 and 551 movements
`M02.STOCKOUTS`, `M02.DAYS_OF_COVER`	Stock-outs, days of cover	(M02 module KPIs that read this ledger)

10. Demo data

- **AXN signature case** (seeded by `neon_setup`, users `…@axion.example`): PO-77410 → ASN-61208 (2 batches, e-way bill) → **GR-90231** 5,400 received → **QL-2210** 5,380 accepted, 20 rejected (POROSITY) → **RD-4410** credit note requested. Sign in as `receiver`, `quality.inspector`, `service.owner`.
- **Pack `inventory_ops` (D13, AXN and KAC)**: opening stock (561), an auto-posted transfer, a blocked batch, a scrap waiting for approval (try approving as its maker: refused by SOD-10, then as `inventory.controller2`), one posted count with a small loss, one count still `COUNTING` with a pending recount, three reservations (open, partially issued, cancelled). Numbers start at STP-1001, PHI-1001, RSV-1001.
- **Pack `inventory_master` (D14)**: MOCK go-live inputs: KAC opening stock of 1,624 lines at plants 1100/1200/1300, two `INVENTORY_CONTROLLER` users per plant (`inventory.controller.<plant>.1/.2`), KAC rule versions `2026.2-KAC-MOCK`.
- Special stocks: pack `purchase_types` (D2, AXN, June 2027) has consignment and subcontracting cases.

11. Known limits and open items

- SAP `STOCK_POSTING` mapping is validated against the loopback mock only; 701/702 may need the physical-inventory document API in a real system; stock-type, reason and unit codes need a sandbox.
- GR reversal (102) is not mapped for the live SAP connector: manual in SAP.
- ERP stock sync writes an empty-batch balance that can double-count batch-managed materials; fractional quantities are accepted for each-unit materials (no unit-of-measure rule).
- Return deliveries keep status `CREATED` / `REPLACEMENT_DUE`; no code path closes them when the credit note or the replacement arrives (observed in code).
- The `/inventory` page header label "INV-01" duplicates the Blueprint ID of the Invoice Inbox.
- `docs/runbooks/inventory_operations.md` and the `inventory_ops` pack docstring point to `POST /v1/erp/sync` for loading stock; that route does not exist: use `POST /v1/admin/erp-sync/STOCK/run` (or `/payload`) or the opening-stock loader.
- Not yet run in the browser per the inventory analysis: block/unblock postings, rejecting a posting, Finance Controller approval, a controller-ordered recount.
- Migrations 0018/0019 (inventory) must be applied to production in a planned window before go-live.

M10 Invoice Matching, Payables & Finance

1. Purpose and scope

M10 (Blueprint §16, flow F7 "Invoice → match → post → pay", process key `pic.invoice`; TD §5-6, §7.3 DMN-03, §9.4, §12.3) takes the supplier's invoice from capture to cash: capture by portal, AP clerk, PDF/OCR, GST e-invoice (IRN/QR) or EDI; duplicate check; 2-, 3-, 4-way, service and self-billing (ERS) match against tolerances; exceptions and their resolution; credit and debit notes; retention and advances; approval and idempotent ERP posting; payment proposal and run with Indian withholding (TDS/TCS, s.197 certificates), the MSME 45-day rule and interest, foreign currency; bank files and bank statements; returned payments; posting periods, cut-off and GR/IR.

2. Roles

Role	What they do here	Key scopes	SoD constraints
SUPPLIER_USER	Submits invoices, credit notes and debit notes in the portal; accepts/declines early-payment offers; elects SCF	`invoice:submit`	Own documents only
AP_CLERK	Captures invoices, re-runs the match, resolves exceptions, rejects, approves and posts invoices, raises debit notes for the supplier, records bank results with finance	`invoice:capture`, `invoice:resolve`, `invoice:approve`, `invoice:post`	SOD-08: whoever captured an invoice may not approve it. SOD-04: the PO creator may not resolve exceptions on that PO
BUYER	Decides price-variance acceptance tasks (`PRICE_VARIANCE`) and the buyer chain of debit notes	(task chain role)	Excluded from tasks on POs they created
FINANCE_CONTROLLER	Approves invoices, runs payment proposals and runs, generates bank files, records bank results, imports bank statements, lifts payment blocks, closes and reopens periods, maintains supplier finance data and TDS challans, validates savings	`invoice:approve`, `payment:run`, `period:close`, `savings:validate`	SOD-09: whoever approved an invoice may not release its payment. SOD-02/02a: whoever changed the supplier's bank details may not pay or post its invoices
SUPPLIER_MANAGER	Maintains supplier finance data (MSME class, Udyam, PAN, TDS section, s.197 certificates) and bank changes	`supplier:manage`, `supplier:bank`	SOD-02: `supplier:bank` and `payment:run` never on one person
PROCUREMENT_HEAD	Second step of the debit-note buyer chain above ₹1 L (default rule)	(task chain role)	

Approval value limits come from the user's `value_limit_minor` / `approval_limits` attribute (ABAC actions `INVOICE_APPROVE`, `INVOICE_POST`, `PAYMENT_RUN`), compared in the company-code currency. Company-code ABAC also applies to capture (`INVOICE_CAPTURE`), match/resolve/reject/release (`INVOICE_RESOLVE`) and period close (`PERIOD_CLOSE`), and to every finance list and register (other company codes answer 404).

3. Screens

Screen (BP ID)	Route under /pic	What it shows	Actions
INV-01 Invoice Inbox	`/invoices`	Worklist with filters (status, supplier, company, text, dates, payment status, block, match type, exceptions, MSME, amount); GR/IR accruals; payment runs and bank files; posting periods card	Capture invoice, Payment run (Preview proposal, Execute run), Consignment settlement, Generate bank file, Record bank status, Bank results, Import bank statement (Analyse dry run, Apply), Close period / Preview cut-off / Reopen, TDS return export
INV-02 Match & Exception Resolver	`/invoices/[id]`	PO × GR × invoice per line with match detail, exceptions with cause and suggested action, credit/debit notes, e-invoice verification, MSME/TDS/FX data, early-payment offers, lineage, audit	Re-run match, Resolve, Reject invoice, Approve, Post to ERP / Retry post (idempotent), Release payment block, Raise debit note, Approve / Reject / Post debit note, Verify e-invoice, Withdraw offer, Elect supply-chain finance
INV-03 Posting & Reconciliation	`/invoices/reconciliation`	Reconciliation runs (count, amount, document number vs the ERP), differences, integration messages	Run reconciliation, resolve an item with a reason
INV-04 Payables Insights	`/invoices/insights`	Due-date ladder, discount capture, DPO, first-pass match, GR/IR ageing, MSME, exception ageing, TDS	
INV-05 (code label) TDS, Form 16A & MSME interest	`/invoices/tax` tabs **Challans & reconciliation** and **Form 16A data** (finance only), **TDS register & return**, **MSME interest**	Challans vs deducted TDS, Form 16A data per deductee and quarter, register, MSME interest accrued	Record / void challan, download data
Supplier finance card	`/suppliers/[code]` (finance section)	MSME class, Udyam, PAN (masked), TDS section, written agreement, s.197 certificates	Save (If-Match), add/revoke certificate
Supplier portal	`/portal` tab **Invoices**	Supplier's invoices, notes and offers	Submit invoice, credit/debit note, accept offer, elect SCF
Self-billing / advances (template)	`/industry` tabs **Self-billing (ERS)**, **Advances**	Open consignment withdrawals; construction advances	Settle; request advance

4. End-to-end workflow

1. **Capture (F7-01).** Supplier (portal) or AP clerk (`/invoices` → Capture invoice) calls `POST /v1/invoices` with channel `PORTAL`, `PDF_OCR`, `E_INVOICE` or `EDI` (EDI arrives through `POST /v1/edi/inbound`; OCR pre-fills the form through `POST /v1/invoices/documents`, nothing stored, 503 `OCR_NOT_CONFIGURED` without `PIC_OCR_URL`). Checks:
   - ABAC `INVOICE_CAPTURE`; supplier invoice number 1-40 printable characters (blanks normalised); IRN ≤ 64 characters;
   - exact duplicate on company code + supplier + number + fiscal year → 409 `DUPLICATE_INVOICE`; the same IRN on another non-rejected invoice → 409 `DUPLICATE_IRN` (both leave audit `DUPLICATE_BLOCKED`);
   - supplier `BLOCKED` / `MERGED` → 409 `SUPPLIER_BLOCKED`; only the supplier's own POs in that company code; one currency per invoice (the PO currency; 422 `MIXED_CURRENCY`);
   - rule `MANDATORY-FIELDS` (document `INVOICE_CAPTURE`); consignment lines are refused (settled by ERS); quantities/prices positive and within range; GST rate 0-100;

- GST: place of supply (company code/plant state vs supplier state) gives `INTRA` (CGST+SGST) or `INTER` (IGST); declared heads must add up to the line tax; reverse charge (rule `GST-RCM`, by HSN prefix) sets the supplier's tax to 0 and records the self-assessed tax;
- e-invoice: IRN and signed QR verified (IRP lookup when `PIC_IRP_*` is configured, otherwise local check of the IRN hash = SHA-256 of GSTIN + FY + doc type + number and of the QR payload); result in `irn_valid` and audit `E_INVOICE_VERIFIED`;
- retention per PO (`retention_pct`) and construction advance recovery are computed. Result: invoice `CAPTURED`; SOD-08 records the capturer; event `invoice.captured` (consumer `notify-invoice-captured`). The same call continues:

2. **Duplicate check (F7-02, `DUPLICATE_CHECK`).** Rule `DMN-03` : same supplier, gross within `dupAmountMinor` (₹1) and invoice date within ± `dupDays` (3), or an OCR/formatting variant of an earlier number (INV-0012 / inv 12 / INVO12). A hit sets a `DUPLICATE` exception and payment block → `EXCEPTION`; the match is refused (409 `DUPLICATE_HOLD`) until AP resolves it `NOT_DUPLICATE` or rejects the invoice.
3. **Match (F7-03/04, `MATCHING`).** Per invoice line, net of quantity credit notes, after row-locking the PO lines and counting other unposted invoices that already claim them:
   - `SES` : invoiced value ≤ approved service value − invoiced − pending → else `SERVICE`;
   - `ERS` (consignment): quantity ≤ withdrawn − settled → else `QUANTITY`;
   - `3WAY` : quantity ≤ received − rejected − invoiced − pending (+ quantity tolerance); `4WAY` (quality-managed lines): ≤ inspection-accepted quantity; no receipt → `MISSING_GR`;
   - `2WAY` (receipt-less lines): ≤ PO quantity − invoiced − pending;
   - price: line variance = net qty × invoice price − value credits − net qty × PO price; tolerance = min(`pricePct` % of the line value at PO price, `priceAbsMinor`) → else `PRICE`;
   - tax: rate and HSN equal to the PO line, tax = net × rate (0 under reverse charge), IRN valid for an `E_INVOICE`; correct heads for the place of supply; scheduled GST rate for the HSN/SAC (rule `HSN-GST`, CBIC master; skipped for a foreign supplier) → else `TAX`;
   - logistics template: freight audit against the contracted lane rate × approved proof-of-delivery quantity (rule `FREIGHT-AUDIT`) → `FREIGHT_AUDIT`. No exception → `MATCHED` (first-pass flag set; payment block only if the supplier is blocked). Exceptions → `EXCEPTION`, payment block on, event `invoice.exception.raised` (consumer `notify-invoice-exception`); the Invoice Exception Agent (L2) drafts the explanation and proposal.
4. **Resolve exceptions (F7-05, `/invoices/[id]` → Resolve, `POST /v1/invoices/{ref}/exceptions/{id}/resolve` , scope `invoice:resolve`).** SOD-04 is checked against the PO. Allowed resolutions by type: `DUPLICATE` → `NOT_DUPLICATE` or `DISPUTE`; `PRICE` → `CREDIT_NOTE_RECEIVED`, `PRICE_ACCEPTED`, `DISPUTE`; all others → `CREDIT_NOTE_RECEIVED`, `GR_POSTED`, `DISPUTE`.
   - Credit note: supplier or AP posts `POST /v1/invoices/{ref}/credit-notes` (quantity credit, or value credit with `unit_credit_minor`; never above the invoice gross less earlier notes; period of posting day and note date must be open). Then resolve `CREDIT_NOTE_RECEIVED` with its number. A credit note on an already posted invoice goes to the ERP as `SUPPLIER_CREDIT_MEMO`. Event `credit_note.received`.
   - `PRICE_ACCEPTED` (reason required) opens a `PRICE_VARIANCE` task for a BUYER other than the PO creator; approval re-prices the PO line (audited `PRICE_VARIANCE_ACCEPTED`) and re-matches.
   - `DISPUTE` keeps the exception open and the block on (event `invoice.disputed`). `GR_POSTED` after the missing receipt is posted.
   - When no exception is open the invoice is re-matched automatically. `POST /v1/invoices/{ref}/reject` (reason) ends it as `REJECTED`.
5. **Approve (`POST /v1/invoices/{ref}/approve` , scope `invoice:approve` , If-Match).** ABAC `INVOICE_APPROVE` incl. value limit; SOD-02a and SOD-08 guarded; if the receipt was used up meanwhile, the invoice goes back through the match. Result `APPROVED`; SOD-09 records the approver. (One approval command, not a multi-step chain.)
6. **Post to ERP (F7-06, `POST /v1/invoices/{ref}/post`).** `Idempotency-Key` is mandatory (428) and must be the posting key `tenant|company code|supplier|invoice number` (percent-encoded if it has spaces or non-ASCII). Checks: status `APPROVED`, no open exception, ABAC `INVOICE_POST`, SOD-02a, connector maps `SUPPLIER_INVOICE`, nothing over-claimed, period open. The FX rate (invoice date) and TDS/TCS are fixed once, then message `SUPPLIER_INVOICE` (key = posting key) carries gross less credit notes, retention and net payable, tax heads, reverse charge, TDS/TCS. ERP down → answer `POSTING` with the retry status; a retry with the same key re-runs (never a stale replay); a lost response is resolved by looking the document up in the ERP first. On ERP confirmation: `POSTED`, due date set (see §6), event `invoice.posted` (consumers `savings-realised` → M11, `spend-cube`, `supplier-scorecard`, `rebate-accrual`).
7. **Payment proposal and run (F7-07).** The `payment-due` job (06:30) creates a `PAYPROP-<date>` approval task for the `FINANCE_CONTROLLER` with invoices due within `horizonDays` and the terms-discount windows closing in that horizon; approving runs the normal payment run. Manually: `/invoices` → Payment run → Preview proposal (`GET /v1/payments/proposal`) → Execute run

(`POST /v1/payments/runs` , scope `payment:run` , Idempotency-Key; `run_date` , optional `supplier_codes` / `invoice_numbers` , an empty list is 422 `EMPTY_SELECTION`). Selected invoices are `POSTED` , unpaid, unblocked and due (locked FOR UPDATE). Per invoice a skip reason or a payment:
    - skipped: supplier blocked/merged; period of the run date closed; nothing payable; connector does not map `PAYMENT` ; no / more than one active bank account; bank change cooling-off; ABAC `PAYMENT_RUN` limit; SoD (SOD-09, SOD-02); foreign beneficiary data incomplete; no FX rate for the payment date;
    - amount = gross − credit notes + posted debit notes − retention − early-payment discount − advance recovery − TDS (+ TCS when `addToPayable`);
    - payment `PAY-…` `COMPLETED` in run `PRUN-…` , invoice `payment_status` `IN_PAYMENT` , MSME interest accrued if late, ERP message `PAYMENT` (key `PAY|<payment>`); bank change within 30 days → audit `BANK_CHANGE_RECENT_ALERT` . Late debit notes on already paid invoices are paid in the same run. When the ERP confirms the clearing: invoice `PAID` , event `payment.completed` (consumers `notify-payment-completed` , `advance-paid`), process F7-09 "paid and reconciled".
8. **Bank file (`POST /v1/payments/runs/{run}/file` , `ISO20022_PAIN001` or `NEFT_CSV`).** Debtor account from rule `PAYMENT-FILE` (409 until configured); one PmtInf per company code, date and currency; pain.001.001.09 with BIC/intermediary/charge bearer/purpose code when a foreign beneficiary is in the file; NEFT refuses non-INR or foreign accounts (422 `NEFT_INR_ONLY`). Content stored field-encrypted with its SHA-256; event `payment_file.generated` . Delivery is manual (download, upload in the bank portal) or `POST /v1/payment-files/{number}/transmit` when the SFTP adapter (`PIC_BANK_SFTP_*`) is configured. Status `POST .../status` : `SENT` , `ACKNOWLEDGED` (bank reference required), `REJECTED` .
9. **Bank results.** Per payment `POST /v1/payment-files/{number}/bank-results` (`CREDITED` with UTR, or `RETURNED` with reason), or import a statement `POST /v1/bank-statements/import?dry_run=true|false` (camt.053, camt.052, camt.054, MT940; idempotent per statement id; analysis states `READY` , `UNCHANGED` , `CONFLICT` , `AMOUNT_MISMATCH` , `FILE_NOT_SENT` , `UNMATCHED` , `AMBIGUOUS` , `NO_REFERENCE` ; batch entries split per transaction). A `RETURNED` payment makes the invoice `PAID` → `POSTED` , `UNPAID` again (event `payment.returned`) and enqueues `PAYMENT_REVERSAL` (key `PAYREV|<payment>`); the reconciliation item `PAYMENT_RETURN_OPEN` stays until the ERP confirms the reversal.
10. **Reconciliation (INV-03).** Nightly job `erp-reconciliation` or `POST /v1/reconciliation/runs` : posted invoices, payments, credit and debit memos per company code vs ERP documents; differences resolved with a reason (`POST /v1/reconciliation/items/{id}/resolve`).

Alternative documents

- Debit note (`POST /v1/invoices/{ref}/debit-notes`): `RECEIVED` → `MATCHED` when the price correction stays within PO price + `unitTolerancePct` , otherwise (and every additional charge) → `PENDING_APPROVAL` for the buyer chain (rule `DEBIT-NOTE` , PO creators excluded) → AP approve (`invoice:approve`) → post (`invoice:post` , period checked, message `SUPPLIER_DEBIT_MEMO` , TDS on the note fixed at posting) → nets into the invoice's payment. Max `maxPctOfInvoice` (25 %) of the invoice net.
- Retention: withheld at posting as a blocked part of the liability; `GET /v1/purchase-orders/{ref}/retention` ; released by `POST /v1/purchase-orders/{ref}/retention-release` (an AP document of match type `RETENTION` ; the clerk is recorded as capturer and approver for SoD).
- Consignment self-billing (ERS): `POST /v1/consignment/settlements` builds an invoice of match type `ERS` from withdrawals up to `period_to` ; the builder is recorded as capturer (SOD-08), so another user approves it.
- Advances (construction template): `POST /v1/purchase-orders/{ref}/advances` ; TDS deducted on the advance is not deducted again on the bill it is recovered from.
- Early payment: `GET /v1/invoices/{ref}/early-payment` , `POST .../early-payment/offers` , `.../offers/{id}/decision` , `.../offers/{id}/cancel` (sources `TERMS` , `BUYER` , `SUPPLIER` ; rule `EARLY-PAYMENT` : offer valid 7 days, ≥ 3 days early, ≤ 5 % discount); accepting moves the due date to the pay-on date; a missed pay-on date lapses the offer. SCF: `POST /v1/invoices/{ref}/scf` (≥ ₹1 L by default) sets payment option `SCF` , event `invoice.scf.elected` , message `SCF_FINANCING` ; paid at maturity by the normal run.
- Payment block release: `POST /v1/invoices/{ref}/payment-block/release` (scope `payment:run` and role FINANCE_CONTROLLER, reason ≥ 5 characters) for a posted, unpaid invoice whose block no exception explains (e.g. loaded from the ERP); audit `PAYMENT_BLOCK_LIFTED` .

5. Document states

```
Invoice:  CAPTURED -check_duplicates-> DUPLICATE_CHECK -match-> MATCHING -matched-> MATCHED -approve-> APPROVED -post-> POSTED -pay-> PAID
          DUPLICATE_CHECK -hold-> EXCEPTION ;  MATCHING -exception-> EXCEPTION -rematch-> MATCHING
          MATCHED / APPROVED -rematch-> MATCHING (receipt used up meanwhile)
```

```
          PAID -return_payment-> POSTED (bank returned the credit)
          CAPTURED / EXCEPTION -reject-> REJECTED
Invoice payment_status: UNPAID -> IN_PAYMENT (run) -> PAID (ERP clearing); back to UNPAID on a returned payment
Exception: OPEN -> RESOLVED | SUPERSEDED (re-match) ; DISPUTE keeps it OPEN
Debit note: RECEIVED -match-> MATCHED | -refer-> PENDING_APPROVAL -approve_variance-> MATCHED -approve-> APPROVED -post-> POSTED ;
            RECEIVED / MATCHED / PENDING_APPROVAL / APPROVED -reject-> REJECTED
Payment file: GENERATED -send-> SENT -acknowledge-> ACKNOWLEDGED ; GENERATED / SENT -reject-> REJECTED
Posting period (company code + YYYY-MM): OPEN -> CLOSED (close) -> OPEN (reopen, reason >= 10 chars)
```

Illegal transitions return 409 `ILLEGAL_TRANSITION`.

6. Business rules and controls

Rule set	Controls (defaults)
`DMN-03`	`pricePct` 1, `priceAbsMinor` 50,000 (₹500), `qtyPct` 0, optional `qtyAbs` (units; the lower of % and absolute applies), `sesPct` 0, `dupAmountMinor` 100, `dupDays` 3. Scoped by company code and supplier group
`MSME`	`maxDays` 45, `noAgreementDays` 15, classes MICRO/SMALL, `bankRatePct` null (no interest until published), `interestMultiple` 3
`TDS`	`noPanRatePct` 20; sections 194C-IND (1 %), 194C-OTH (2 %), 194J-TECH (2 %), 194J-PRO (10 %), 194H (2 %), 194I-LB (10 %), 194I-PM (2 %), 194Q (0.1 % on the excess over ₹50 L); single and annual thresholds; `tcs` (off: 206C(1H), 1 %); `deductor` TAN/PAN/name. Illustrative until the tax team signs off
`GST-RCM`, `HSN-GST`	Reverse-charge HSN prefixes; CBIC rate master (`CBIC-2025.09` loaded by the reference-data pack)
`PAYMENT-FILE`	`format`, `debtorName`, `debtorAccountRef`, `debtorIfsc` (per company code)
`PAYMENT-RUN`	`horizonDays` 3, `approverRole` FINANCE_CONTROLLER, `slaBusinessDays` 1; `autoRun` is ignored by policy (the job never pays by itself)
`DEBIT-NOTE`, `EARLY-PAYMENT`, `FREIGHT-AUDIT` (logistics), `MANDATORY-FIELDS` (`INVOICE_CAPTURE`)	as described in §4

Calculations:

- **Due date** = invoice date + net days of the payment terms; for a MICRO/SMALL supplier capped at acceptance date + 45 days (15 days without a written agreement). Acceptance date = latest non-reversed GR posting date or approved SES period end not after the invoice date, else the invoice date.
- **MSME interest** (when `bankRatePct` is set) = amount × ((1 + bankRate × 3 / 100 / 12)^(days late / 30) − 1), recorded per late payment (`msme_interest`, event `msme.interest.accrued`) and sent in the PAYMENT payload.
- **TDS** fixed at first posting: base = taxable value; rate = section rate (no PAN on file → at least 20 %); thresholds: single-bill and annual cumulative per supplier and section, catch-up of earlier untaxed bills once the annual threshold is crossed; 194Q on the excess only. At payment a valid s.197 certificate (`POST /v1/suppliers/{code}/tds-certificates`: number, rate, validity, section, optional amount cap `max_base_minor`) applies its rate up to the cap, the section rate to the rest. Debit notes are taxed too. TCS when enabled.

- **FX**: invoice currency = PO currency; rate of the invoice date fixed at posting (409 `FX_RATE_MISSING` without one); at payment, gain/loss = amount × invoice rate − amount × payment-date rate (stored on the payment and sent to the ERP).
- **Posting key** `tenant|company|supplier|invoice no.` + unique `integration_message` per key: retries never double-post; one live payment per invoice (unique index); concurrent runs pay each invoice once.

Controls: ETag / If-Match on invoice writes (412 `STALE_WRITE`; required when `PIC_REQUIRE_IF_MATCH` is on); Idempotency-Key on capture, credit note, resolve, payment run, bank file, bank results, period close/reopen, challans. **Period close** (`POST /v1/finance/periods/{YYYY-MM}/close`, scope `period:close`): only after the period has ended (409 `PERIOD_NOT_ENDED`); stores the cut-off snapshot (unposted invoices by status, GR/IR accrual) and a line-by-line GR/IR snapshot; event `finance.period.closed`. Any GR, invoice post, credit/debit note, payment run, stock movement dated into a closed period → 409 `PERIOD_CLOSED` (with the reopen rule) and audit `POSTING_REFUSED`. Reopen with a reason ≥ 10 characters (`finance.period.reopened`). **GR/IR** (`GET /v1/grir`) = accepted not yet invoiced value per PO line.

7. Integrations and events

Message (outbound, key)	ERP document (SAP simulator type)	Live connectors
`SUPPLIER_INVOICE` (posting key)	RE 5100…	SAP S/4 (API_SUPPLIERINVOICE_PROCESS_SRV / INVOIC02), Oracle, Dynamics, Tally, Odoo (connector-specific)
`SUPPLIER_CREDIT_MEMO` (`	CN	`)
`SUPPLIER_DEBIT_MEMO`, `PAYMENT_REVERSAL` (`PAYREV	`), SCF_FINANCING`	KD, ZR, ZF
`PAYMENT` (`PAY	`)	ZP

Inbound: EDIFACT D.96A INVOIC 380/381/383 via `POST /v1/edi/inbound` with CONTRL and APERAK acknowledgements (re-delivery returns the stored acknowledgement; 409 `EDI_REFERENCE_REUSED`); IRP lookup (`PIC_IRP_URL`, `_CLIENT_ID`, `_CLIENT_SECRET`, `_GSTIN`); bank statements (camt/MT940); ERP payment-status sync (`PAYMENT_STATUS` kind, event `invoice.paid_in_erp`).

Events: `invoice.captured`, `invoice.exception.raised`, `invoice.disputed`, `credit_note.received` (→ spend cube), `debit_note.received/rejected/posted`, `invoice.posted`, `invoice.early_payment.offered/accepted`, `invoice.scf.elected`, `payment.completed`, `payment.returned`, `payment_file.generated/sent/acknowledged/rejected/results_recorded`, `msme.interest.accrued`, `tds.challan.recorded`, `finance.period.closed/reopened`. Notification kinds: `INVOICE_EXCEPTION` and task notifications.

8. Scheduled jobs

Job key	Schedule	What it does
`payment-due`	daily 06:30	Proposal task `PAYPROP-<date>` for invoices due within `horizonDays` (MSME deadlines included) and closing discount windows; never pays by itself
`erp-reconciliation`	daily 01:30	Reconciles invoices, payments, credit and debit memos per company code with the ERP; notifies finance of differences
`idempotency-purge`	hourly :15	Deletes idempotency records older than 24 h

9. KPIs

Code	Name	Formula
`M10.FIRST_PASS_MATCH`	First-pass match %	invoices matched without an exception / invoices matched
`M10.EXCEPTION_CYCLE`	Exception cycle time (h)	mean time to resolve an invoice exception
`M10.DUPLICATES_PREVENTED`	Duplicates prevented	held as duplicate and not released + refused at capture
`M10.DISCOUNT_CAPTURE`	Discount capture %	early-payment discount captured / available
`M10.DPO`	DPO (days)	payment-weighted days from invoice date to payment (company currency)
`M10.TOUCHLESS_INVOICE`	Touchless invoice %	matched first time, no exception, credit or debit note / posted or paid
`M10.MSME_ON_TIME`	MSME payment compliance %	MSME payments made by their due date / completed MSME payments
`M10.EXCEPTION_AGEING`	Open exceptions > 7 days	count

10. Demo data

- **AXN signature case: INV-TF-8812** (E_INVOICE, supplier TF001) billed 5,400 → `QUANTITY` exception +20 → credit note **CN-TF-221** → matched → posted as ERP **5100004421** → paid **PAY-880002** in **PRUN-2611**. Try the posting retry with Idempotency-Key `AXN|1000|TF001|INV-TF-8812`. Operating case 1 (docs/PIC_Process_Flows.md): KD/2702-118 price exception (410 vs 405) accepted, PRUN-2612. Users `ap.clerk`, `finance.controller`, `finance.controller2`, `buyer`.
- **Pack `finance_ops` (D15, MOCK)**: KAC 1 Sep-31 Oct 2026: exceptions in every resolution state, rejected and disputed invoices, foreign-currency imports, credit/debit notes, early payment and SCF, seven payment runs with pain.001/NEFT files and bank acknowledgements, TDS and s.197 certificates, MSME suppliers paid on time and late with interest, returned payments, camt.053/MT940 statements, periods April-September closed (September reopened once), October open. AXN gets non-signature additions only.

11. Known limits and open items

- Invoice approval is one command with a per-user value limit; no multi-step value-band chain and no delegation for invoice approval (customer bands needed).
- No partial payment of an invoice (the run pays the full net payable).
- Live ERP: `PAYMENT_REVERSAL`, `SUPPLIER_DEBIT_MEMO`, `SCF_FINANCING` unmapped outside the simulator.
- TDS: no Form 16A PDF and no TRACES certificate number (data export only); no s.195 foreign-remittance TDS; debit notes carry a single tax amount (no heads); default sections illustrative.
- MSME interest is recorded at payment only (overdue unpaid invoices are shown in the due ladder, not accrued daily).
- Import duty / IGST on the bill of entry is not built.
- Live IRP verification and bank host-to-host have no sandbox test; bank statement fixtures are synthetic.
- `release_payment_block` and savings validation check the built-in role FINANCE_CONTROLLER by name, so a custom role holding `payment:run` / `savings:validate` cannot use them (observed in code).
- `docs/runbooks/scheduled-jobs.md` still says `autoRun=true` pays what is due today; the code ignores `autoRun` (reported as `autoRunIgnored`).
- `docs/PIC_Finance_Module_Analysis.md` §8.5 says the packs `purchase_types` and `industry_healthcare` approve an ERS settlement with the capturing clerk; that is outdated: both packs now approve with `finance.controller` / `finance.controller2`.

M11 Savings & Value Realisation

1. Purpose and scope

M11 (Blueprint §17, step F7-08, TD §9.5, principle P12 "three kinds of savings") tracks money saved the way finance signs it off. Every saving is a ledger entry with its formula and source document: **negotiated** (price vs baseline × volume), **cost avoidance** (an increase avoided), **realised** (baseline vs actual invoiced price × actual volume, only from posted invoices). Initiatives move through Identified → Negotiated → Contracted → Realising → Validated (or Lost); finance validates realised savings (SOD-07), and only validated realised savings reduce budget (P&L view).

2. Roles

Role	What they do here	Scopes	SoD
CATEGORY_MANAGER	Owns initiatives: creates them, records a negotiated price, marks them lost; sourcing events create initiatives automatically for the event owner	`savings:manage`	The owner never validates (SOD-07)
FINANCE_CONTROLLER	Validates realised savings	`savings:validate` (and the role itself: `validate` checks the role FINANCE_CONTROLLER)	SOD-07 rule (`SAVINGS_INITIATE` vs `SAVINGS_VALIDATE`) plus a direct owner check (409 `SOD_CONFLICT`)
Savings Validation Agent (`SAVINGS_VALIDATION` , L3, runs as CATEGORY_MANAGER)	Logs the realised-savings computation from each posted invoice as an agent run		Guarded with the SoD action `SAVINGS_VALIDATE` ; it never validates
All internal users	Read the register, waterfall, pipeline and P&L	`pic:read`	

3. Screens

Screen (BP ID)	Route under /pic	What it shows	Actions
SAV-01 Savings Register	`/savings` tab **Register (SAV-01/02)**	Initiatives with stage, baseline method, negotiated / avoidance / realised (₹ and crore), validated flag	Create initiative, Record negotiated price, Mark lost, Validate realised savings
SAV-02 Realisation Waterfall	`/savings` (initiative detail in the register)	Negotiated → contracted → realised with leakage (off-contract, price drift) and every ledger entry with its formula and source	
SPD-08 Pipeline tracker	`/savings` tab **Pipeline tracker (SPD-08)**	Funnel by stage, forecast vs realised, gap, filters (stage, category, owner, created from/to, minimum forecast)	Open an initiative (`?ref=`)
M11-C4 P&L and budget impact	`/savings` tab **P&L and budget impact (M11-C4)**	Fiscal-year months and categories: realised, validated, pending validation, negotiated, avoidance; plan target and attainment %; validation queue	Validate from the queue

Screen (BP ID)	Route under /pic	What it shows	Actions
Copilot	`/copilot`	"savings" questions answer from the register	

4. End-to-end workflow

1. **Identified.** Two ways:
   - Automatically: when a sourcing event with a baseline price is published (event `sourcing.event.published`, consumer `savings-identify`), an initiative `SV-…` is created for the event owner with the event's baseline method and price.
   - Manually: `/savings` → Create initiative (`POST /v1/savings/initiatives`, scope `savings:manage`): title, category, annual volume, optional material, baseline method `LAST_PRICE`, `WAVG_12M`, `STANDARD` or `SHOULD_COST`. The baseline is fixed at creation (see §6); SOD-07 records the creator as initiator. Audit `IDENTIFIED` with the derivation.
2. **Negotiated.**
   - From sourcing: `sourcing.event.awarded` (consumer `savings-negotiated`) books `NEGOTIATED` = (baseline − weighted award price) × annual quantity, keeps the awarded supplier prices; an award above baseline is booked as 0 (cost increase audited, never a negative saving).
   - Manually (owner only, else 403): Record negotiated price (`POST /v1/savings/{ref}/negotiate`: new price, supplier, evidence) books (baseline − new price) × annual volume.
   - Negotiated savings never count toward budget (`counts_toward_budget = false`).
3. **Contracted.** `contract.activated` (consumer `savings-contracted`): when every contract from the initiative's sourcing event is `ACTIVE` or `EXPIRING`, stage `CONTRACTED` (initiatives created manually skip this stage).
4. **Cost avoidance.** When a contract index reset absorbs part of a market increase, `AVOIDANCE` = (market price − agreed price) × remaining contract quantity is booked (never counts toward budget).
5. **Realising (F7-08).** `invoice.posted` (consumer `savings-realised`): for each invoice line whose material has an initiative in `NEGOTIATED` / `CONTRACTED` / `REALISING` / `VALIDATED` and whose supplier is one of the agreed suppliers, `REALISED` = baseline × net quantity − value actually paid (quantity credit notes reduce the quantity, value credit notes reduce the value). The entry counts toward budget. The first realised entry moves the stage to `REALISING`; event `savings.realised` (consumer `notify-savings-realised`); process step F7-08 on the invoice.
6. **Validated.** Finance opens the P&L tab's validation queue (initiatives with realised savings, `canValidate` false for one's own) or the register → Validate realised savings (`POST /v1/savings/{ref}/validate`, scope `savings:validate`, If-Match with the waterfall's ETag; 412 when stale). Checks: role FINANCE_CONTROLLER (403), SOD-07 (409), stage `REALISING`. Result `VALIDATED`, validator and time stored. Realised entries posted later still add to the initiative.
7. **Lost.** Owner marks it lost with a reason (`POST /v1/savings/{ref}/lose`); a cancelled sourcing event also loses its initiative automatically (consumer on `sourcing.event.cancelled`).
8. **Reporting.** `GET /v1/savings` (register), `/v1/savings/{ref}/waterfall`, `/v1/savings/pipeline`, `/v1/savings/pnl?fiscal_year=2026-27`. Hand-off: realised savings feed the KPI explorer, category plans (target attainment) and the CPO command centre.

5. Document states

From	Command / trigger	To
`IDENTIFIED`	negotiate (award or manual)	`NEGOTIATED`
`NEGOTIATED`	contract (all event contracts active)	`CONTRACTED`
`NEGOTIATED`, `CONTRACTED`	realise (first realised entry)	`REALISING`
`REALISING`	validate (finance, not the owner)	`VALIDATED`

From	Command / trigger	To
`IDENTIFIED`, `NEGOTIATED`	lose	`LOST`

Other transitions are refused with 409.

6. Business rules and controls

- **Baselines** (fixed per initiative at creation): `LAST_PRICE` = price of the latest PO line for the material, else the purchasing info record; `WAVG_12M` = Σ(qty × price) / Σ qty of invoice lines of the last 12 months; `STANDARD` = the material's standard price; `SHOULD_COST` = the should-cost price supplied by the category manager. Errors: `NO_PRICE_HISTORY`, `NO_STANDARD_PRICE`, `MATERIAL_REQUIRED`, `BASELINE_REQUIRED`. (The Blueprint's "budget price" is implemented as the standard price.)
- **Formulas:** negotiated = (baseline − new price) × annual volume; avoidance = (market − agreed) × remaining volume; realised = baseline × (invoiced − credited qty) − (net value − value credits).
- **Leakage** (waterfall): `OFF_CONTRACT` = invoiced on a PO of a supplier outside the award (invoiced qty × (PO price − new price)); `PRICE_DRIFT` = invoiced above the agreed price.
- **Budget impact** = REALISED entries of VALIDATED initiatives only; pending validation shown separately; negotiated and avoidance reported but never counted.
- One ledger entry per initiative, type and source document (idempotent consumers).
- SOD-07 (High risk; can be switched to MITIGATE for a time-boxed exception through rule `SOD-MITIGATION`).
- Fiscal year April-March (`2026-27`).

7. Integrations and events

Consumes `sourcing.event.published`, `sourcing.event.awarded`, `sourcing.event.cancelled`, `contract.activated`, `invoice.posted`. Emits `savings.realised`. No ERP message: savings are a PIC ledger; budget reduction is reported, not posted.

8. Scheduled jobs

None specific. `spend-forecast-snapshot` (monthly, 2nd at 05:30) and `spend-cube-refresh` (every 5 minutes) keep the spend side current.

9. KPIs

Code	Name	Formula
`M11.NEGOTIATED_SAVINGS`	Negotiated savings	sum of NEGOTIATED ledger entries
`M11.REALISED_SAVINGS`	Realised savings	sum of REALISED ledger entries
`M11.REALISATION_RATE`	Realisation rate %	realised / negotiated (target 70)
`M11.SAVINGS_LEAKAGE`	Savings leakage	invoiced above the agreed price or outside the award

10. Demo data

- **AXN signature case: SV-0412** (RFQ-2231 → SA-5520 → PO-77410): negotiated 0.99 Cr/yr (weighted 400.8 vs baseline 410 on 1.08 M units/yr), cost avoidance 0.32 Cr (LME increase absorbed), realised from INV-TF-8812 and later invoices. Operating case 1 shows **leakage** of ₹17,100 on SV-0412 (KD/2702-118 billed 410 against 405). **SV-0413** is a fastener initiative. Validate as `finance.controller`; try as the owner `category.manager` to see the SOD-07 refusal.
- Pack `sourcing_ops` (D17, AXN and KAC, MOCK) adds events in every state, with their initiatives.

11. Known limits and open items

- Validation and the negotiate command check the built-in role / owner, not only the scope; a custom role with `savings:validate` cannot validate.
- Volume shortfall (the Blueprint's third leakage kind) is not computed; leakage covers off-contract and price drift.
- Realised savings attach to the oldest initiative for the material and agreed supplier only; spend with other suppliers shows as off-contract leakage.
- No ERP posting of budget impact; category plan budgets are read, not changed.

M12 Predictive Intelligence, Agents & Copilot

1. Purpose and scope

M12 (Blueprint §18, TD §13) predicts, recommends and automates routine work inside governed autonomy: a catalogue of AI agents with autonomy levels L0-L4, value and category limits, a per-agent kill switch and a tenant-wide AI switch; the Procurement Copilot with the answer contract (Answer · Evidence · Confidence · Explanation · Recommended action · Permission); an optional LLM gateway that may only rephrase or second-guess deterministic results and always falls back; a model registry with versions, drift monitoring and automatic fallback; and the KPI explorer over all modules. Numbers always come from SQL over the live tables, never from a model.

2. Roles

Role	What they do here	Scopes
CPO, TENANT_ADMIN	Set agent autonomy, value limit and categories; turn an agent's kill switch on/off; switch AI off for the whole organisation; manage models (activate, suspend, train, drift check); see the LLM gateway status	`agents:manage`
Every internal user	Ask the Copilot; see agents, runs and recommendations; give feedback on recommendations (accept/reject); trigger on-demand runs for agents whose underlying scope they hold	`pic:read`
Supplier users	Ask the Copilot about their own PO / invoice / RFQ numbers only	`supplier:portal`
Agent identities	Each agent runs as a named role (e.g. `INVOICE_EXCEPTION` as AP_CLERK) and holds only that role's scopes	

SoD applies to agents: before an agent acts, the SoD rules are evaluated for the agent's own identity and for the person who triggered it; a conflict is logged as a violation and the agent is denied.

3. Screens

Screen (BP ID)	Route under /pic	What it shows	Actions
AIX-01 Procurement Command Centre	`/pic` (home page, header "AIX-01 · Procurement command centre")	CPO view: spend, savings, risk, pipeline, exceptions	
AIX-02 Copilot	`/copilot`	Conversation; each answer with evidence links, confidence label, method, model version, data freshness, recommended action and permission	Ask
AIX-03 Agent Console	`/agents`	Autonomy levels (L0 off … L4 autonomous within policy), agents with run-as role, value limit, categories, kill switch, model version, acceptance rate; recommendations inbox; AI switch card; model registry and drift monitoring; LLM gateway card	Autonomy (Save autonomy), Turn kill switch on/off (reason required), AI switch (reason required), accept/Reject recommendation, activate/suspend model

Screen (BP ID)	Route under /pic	What it shows	Actions
KPI explorer (BP Table 28)	`/kpis`	Every module KPI with formula, source, target/warn thresholds, value and month-end trend; filters by module, period, plant	
Module KPI strips	headers of `/receiving` (M09), `/invoices` (M10), `/savings` (M11), `/agents` (M12), `/admin` (M13)	The module's KPIs	

4. End-to-end workflow

A. An agent acts (every call site uses `agents.permit`).

1. A business event or screen triggers an agent (e.g. the match raises exceptions → `INVOICE_EXCEPTION`; a posted invoice → `SAVINGS_VALIDATION`; demand data → `DEMAND_SENSING`).
2. `agents.guard` checks, in order: tenant AI switch on; agent kill switch off; agent autonomy ≥ the level the action needs; the run-as role holds the scope; value ≤ the agent's `value_limit_minor`; category in the agent's categories; automatic suspension (acceptance below the minimum); SoD for the action (agent and requesting user).
3. Allowed: the run is stored (`agent_run`: inputs, evidence, model version, output, confidence, action taken `DRAFTED` / `EXECUTED`). Denied: a `DENIED` run with the reason; the human path continues unchanged.
4. L1/L2 output becomes a **recommendation** for a candidate role (event `agent.recommendation.created`, consumer `notify-recommendation`). The user accepts or rejects it in the Recommendations inbox (`POST /v1/recommendations/{id}/feedback`); the feedback feeds the acceptance rate.
5. **Automatic suspension:** with at least `minFeedback` (20) reviewed suggestions in the last `window` (100) and an acceptance rate below the agent's minimum (70 % by default, rule `AI-CONTROL`), the agent's kill switch is turned on, the reason stored, audited, event `agent.suspended` (consumer `notify-agent-suspended`). Turning the switch off (`PUT /v1/agents/{code}/kill-switch`, reason) restarts the window.
6. **Tenant AI switch:** `PUT /v1/admin/ai/switch` (reason required, ETag) stops every agent at once; event `ai.switch.changed`. Work falls back to the human role.
7. **On-demand runs:** `POST /v1/agents/{code}/runs` exists for `DEMAND_SENSING` (needs `mrp:run`), `SUPPLIER_RISK` (`supplier:manage`), `SPEND_CLASSIFICATION` (`spend:read`), `SOURCING_STRATEGY` (`sourcing:manage`), or `agents:manage`; other agents run on their trigger events (422 `NOT_RUNNABLE`). A switched-off agent answers 409 `KILL_SWITCH`.

B. Copilot (`/copilot`, `POST /v1/copilot/ask`).

1. The question is routed deterministically inside the user's tenant transaction (row-level security) and scopes: a document number (`PO-…`, `REQ-…`, `RFQ-…`, `SA-…`, invoice numbers starting `INV`, `HTS` or `KD`) → document lookup; "task / approval / pending / inbox / waiting" → the user's open and overdue tasks; "spend / maverick" → spend cube (needs `spend:read`); "saving" → savings register; "exception / blocked invoice / payment block" → open invoice exceptions; "touchless / first-pass / kpi" → KPIs; a supplier code or name → supplier profile.
2. The answer contract is built: answer, evidence (type, ref, link), confidence score and label (≥ 0.8 high, ≥ 0.5 medium, else low, with a note), explanation (method, `modelVersion` `copilot-rules-v1`, data freshness), recommended action, permission (role, autonomy L1, scopes).
3. If the LLM gateway is configured, the confidence is ≥ 0.5 and the user is not a supplier, the gateway may rephrase the answer: its JSON must contain every number of the deterministic answer and no other, ≤ 1,200 characters. Any failure keeps the deterministic answer; a rephrased one says "+ gateway phrasing" and keeps `deterministicAnswer`.

C. LLM gateway. `PIC_LLM_GATEWAY_URL` points to a LiteLLM-compatible gateway (chat and embeddings). Without it every caller keeps its deterministic path. Before a call: global kill switch `PIC_LLM_KILL_SWITCH`, per-tenant rate (`ratePerMinute` 30) and monthly budget (`monthlyBudgetMicro`, 0 = unlimited) from rule `LLM-GATEWAY`; every message is redacted (GSTIN, PAN, bank account,

IFSC, e-mail, phone, Aadhaar, card, UPI). The call log keeps counts and SHA-256 digests, never the text, for `retentionDays` (90). Outcomes `ERROR` / `FALLBACK` (validator refused the answer) fall back. Other gateway users: spend classification second opinion and contract redline suggestion (`ai_assist.py`); neither writes business data. Status: `GET /v1/ai/gateway` (`agents:manage`).

D. Model registry and drift. Each model is a row (code + version) with status `CANDIDATE` , `ACTIVE` (one per code), `RETIRED` , `SUSPENDED` . Built-in baselines are registered so a fallback always exists: `demand_forecast` (exponential smoothing `demand-es-v1` ; shipped `demand-sgb-v1`), `late_delivery` (`eta-v3` twin), `price_trend` (`price-trend-none` ; shipped `price-holt-v1`), `invoice_exception_cause` (`exception-rules-v1` ; shipped `exc-nb-v1`), `supplier_dedup` (`dedup-rules-v1` ; shipped `dedup-emb-v1`), `spend_classification` (`keywords-v1`), `copilot` (`copilot-rules-v1`). Routes: `GET /v1/ai/models` , `POST /v1/ai/models/train` , `.../{code}/{version}/activate` , `.../suspend` , `POST /v1/ai/models/{code}/drift-check` . Drift: PSI ≥ 0.25 or an accuracy drop ≥ 0.10 against the offline accuracy suspends the model and re-activates its predecessor (events `ml.drift.alert` , `ml.model.suspended` , `ml.model.activated`); a human re-enables a suspended version deliberately. Prediction endpoints: `POST /v1/ai/late-delivery/refresh` , `GET /v1/ai/price-trend` , `GET /v1/ai/invoice-exceptions/{id}/cause` , `POST /v1/ai/should-cost` , `POST /v1/ai/duplicate-score` .

E. KPI explorer. `GET /v1/kpis?module=M01..M13|TEMPLATE&period=&plant=&trend=` computes each KPI from live tables (value `null` , status `NA` when there is nothing to measure; never an invented number); `GET /v1/kpis/registry` , `GET /v1/kpis/{code}` . Targets/warn levels default in code and are overridden per code in rule `KPI-THRESHOLDS` . Month-end values are persisted by `POST /v1/admin/kpis/snapshot` (scope `admin:integration`).

5. Document states

Object	States
Agent	autonomy `L0` - `L4` ; `kill_switch` on/off (manual or automatic suspension with reason); tenant flag `AI_SWITCH` enabled/disabled
Agent run	action taken `DRAFTED` , `EXECUTED` , `DENIED` ; human feedback `ACCEPTED` / `REJECTED`
Recommendation	`OPEN` → accepted or rejected through feedback (listing defaults to `OPEN`)
ML model version	`CANDIDATE` → `ACTIVE` → `RETIRED` (replaced) or `SUSPENDED` (drift / admin); predecessor re-activated on suspension

6. Business rules and controls

Agent (code)	Default autonomy	Runs as	Limit
Demand Sensing (`DEMAND_SENSING`)	L1	MRP_CONTROLLER	
MRP Exception (`MRP_EXCEPTION`)	L2	MRP_CONTROLLER	
Guided Buying (`GUIDED_BUYING`)	L2	REQUESTER	
Material Dedup (`MATERIAL_DEDUP`)	L2	MDM_STEWARD	
Spend Classification (`SPEND_CLASSIFICATION`)	L3	CATEGORY_MANAGER	
Sourcing Strategy (`SOURCING_STRATEGY`)	L1	CATEGORY_MANAGER	
Bid Analysis (`BID_ANALYSIS`)	L2	CATEGORY_MANAGER	

Agent (code)	Default autonomy	Runs as	Limit
Contract Review (`CONTRACT_REVIEW`)	L2	CONTRACT_MANAGER	
PO Validation (`PO_VALIDATION`)	L3	BUYER	
Touchless PO (`TOUCHLESS_PO`)	L4	BUYER	₹25 L value limit
Expediting (`EXPEDITING`)	L2	BUYER	
Invoice Exception (`INVOICE_EXCEPTION`)	L2	AP_CLERK	
Supplier Risk (`SUPPLIER_RISK`)	L1	SUPPLIER_MANAGER	
Savings Validation (`SAVINGS_VALIDATION`)	L3	CATEGORY_MANAGER	
Procurement Copilot (`COPILOT`)	L1	CATEGORY_MANAGER	

- Blueprint rule "L3/L4 only for configured low-risk categories and values": the screen requires at least one category for L4; the guard enforces value and category limits when set.
- Rules: `AI-CONTROL` (`minFeedback` 20, `window` 100; the minimum acceptance is per agent, 70 % by default), `LLM-GATEWAY`, `KPI-THRESHOLDS` (`minutesSavedPerAgentRun` 6 for Hours saved), `DEMAND-SENSING` (`alpha` 0.3, `biasThreshold` 0.15, `horizon` 3).
- Every switch change needs a reason and is audited; the AI switch write carries an ETag.

7. Integrations and events

Events: `agent.recommendation.created` , `agent.suspended` , `ai.switch.changed` , `ml.drift.alert` , `ml.model.activated` , `ml.model.suspended` . External: the LLM gateway only (no model vendor is called directly); the twin (`/v1/twin/status` , `/twin/refresh` , scorecard write-back) feeds late-delivery probability.

8. Scheduled jobs

None specific to M12. Drift checks, model training and KPI snapshots run on demand (see §11).

9. KPIs

Code	Name	Formula
`M12.AGENT_ASSISTED`	Agent-assisted transactions %	agent-executed decisions / all decisions
`M12.RECOMMENDATION_ACCEPTANCE`	Recommendation acceptance %	accepted / decided recommendations
`M12.PREDICTION_ACCURACY`	Prediction accuracy %	100 − forecast MAPE
`M12.HOURS_SAVED`	Hours saved	executed agent runs × minutes per run (rule `KPI-THRESHOLDS`)

10. Demo data

- AXN: the agent catalogue with runs from the signature case (e.g. the `INVOICE_EXCEPTION` draft on INV-TF-8812, the `TOUCHLESS_PO` release of PO-77410, a Demand Sensing recommendation accepted and released to MRP). Ask the Copilot "INV-TF-8812", "my pending approvals", "open invoice exceptions" or "savings".
- Pack `ai_eval` (D9): labelled spend lines, forecast actuals, duplicate pairs and Copilot questions for evaluation runs (`tools/ai_eval.py` , `tools/ai_train.py`).

11. Known limits and open items

- No scheduled drift check, model retraining or KPI month-end snapshot: `kpi.py` says "the scheduler calls it at month end", but no job is registered; run `POST /v1/admin/kpis/snapshot` and `POST /v1/ai/models/{code}/drift-check` manually or add jobs.
- The Copilot is intent-routed by keywords; invoice numbers are recognised only with the prefixes `INV` , `HTS` , `KD` (other tenants' invoice numbers, e.g. KAC's, are not detected as documents).
- Without `PIC_LLM_GATEWAY_URL` no model is used anywhere; with it, a live gateway has not been tested in this repository's CI.
- On-demand runs exist for four agents only.

Platform & Enterprise Administration

1. Purpose and scope

The platform layer (Blueprint M13 §19, screens ADM-01..ADM-06; TD §3-§4 tenancy and security, §8 workflow, §10 API, §11 events, §12 integration, §14 audit and privacy) is what every module runs on: tenants and their provisioning; the organisation masters; users, roles, custom roles, scopes, SoD and ABAC; effective-dated rule sets; the workflow engine with its designer, task inbox, escalation and delegation; the hash-chained audit trail; idempotency and ETags; the transactional outbox, worker and scheduled jobs; the ERP integration hub (connectors, SAP mapping, field mappings, inbound sync, cutover loader, onboarding pack); industry templates; notifications; privacy; dataset packs; and the GitHub → Neon setup path.

2. Roles

Role / actor	What they do here	Scopes
Platform operator (no tenant role)	Provisions tenants and reissues the first admin's link from the command line; runs `neon_setup`, `erp_load`, `onboarding_pack`, `stock_opening_load`; owns GitHub secrets and Neon	database access (`DATABASE_URL`, `PIC_FIELD_KEY`)
TENANT_ADMIN	Organisation masters, users, custom roles, templates, rule versions, SoD modes, mandatory fields, workflow designer, integration monitor and replay, connectors and field mappings, ERP sync, jobs, delegations, audit, privacy requests, AI switch, KPI snapshots	`admin:rules`, `admin:audit`, `admin:integration`, `admin:ops`, `admin:workflow`, `agents:manage` (platform scopes held only by this role; MFA enforced by the provisioning tool)
PROCUREMENT_HEAD, CPO	Approve SoD mitigation requests (rule `SOD-MITIGATION`: Low/Medium PROCUREMENT_HEAD; High PROCUREMENT_HEAD then CPO; Critical never)	`tasks:work`
Every internal user	Works the task inbox; requests an SoD mitigation; delegates their own approvals; reads organisation, permission matrix, notifications	`pic:read`, `tasks:work`

Built-in roles (22, read-only, `platform/auth.py ROLE_SCOPES`): REQUESTER, COST_OBJECT_OWNER, DEPT_HEAD, MRP_CONTROLLER, BUYER, CATEGORY_MANAGER, SUPPLIER_MANAGER, CONTRACT_MANAGER, LEGAL, MDM_STEWARD, RECEIVER, INVENTORY_CONTROLLER, QUALITY_INSPECTOR, SERVICE_OWNER, AP_CLERK, FINANCE_CONTROLLER, PROCUREMENT_HEAD, CPO, CAPEX_COMMITTEE, CLINICAL_LEAD, TENANT_ADMIN, SUPPLIER_USER.

3. Screens

Screen (BP ID)	Route under /pic	What it shows / does
ADM-01 Organisation & Tenant Setup	`/admin` tab **Organisation (ADM-01)**	Company codes, plants (with holidays), storage locations, cost objects, purchasing orgs and groups, currencies, FX rates; create, edit, deactivate/activate, delete
ADM-02 Rules & Thresholds	tabs **Rules & thresholds**, **Mandatory fields**	Every rule set version with key, version, scope, effective dates, body; "New version"; mandatory field sets per document type

Screen (BP ID)	Route under /pic	What it shows / does
ADM-03 Workflow & Approval Designer	tabs **Approval matrix (ADM-03)**, **Approval simulation (ADM-03)**, **Workflow designer**	DMN-01 grid saved as a new version; simulation of an approval route for a date and scope; versioned process definitions F1-F9 (draft, validate, publish, retire, diff, BPMN import/export)
ADM-04 Roles & SoD	tabs **Users & roles (ADM-04)**, **Permission matrix**, **Roles & SoD**	Invite / import users, attributes, resend link, password reset, revoke sessions, reset MFA, activate/deactivate; built-in and custom roles with scopes and SoD preview; SoD rules with mode, violations, mitigations
ADM-05 Integration Monitor	tabs **Integration monitor**, **ERP connectors & imports**, **ERP field mappings**, **ERP sync**	Messages by status with attempts and last error, Replay; connector configuration and test; mapping drafts, preview, publish, retire; sync runs per object kind
ADM-06 Audit Explorer	tab **Audit explorer**	Audit events by object/actor/action; hash-chain verification; archive
(operations)	tabs **Operations**, **Jobs**, **Delegations**, **Industry templates**	Run worker tick now; jobs with schedule, last run, failures, dry run; all delegations; template preview/apply
Task inbox	`/tasks`	Open tasks the user may work (own role, escalation, delegation); approve / reject / return with comment; bulk complete
Process monitor	`/processes`	Running and finished process instances with the step where the token waits
Notifications	`/notifications`	Unread / all; mark read, mark all read
Account	`/account`	Authenticator (MFA) enrolment, own delegations of authority, password change
Activation	`/activate?token=…`	Public page to set the password from an invitation or reset link
Industry operations	`/industry`	Template-specific panels (Kanban, ERS, advances, guarantees, licences, accessorials, cold chain, par levels, P-cards) shown only when the template enables them

4. Key administration procedures

P1. Provision a tenant (operator, command line only; no web API by design).

1. On a machine with the environment's `DATABASE_URL` and `PIC_FIELD_KEY`, from `backend/`: `python -m app.pic.tools.tenant_admin create --code ACME --name "…" --template MANUFACTURING|CONSTRUCTION|IT_SERVICES|LOGISTICS|HEALTHCARE|GENERAL --admin-email … [--admin-name] [--company-code 1000] [--company-name] [--gstin] [--plant P100] [--plant-name] [--state-code 33]`.
2. It creates the tenant (MFA enforced for approver, finance and admin roles), number ranges, currencies, SoD rules, agents, the template's rule sets, one company code, plant and storage location `0001`, and the first administrator without a password; it prints the single-use activation link once (also e-mailed when `PIC_SMTP_HOST` is set; only its SHA-256 is stored). Audit `TENANT PROVISIONED` by `agent:tenant_admin_tool`. Re-running for a tenant that has users exits with code 2.
3. The admin opens `/pic/activate?token=…` (valid `PIC_ACTIVATION_HOURS`, 72), sets a password (≥ 12 characters, not common or breached), signs in with the organisation code and enrols an authenticator. `tenant_admin reissue` makes a new link; `tenant_admin list` lists tenants.

P2. Build the organisation (ADM-01, `/v1/admin/org/{kind}`). Kinds `company-codes`, `plants`, `storage-locations`, `cost-objects`, `purchasing-orgs`, `purchasing-groups`, `currencies`. Validation: company code 1-10 characters, GSTIN format/state/check character, active currency (fixed once documents exist); plant with an active company code, GST state 01-38 or 97, working days; storage

location unique per plant; cost object type `COST_CENTER` / `WBS` / `ORDER` / `ASSET`; purchasing org belongs to a company code; a group may name an org, a plant and buyer users. Codes never change. Holidays: `POST /v1/admin/org/plants/{id}/holidays` (source mandatory, add-only; delete with a reason). `DELETE` deletes an unreferenced record and deactivates a referenced one (`outcome: DELETED | DEACTIVATED`; 409 `HAS_ACTIVE_CHILDREN`). Inactive records are refused on every new requisition, PO and GR (422 `ORG_RECORD_INACTIVE`). Bulk path: pack **`org_masters` (D19)** loads company codes (legal entities with GSTIN, currency, fiscal-year start), purchasing orgs and purchasing groups (with buyer users) for every tenant from `data/reference/org_masters.csv`, audited; the client's own file is the onboarding kit template `PIC_MAP_ORG_MASTERS.csv`, applied with `onboarding_pack --apply-org-masters`.

P3. Onboard a user (ADM-04). `POST /v1/admin/users {email, name, roles, attributes}`; attributes (ABAC) `plants`, `company_codes`, `cost_objects` (`COST_CENTER:CC-100`), `categories`, `steward_domains` (codes or `*`), `value_limit_minor`, `approval_limits`. Role combinations that span an SoD pair are refused (409 `SOD_CONFLICT`). The response carries the activation link once; the user is INVITED until used (`resend-invitation` revokes the old link: `LINK_REVOKED`). Bulk: `POST /v1/admin/users/import {"csv": …, "dry_run": true}` (columns `email,name,roles,plants,company_codes,value_limit_minor`, ≤ 500 rows, all-or-nothing). Lifecycle: `PUT …/roles` and `…/attributes` (reason; ends sessions), `PUT …/active`, `POST …/password-reset` (single-use link, `PIC_RESET_HOURS` 2), `POST …/revoke-sessions` (next request 401 `SESSION_REVOKED`), `POST …/mfa/reset`. Sign-in: `POST /v1/auth/token`, MFA `POST /v1/auth/mfa`, optional OIDC SSO `/v1/auth/oidc/{tenant}/start`. Rule `MFA` lists the roles that must enrol (`enforce` per tenant).

P4. Define a role. `/admin` → Permission matrix. `GET /v1/admin/roles/matrix` gives the assignable scope catalogue; platform scopes (`admin:*`, `agents:manage`) and supplier-portal scopes are never assignable (422 `SCOPE_NOT_ASSIGNABLE`). `POST /v1/admin/roles/preview {code, scopes}` shows the SoD impact; create `POST /v1/admin/roles` (code 3-40 capitals/digits/underscore, not a built-in name). A role may not span a scope-level SoD pair: SOD-02 (`supplier:bank` + `payment:run`), SOD-03 (`po:create` / `po:release` + `gr:post`), SOD-04 (`po:create` / `po:release` + `invoice:capture` / `invoice:resolve` / `invoice:post`), SOD-11 (`inventory:count` + `inventory:approve`), and no current holder may end up spanning one. Changing, deactivating or reactivating a role ends its holders' sessions; delete removes an unused role, otherwise deactivates it. Custom roles can be approver roles in workflow definitions.

P5. Publish a rule version (ADM-02). Rules are never edited. `POST /v1/admin/rules {key, version, effective_from, [effective_to], [company_code|plant|category|purchase_type|supplier_group], body, description, [supersedes]}` (scope `admin:rules`). Resolution on the business date: most specific scope match wins, then latest effective date, then the version created last. A version label is used once per key; a second version with the same scope and date must name the one it `supersedes` (else 409). Inventory, sourcing and spend keys are validated (422 `RULE_INVALID`). Documents store the `rule_set_version` they used, so in-flight documents finish on their version. Test with `POST /v1/admin/rules/simulate` (Approval simulation tab) for a date after `effective_from`. Mandatory fields: `POST /v1/admin/mandatory-fields`. Industry templates: `GET /v1/admin/templates/{code}/preview` (rules added / changed / reverted / left alone because customised), `POST …/apply` (new versions only; audit `TEMPLATE_APPLIED`; idempotent `noop`).

P6. Design a workflow (ADM-03). `POST /v1/process-definitions/{key}/drafts` (scope `admin:workflow`) → edit `PUT …/versions/{version}` (steps, flows, lanes, branches; ≤ 120 steps) → `POST …/validate` (structure, every step reachable and able to reach an end, gateways with both outcomes, roles known, task kinds with a code handler, service-driven step ids kept) → `POST …/publish` (event `process.definition.published`; new instances start on it, running instances keep theirs) → `retire`; `GET …/diff`; BPMN in/out (`…/bpmn`, `POST /v1/process-definitions/bpmn/parse`). Approval chains come from rule sets (e.g. `DMN-01` bands edited in the Approval matrix grid).

- **Task inbox** (`/tasks`, `GET /v1/tasks`, `POST /v1/tasks/{id}/complete` with `APPROVE | REJECT | RETURN`, comment mandatory for reject/return; `POST /v1/tasks/bulk-complete`). Four-eyes: earlier approvers and initiators in a chain are excluded.
- **Escalation** (rule `ESCALATION`, SLA timers fired by the worker): level 1 reminds, level 2 adds the manager role (`managers` map, e.g. AP_CLERK → FINANCE_CONTROLLER, INVENTORY_CONTROLLER → DEPT_HEAD), level 3+ reassigns (`mode` `{"2": "ADD", "default": "REASSIGN"}`); never to a role nobody eligible holds.
- **Delegation** (`/account` → Delegation of authority, `POST /v1/me/delegations`; admins `POST /v1/admin/delegations`): delegate is another active internal user; period from today, ≤ `DELEGATION.maxDays` (90); roles ⊆ the delegator's; optional value limit; no overlaps or two-way delegations; four-eyes still applies per task. Events `delegation.created/revoked`.

P7. Handle a stuck message or approval.

1. Integration monitor (`GET /v1/integration/messages`, filter `DEAD_LETTER`). Messages are written in the business transaction (`PENDING`), posted by the worker, retried with exponential back-off (`RETRY`) and dead-lettered after 8 attempts (`DEAD_LETTER`, audit `INTEGRATION_DEAD_LETTER`). Before any retry the connector looks the document up by its PIC reference, so a lost response

never double-posts.
2. Many messages with one error → connector problem: ERP connectors tab → Test connection (`POST /v1/admin/connectors/{system}/test`), credentials, egress. One document → data problem: fix master data first.
3. Replay (`POST /v1/integration/messages/{id}/replay` , scope `admin:integration`): back to `PENDING` , audit `INTEGRATION_REPLAY` then `ERP_POSTED` . Never edit a payload; correct the business document instead (it enqueues a new key).
4. Worker down: `/readyz` → `checks.worker` ; Operations → Run worker tick now (`POST /v1/admin/ops/tick`). Outbox events that fail 8 times are dead-lettered too.
5. Stuck approval: the process view shows where the token waits; give the role/attributes to an eligible user (never complete it as someone else); a wrong chain is corrected by a new rule version.

P8. Connect the ERP. Target system per company code from rule `ERP-TARGET` (default `SAP_S4`). Connectors: `SAP_S4` (simulator until `PIC_ERP_SAP_S4_URL` or an IDoc directory is set; transports `odata` , `idoc` , `idoc-file` ; `COMPANY_MAP` , purchasing org/group, tax code, GL accounts, reason and stock-type maps), `ORACLE_FUSION` , `DYNAMICS_365` , `TALLY_PRIME` , `ODOO` . A message type a connector does not map is reported (`connectors.require_mapped` ; inventory documents record `ERP_NOT_MAPPED`). SAP mapping (`platform/sap_mapping.py`): released OData APIs for PR, PO, GR, invoice, credit memo, supplier (business partner), material, stock posting; IDocs PORDCR105, MBGMCR03, INVOIC02, CREMAS05, MATMAS05, FIDCCP02 (payment). Field mappings (`/v1/admin/field-mappings` : draft, preview, publish, retire) adjust payloads per tenant (e.g. SAP alpha conversion).

- **Cutover:** `python -m app.pic.tools.erp_load --tenant KAC --dir data/erp_mock/KAC [--validate-only] [--report]` loads an SAP extract (one CSV per table + MANIFEST.json, spec `docs/PIC_ERP_Extract_Spec.md`) in order org → vendors (GSTIN/bank encrypted, duplicate and sanctions screening) → materials (HSN/GST checked) → agreements → open items.
- **Inbound delta sync:** `POST /v1/admin/erp-sync/{kind}/run` (pull through the connector or a file drop) or `/payload` (rows pushed by an integrator); kinds `PLANTS` , `COST_OBJECTS` , `VENDORS` , `MATERIALS` , `STOCK` , `OPEN_POS` , `PAYMENT_STATUS` , `PLANNED_ORDERS` ; delta cursor with one day of overlap, content hash per object, rejects reported; bank and tax-ID changes on existing vendors become MDM-04 change requests, never applied directly; `POST /v1/admin/erp-sync/retry-failed` ; event `erp.sync.finished` . Planned orders also via `POST /v1/integration/planned-orders/import(/csv)` (event `integration.batch.imported`).
- **Onboarding pack** (`python -m app.pic.tools.onboarding_pack --tenant KAC|--all-tenants [--apply-mappings] [--apply-org-masters] [--apply-all] [--dry-run] [--validate-only]`): validates and stores the client's production field mappings from `data/onboarding_kit/rules` (SAP_S4 PURCHASE_ORDER, SUPPLIER_INVOICE, GOODS_RECEIPT, PAYMENT; ORACLE_FUSION PURCHASE_ORDER) and the org masters template.

P9. Deploy to Neon (and Vercel).

1. Local first run: `backend\.env` with the **direct** (non-pooler) Neon URL, `PIC_JWT_SECRET` , `PIC_FIELD_KEY` ; `python -m app.pic.tools.neon_setup` runs (1) Alembic migrations (head is 0022 today), (2) the AXN signature-case seed if AXN does not exist (demo users `…@axion.example` , password from `PIC_SEED_PASSWORD` or printed once), (3) the legacy snapshot copy and migration into tenants GMC and DBY with a reconciliation report, then every dataset pack once per tenant, (4) verification: row counts, RLS coverage, audit chain per tenant, pending events. Safe to re-run; `--verify-only` , `--skip-seed` , `--skip-legacy` , `--skip-datasets` , `--fx-rate` .
2. GitHub: workflow `neon.yml` on push to `main` (paths `backend/**`) runs the same `neon_setup` in environment `neon-production` (secrets `DATABASE_URL` , `PIC_JWT_SECRET` , `PIC_FIELD_KEY` , `PIC_SEED_PASSWORD` ; the job fails if one is missing); pull requests get a throwaway Neon branch (secret `NEON_API_KEY` , variable `NEON_PROJECT_ID`) deleted on close. `ci.yml` runs migrations, the test suite with a coverage gate, `tsc` , ESLint, `npm audit` and the build; `security.yml` weekly; `release.yml` builds images.
3. Vercel: API project with `PIC_INLINE_WORKER=true` (drain after every write) and `CRON_SECRET` for `/internal/worker/tick` ; `backend/vercel.json` (region `sin1`) schedules the tick every minute (Hobby plans only allow a daily cron).

5. States of platform objects

Object	States
User	INVITED → active; deactivated; erased (pseudonymised); links `LINK_USED` / `LINK_EXPIRED` / `LINK_REVOKED`

Object	States
Custom role	active ↔ inactive; deleted when unused
Org record	active ↔ inactive (deactivate/activate); deleted when unreferenced
Rule set version	effective from/to; never edited; superseded by a later or more specific version
Process definition version	draft → published → retired
Task	`OPEN` → `COMPLETED` (outcome APPROVED / REJECTED / RETURNED) or `CANCELLED`; escalated (candidates added or reassigned)
SoD rule	mode `BLOCK` or `MITIGATE`, active/inactive; mitigation request → approved grant (time-boxed, use-limited) or rejected
Integration message	`PENDING` → `POSTED`; `RETRY` → `DEAD_LETTER` (after 8) → replay → `PENDING`
Outbox event	pending → published; dead-lettered after 8 attempts
Scheduled job run	running → success / partial (per-unit errors) / failed

6. Business rules and controls

- **Tenancy:** the tenant comes from the token, never the payload; PostgreSQL row-level security on every tenant table (`pic_app` role).
- **RBAC + ABAC:** the route checks the scope; services call `abac.guard` with actions (`REQ_*`, `PO_*`, `GR_POST`, `INVOICE_*`, `PAYMENT_RUN`, `CONTRACT_SIGN`, `AWARD_APPROVE`, `SUPPLIER_CHANGE`, `MASTER_CHANGE`, `STOCK_MOVE`, `INVENTORY_*`, `PERIOD_CLOSE`) on company code, plant, category and value limit (company currency). Rule `ABAC` mode `ENFORCE` (403) | `AUDIT` | `OFF`; a user without an attribute or with `*` is unrestricted on that dimension; denials are audited in their own transaction.
- **SoD rules** (tenant data, evaluated at command time, violations kept even when the command rolls back): SOD-01 supplier creator vs PO (High), SOD-02 bank change vs payment (Critical), SOD-02a bank change vs invoice posting (Critical), SOD-03 PO vs GR, SOD-04 PO vs invoice exception, SOD-05 sole evaluator vs award (Medium), SOD-06 material requester vs steward (Medium), SOD-07 savings owner vs validator, SOD-08 invoice capture vs approval, SOD-09 invoice approval vs payment, SOD-10 stock posting maker vs approver, SOD-11 counter vs count approver (all High unless stated). `PUT /v1/admin/sod/{code}` switches BLOCK ↔ MITIGATE (Critical cannot be mitigated, 422 `NOT_MITIGABLE`). Mitigation (`POST /v1/sod/mitigations`, justification ≥ 20 characters) → approver task → grant valid `maxValidityHours` (Low 168, Medium 72, High 24) for `maxUses` (5/3/1) with the compensating-control text.
- **Idempotency-Key** on every `/v1` write: reserved per user; a concurrent duplicate is 409 `REQUEST_IN_PROGRESS`; same key with a different body 422 `IDEMPOTENCY_KEY_REUSED`; 2xx replayed for 24 h (`Idempotent-Replayed: true`); failures release the key. **ETag / If-Match** on every mutable document (`row_version`): stale 412 `STALE_WRITE`; required (428) on listed routes when `PIC_REQUIRE_IF_MATCH` is on.
- **Audit:** append-only (the database role cannot update or delete), hash chain `sha256(prev_hash + canonical event)` per tenant, verified by `GET /v1/audit/verify`; archive to `audit_event_archive` in verified batches (`POST /v1/admin/audit/archive`, refused if the chain is broken); statutory retention `audit_actor.days` (2920 days).
- **Outbox:** events are written in the business transaction; consumers are idempotent (`processed_event`); event catalogue and JSON schemas derived from code (`python -m app.pic.events.catalogue write`, `docs/PIC_Event_Catalogue.md`).
- **Privacy (DPDP):** `GET /v1/admin/privacy/users/{id}/export`, `POST …/erase` (pseudonymise, keep id; refused with open tasks, for oneself or the last admin), `GET /v1/admin/privacy/retention`, `POST /v1/admin/privacy/retention/purge`; rule `PRIVACY-RETENTION` (dry run until `purge.dryRun` false).
- **Notifications:** in-app for the kinds in rule `NOTIFICATIONS` (TASK_ASSIGNED, TASK_ESCALATED, RECOMMENDATION, PO_TRANSMITTED, INVOICE_EXCEPTION, …); e-mail when SMTP is configured and `email` is true; admins notified after 3 consecutive job failures.

7. Integrations and events

Platform events: `process.definition.published`, `delegation.created/revoked`, `sod.mitigation.decided`, `erp.sync.finished`, `integration.batch.imported`, `ai.switch.changed`. Internal endpoints: `/internal/worker/tick` (Bearer `CRON_SECRET`), `/readyz`, `/metrics` (`pic_job_*`, `pic_integration_messages`, `pic_outbox_lag_seconds`, `pic_worker_heartbeat_age_seconds`). Worker: `python -m app.pic.worker` (outbox relay, workflow timers, ERP retries, jobs; heartbeat in `application_name`; use the direct Neon host; run one worker only).

8. Scheduled jobs

Job key	Schedule (tenant time zone)	What it does
`payment-due`	06:30 daily	payment proposal task (M10)
`erp-reconciliation`	01:30 daily	ERP reconciliation per company code (M10)
`mrp-schedule`	02:00 daily	MRP for the plants in rule `MRP-SCHEDULE` (none until listed)
`inventory-reservations`	02:45 daily	reservation expiry (M09)
`scorecard-monthly`	03:00 on the 1st	supplier scorecards of the previous month
`retention-purge`	03:30 daily	privacy retention purge (dry run until enabled)
`audit-archive`	04:00 Sunday	audit archive (dry run until enabled)
`idempotency-purge`	hourly :15	expired idempotency records
`spend-cube-refresh`, `spend-sla`, `spend-quality`, `spend-forecast-snapshot`	every 5 min; hourly :15; 05:00 daily; 05:30 on the 2nd	spend module jobs

One run per job and tenant at a time (claim row + advisory lock; an abandoned claim older than the timeout is taken over); failures roll back that run only. Admin: `GET /v1/admin/jobs`, `PUT /v1/admin/jobs/{key}` (schedule, on/off), `POST …/run {"dryRun": true}`, `GET …/runs`.

9. KPIs

Code	Name	Formula
`M13.INTEGRATION_SUCCESS`	Integration success %	messages posted / messages
`M13.MTTR_INTEGRATION`	Mean time to resolve integration error (h)	first attempt to posting, for messages that needed a retry
`M13.SOD_OPEN`	SoD conflicts open	violations blocked or awaiting mitigation

10. Demo data

- Tenants after `neon_setup`: AXN (signature case and packs D1, D2, D8, D13-D17), industry demo tenants CON, ITS, LOG, HLT, GEN (D3-D7), KAC (mock SAP S/4HANA cutover D10, contracts D12, delta sync D18), legacy tenants GMC and DBY; `reference_data` (D11: HSN/SAC and GST rates, holiday calendars, UNSPSC, FX) and `org_masters` (D19) on every tenant. Packs are recorded per tenant in `pic.dataset_load` and skipped when loaded; `python -m app.pic.seed.datasets --only <pack>` loads one.
- Try: sign in to AXN as `admin` → Integration monitor (the INV-TF-8812 posting message), Audit explorer → verify chain, Roles & SoD (SOD rules and violations), Jobs → dry run `payment-due`; KAC → ERP sync tab (D18 runs).

11. Known limits and open items

- No scheduled job runs the ERP inbound sync: the `erp_sync.py` docstring says the scheduler registers `sync_masters`, `sync_stock`, `sync_open_pos`, `sync_payment_status`, but no `@jobs.job` exists for them; syncs run on demand from the admin screen or API.
- Running two workers concurrently is not proven safe (timers, ERP retries).
- Approval matrix grid edits `DMN-01` (requisitions) only; other chains are edited as rule JSON.
- Outdated docs: `docs/runbooks/admin_operations.md` says 21 built-in roles and lists three scope-level SoD pairs (code: 22 roles, four pairs incl. SOD-11); `docs/runbooks/privacy-requests.md` says the retention purge is not scheduled and that a token stays valid after deactivation (code: job `retention-purge` exists; every request re-checks the user, so a deactivated user gets 401 `ACCOUNT_DISABLED`, a revoked or reset one 401 `SESSION_REVOKED`, a role change 401 `SESSION_OUTDATED`); `docs/PIC_Admin_Module_Analysis.md` (1 Nov) says organisation setup is read-only, purchasing org/group, user creation, custom roles, template apply and provisioning are missing (all built since); `docs/PIC_GitHub_Neon_Setup.md` and the `neon_setup` docstring describe migrations 0001-0003 (head is 0022); `docs/PIC_Vercel_Deployment.md` says the committed cron is daily `0 3 * * *`, while `backend/vercel.json` schedules `* * * * *`; `docs/runbooks/workflow-incidents.md` is written for Kubernetes (`kubectl`) while production runs on Vercel.

Appendix A. Gaps found in the code while writing this guide

These were found by reading the code at `f354bbe` against the workflows above. None of them is hidden in the module sections; this list collects them in one place so they can be planned. They are observations from reading, not results of new tests.

Planning, master data and requisitions (M01 to M04)

#	Module	Gap
A1	M02	MRP exceptions are never closed (there is no command for it), so the `M02.EXCEPTION_BACKLOG` KPI counts exceptions from every past run.
A2	M02	Planned orders have no cancel or unfirm command, although MRP raises CANCEL messages on firmed orders.
A3	M02	Net-change MRP only picks up materials whose demand changed; a stock or PO change alone does not bring a material in, although the MRP-01 screen says "changed demand or supply".
A4	M02	The PERIOD lot-size policy falls back to the plain net quantity, because the run never passes a period quantity.
A5	M02	MRP run, firm and convert have no ABAC plant check.
A6	M02	The `KANBAN` rule flags `purposeCode` and `autoRequisition` are never read; refilling a card is manual and a goods receipt does not refill it.
A7	M03	Material change requests accept MRP types PD, ND, VB and VM, while the planning engine plans PD, PW, PM, VB and KB; VM is not planned at all.
A8	M03	A material request has no withdraw command, so a DUPLICATE_BLOCKED request nobody acts on stays open.
A9	M03	The screen labels the material change request "MDM-04", which is the Blueprint ID of the supplier master change. No dataset seeds a material change request, so there is no demo case.
A10	M04	Requisitions are never sent to the ERP: the `PURCHASE_REQUISITION` mapping exists but nothing queues it.
A11	M04	Hosted catalogue items can only be created by seeds and datasets; there is no API, screen or loader to maintain them.
A12	M04	Category SOW forms send structured attributes the requisition line does not store; the SOW survives only as the 200-character line description, and the API does not limit its length (risk of a database error on a long text; not verified at runtime).
A13	M04	Promoting a free-text line to a material request has an API but no button.
A14	Platform	Inbound ERP syncs (`erp_sync.py`) are not registered as scheduled jobs, although the code's own description says so; they run only when started.

Sourcing, suppliers, contracts and orders (M05 to M08)

#	Module	Gap
B1	M08	The `PO_CREATE` and `PO_RELEASE` ABAC actions exist but the create and release commands never call them; ABAC applies only to PO approval tasks.

#	Module	Gap
B2	M08	A PO can only be created from a requisition (`source.type = REQUISITION`): there is no direct PO from a contract or planned order and no free manual PO. Every new PO gets purchasing organisation `1000` and currency INR.
B3	M08	The contract quantity is checked when the PO is created, not again at release.
B4	M07, M08	With several live contracts for one material and plant, source determination takes the first row of an unordered query.
B5	M07	The contract state `RENEWED` has no caller (renewal extends the same contract), and the `CONTRACT_EXPIRING` notification in the `NOTIFICATIONS` rule is never produced. Contracts are not posted to the ERP; only rebate credit memos are.
B6	M05, M07	Every sourcing event type is numbered `RFQ-nnnn` , and every contract created in PIC `SA-nnnn` .
B7	M06	ABAC does not restrict supplier master governance in practice: `MASTER_CHANGE` checks company code and plant, which a supplier record does not carry, and the bank-change and master-change tasks are not mapped to a document.
B8	M06	After onboarding, a supplier cannot request its own bank change from the portal; there is no command to change an existing supplier's legal name, payment terms or incoterms.
B9	M05 to M07	No scheduled jobs for supplier risk scans, sanctions re-screening, rebate accrual or offer-validity expiry; these run on workflow timers or by hand. The nightly `erp-reconciliation` job does not cover POs.

Receiving, finance, savings, AI and platform (M09 to M12, Platform)

#	Module	Gap
C1	M09	Return deliveries never leave `CREATED` / `REPLACEMENT_DUE` , even after the credit note or the replacement arrives.
C2	M09, M10	The ERP messages for a GR reversal (102), `PAYMENT_REVERSAL` , `SUPPLIER_DEBIT_MEMO` and `SCF_FINANCING` are mapped for the ERP simulator only; with a live SAP system these stay manual steps. The SAP `STOCK_POSTING` mapping has been tested against the mock only.
C3	M10	Invoice approval is one command with a per-user value limit: no approval chain by value band and no delegation. No partial payments, no Form 16A PDF, no s.195 TDS.
C4	M10, M11	Payment-block release and savings validation check for the built-in FINANCE_CONTROLLER role by name, so a custom role holding `payment:run` or `savings:validate` is refused. Only the initiative owner can record a negotiation.
C5	M11	No volume-shortfall leakage.
C6	M12	Copilot only recognises invoice numbers starting INV, HTS or KD, so KAC-style invoice numbers are not found. Model drift checks and retraining run by hand only.
C7	Platform	No job saves the month-end KPI snapshots the KPI module describes; only `POST /v1/admin/kpis/snapshot` does it.
C8	M09	The `/pic/inventory` page header says "INV-01", which is the Blueprint ID of the Invoice Inbox.

Appendix B. Older documents that are now out of date

This guide follows the code. These documents disagree with it in the points listed and should be read with that in mind (or refreshed).

Document	Out of date in
`docs/PIC_Process_Flows.md`	F1: MRP plans PD, PW, PM and VB (Kanban runs outside MRP), raises RESCHEDULE_OUT, CANCEL and NO_REORDER_PARAMS, and counts consignment stock as on-hand. F3: the TCO figures for RFQ-2231 predate the second-pass TCO engine; two-person and scheduled opening and regret letters are not described.
`docs/PIC_Module_Gap_Analysis.md`	Lists as missing items that are now built: ERP planned-order import (M02), tax-ID and address changes (M03/M06), punch-out and Guided Buying Agent logging (M04), clarification board and live auction screens (M05), external e-signature and clause extraction, all seven contract types (M07), EDI/e-mail transmission and inbound ORDRSP/DESADV (M08).
`docs/PIC_Pending_Workflows.md`	Many items listed as not built or partial are built: scheduled MRP, sales-order account assignment, request templates, material change requests, field ownership, SPD-08, Kanban and reorder point, weekly buckets, monthly scorecards, the one-time-vendor block, the last-price check, receipt tolerances, multi-item and lot bidding, bundling, invitation filters, ESG and risk criteria, master agreements and NDAs, jurisdiction templates, rebates, clause extraction, segmentation and development plans.
`docs/PIC_Admin_Module_Analysis.md`	Lists organisation setup, purchasing organisations and groups, internal users, custom roles, template apply and provisioning as missing; all are built.
`docs/runbooks/admin_operations.md`	There are 22 built-in roles (not 21) and 4 SoD scope pairs (SOD-11 included), not 3.
`docs/runbooks/inventory_operations.md`	Stock is loaded with `/v1/admin/erp-sync/{kind}/run` (or `/payload`); the route `POST /v1/erp/sync` it names does not exist.
`docs/runbooks/scheduled-jobs.md`	`autoRun=true` does not make the `payment-due` job pay: the code ignores it (it reports `autoRunIgnored`) and never pays by itself.
`docs/runbooks/privacy-requests.md`	The `retention-purge` job exists, and every request now re-checks the user, so a token stops working after deactivation (`ACCOUNT_DISABLED`, `SESSION_REVOKED`, `SESSION_OUTDATED`).
`docs/runbooks/workflow-incidents.md`	Assumes Kubernetes (`kubectl`); production runs on Vercel with the cron-driven worker drain.
`docs/PIC_GitHub_Neon_Setup.md`	Describes migrations 0001 to 0003; the head is 0022.
`docs/PIC_Vercel_Deployment.md`	The committed cron in `backend/vercel.json` runs every minute (`* * * * *`), not daily.
`docs/PIC_Finance_Module_Analysis.md` §8.5	The `purchase_types` and `industry_healthcare` packs now approve ERS settlements with a second user.
`docs/PIC_Spend_Cube_Analysis.md` §7	The spend screens now exist (it says the frontend is a separate pass).
Technical Document	`requisition.approved` is not consumed by the order and sourcing services; it notifies the requester, and the buyer workbench reads approved lines directly.